

Verification Continuum™

FSDB Writer

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	10
Synopsys Statement on Inclusivity and Diversity	10

1. FSDB Writer	12
Overview	12
Bit Size and Bytes Per Bit	14
Write Design Information	14
Idcode Scheme	14
Handle Scheme	15
Write Value Change	16
Signal Based Scheme	17
Time Based Scheme	18
Glitch and Glitch Number	19
Sequence of Value Changes	20
Duplicate Value Change	21
Value Change Memory Limit	21
General Writing Steps	21
Open an FSDB	22
Set Scale Unit and Other Information	22
Choose Tree Creation Scheme	22
Initialize Data Type Creation If Necessary	22
Create the Design Hierarchy	22
Create the Value Changes	23
Close the FSDB	23
Memory Dumping	23
Transaction Recording	25
Code Examples	26
Diagram of Basic Concept	57
Diagram of Tree Creation	57
Diagram of Value Change Creation	58

2. Function List	59
ffw_AddBusParameter	65
ffw_AddBusStream	66
ffw_AddRelation	67
ffw_AddSupportInfo	68
ffw_BeginTransaction	69
ffw_BeginTree	70
ffw_Close	71
ffw_CreateABBusValueByHandle	71
ffw_CreateABBusValueByIdcode	72
ffw_CreateArrayBegin	73
ffw_CreateArrayEnd	73
ffw_CreateAttr	74
ffw_CreateAttr2	76
ffw_CreateBus	77
ffw_CreateBusBegin	78
ffw_CreateBusElementByName	79
ffw_CreateBusEnd	79
ffw_CreateEnumAttr	80
ffw_CreateEnumAttr2	81
ffw_CreateFixedPointVarByHandle	82
ffw_CreateFixedPointVarByIdcode	83
ffw_CreateFsdbLog	84
ffw_CreateLoopMarkerVarByVarHdl	84
ffw_CreateLoopMarkerVarByVarIdcode	85
ffw_CreateLoopMarkerVarValueByHandle	86
ffw_CreateLoopMarkerVarValueByIdcode	87
ffw_CreateMDAVarValueByHandle	87
ffw_CreateMDAVarValueByIdcode	89
ffw_CreateMemoryRangeVarByHandle	90
ffw_CreateMemoryRangeVarByIdcode	91
ffw_CreateMemoryRangeVarValueByHandle	92

Contents

ffw_CreateMemoryRangeVarValueByIdcode	93
ffw_CreateMessage	94
ffw_CreatePeriodicVarValueByHandle	95
ffw_CreatePeriodicVarValueIdcode	96
ffw_CreatePropVarByHandle	97
ffw_CreatePropVarByIdcode	100
ffw_CreatePropVarValueByHandle	101
ffw_CreatePropVarValueByIdcode	102
ffw_CreateRecordBegin	104
ffw_CreateRecordEnd	104
ffw_CreateRelation	105
ffw_CreateScope	106
ffw_CreateScopeByPath	106
ffw_CreateStream	107
ffw_CreateStructBegin	108
ffw_CreateStructEnd	109
ffw_CreateTreeByHandleScheme	110
ffw_CreateTreeByIdcodeScheme	110
ffw_CreateTriggerPointVarByVarHdl	111
ffw_CreateTriggerPointVarByVarIdcode	111
ffw_CreateTriggerPointVarValueByHandle	113
ffw_CreateTriggerPointVarValueByIdcode	113
ffw_CreateUpscope	114
ffw_CreateVarByHandle	114
ffw_CreateVarByHandleAndPath	116
ffw_CreateVarByIdcode	118
ffw_CreateVarByIdcodeAndPath	119
ffw_CreateVarValueByHandle	121
ffw_CreateVarValueByIdcode	122
ffw_CreateVarValueByVarMapId	122
ffw_CreateXCoorByHnL	123
ffw_Destroy	124

Contents

ffw_DisableDumpOff	124
ffw_DisableSyncTree	124
ffw_DTCreateEnum	125
ffw_DTCreateEnum2	126
ffw_DTCreateEnum3	127
ffw_DTCreateFloating	128
ffw_DTCreateInt	129
ffw_DTCreateIntHnL	129
ffw_DTCreatePhysicalBegin	130
ffw_DTCreatePhysicalEnd	131
ffw_DTCreatePhysicalSecondaryUnit	131
ffw_DumpSeqNumTurnOn	132
ffw_DumpStrengthTurnOn	132
ffw_EnableDumpOff	133
ffw_EnableSyncTree	133
ffw_EndTransaction	134
ffw_EndTree	135
ffw_GetActiveObject	135
ffw_GetActiveObjectFileName	136
ffw_GetAttrByHdl	136
ffw_GetDataTypeCreationReady	137
ffw_GetFileName	137
ffw_GetFsdbDumperVersion	138
ffw_GetFsdbVersion	138
ffw_GetGlitchNum	139
ffw_GetIdcodeByHandle	139
ffw_GetIdent	140
ffw_GetMaxVarIdcode	140
ffw_GetMaxXTag	141
ffw_GetMinXTag	142
ffw_GetRefCount	142
ffw_GetStreamByHdl	143

Contents

ffw_HasGlitchNumBeenSet	144
ffw_InfoSuppress	144
ffw_Init	145
ffw_IsDumpOff	145
ffw_Open	146
ffw_OpenAutoSwitchFile	146
ffw_RecordTransactionAttribute	147
ffw_RegisterAutoSwitchFileClientData	148
ffw_RegisterDumperCBFunc	149
ffw_RegisterErrorCBFunc	150
ffw_RegisterInfoCBFunc	151
ffw_RegisterTwoGCBFunc	151
ffw_RegisterTwoGCBFuncDontCareLFS	152
ffw_RegisterWarnCBFunc	153
ffw_RegisterWriteErrorCB	153
ffw_ResetXCoor	154
ffw_SetAnnotation	154
ffw_SetAsErrorTransaction	155
ffw_SetBusClock	156
ffw_SetCompressionMethod	157
ffw_SetDesignType	157
ffw_SetDumpLimit	158
ffw_SetGlitchNum	158
ffw_SetLifoDumpLimit	159
ffw_SetLockingScheme	160
ffw_SetMemLimit	160
ffw_SetProducer	161
ffw_SetScaleUnit	161
ffw_SetScopeSeparator	162
ffw_SetSimulationDate	162
ffw_SetSimulatorVersion	163
ffw_SetTransactionColorById	163

Contents

ffw_SetTransactionColorByRGB	166
ffw_SetVCMemLimit	167
ffw_SwitchFile	167
ffw_SwitchFileAndCreatelnitVal	168
ffw_Sync	169
ffw_SyncControlTurnOff	169
ffw_SyncControlTurnOn	170
ffw_WarnSuppress	170
ffwCreateEvcdVarValByVarHdl	170
ffwCreateEvcdVarValByVarIdcode	171
ffwCreateMDAVarByHandle	172
ffwCreateMDAVarByIdcode	173
ffwCreatePropVarByHandle2	176
ffwCreatePropVarByIdcode2	177
ffwCreateScope	178
ffwCreateScope2	179
ffwCreateScopeByPath	180
ffwCreateStream2	182
ffwCreateStrValByVarHdl	182
ffwCreateStrValByVarIdcode	183
ffwCreateStrVarByVarHdl	184
ffwCreateStrVarByVarIdcode	185
ffwCreateVarByVarHdl	185
ffwCreateVarByVarHdl2	187
ffwCreateVarByVarHdlAndPath	188
ffwCreateVarByVarHdlAndPath2	189
ffwCreateVarByVarIdcode	191
ffwCreateVarByVarIdcode2	192
ffwCreateVarByVarIdcodeAndPath	193
ffwCreateVarByVarIdcodeAndPath2	194
ffwGetVarIdcodeByVarHdl	196
ffwOpenFile	196

ffwOpenFile2 197

3. FAQs 199

Preface

The FSDB Writer provides detailed information about the FSDB Writer and its function list. It also provides information regarding some of frequently-asked questions.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also

Contents

includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

FSDB Writer

This chapter contains the following topics:

- [Overview](#)
- [Bit Size and Bytes Per Bit](#)
- [Write Design Information](#)
- [Write Value Change](#)
- [Glitch and Glitch Number](#)
- [Sequence of Value Changes](#)
- [Duplicate Value Change](#)
- [Value Change Memory Limit](#)
- [General Writing Steps](#)
- [Memory Dumping](#)
- [Transaction Recording](#)
- [Code Examples](#)

Overview

An FSDB file contains three types of information: design hierarchies, user-defined data types, and value changes. Design hierarchies include the hierarchies between each scope (design entity) and the variables that each scope holds. User-defined data types include enumeration type, physical type, integer type, and so on. VHDL and SystemVerilog drive the capability to have user-defined data types. Value changes are the waveform data from the simulator. Each value change includes a time and a value. To write information into FSDB, the FSDB Writer provides the "creation mode" concept. The FSDB Writer supports two types of creation modes, which are tree creation mode and value change creation mode. Under the tree creation mode, the application calls FSDB Writer APIs to create design hierarchies and user-defined data types. Under the value change creation mode, the application creates waveform data. The application can switch between these two modes at any time. However, FSDB Writer usually enters tree creation mode first

because some signals exist. Then the FSDB Writer enters the value change mode. The tree creation mode begins from FSDB Writer as "begin tree call" and ends in FSDB Writer as "end tree call". After this, the mode is switched to value change creation mode.

The FSDB Writer gives each unique variable an idcode, and uses it to identify the unique variable. The idcode of a variable is the key data for most FSDB Reader APIs. Under tree creation mode, the application must choose a scheme it likes to use to create the trees. The FSDB Writer supports two schemes for tree creation: idcode scheme and handle scheme. In the idcode scheme, the application generates an unique idcode for each unique variable. It ensures that all these idcodes are positive integer and continuous from the smallest to the largest one.

It is recommended to let the application to easily determine the number of unique variables in the FSDB file. In the handle scheme, the application keeps the handle of a variable and the FSDB Writer generates the idcode. The application chooses the handle of a variable. Usually, the handle is a pointer to an in-core object. As each object in the memory has its own virtual address, a unique variable is represented by a unique virtual address. If tree creation uses the handle scheme, then the FSDB Writer maps each unique variable handle to a unique idcode through its hash mechanism. A variable may have more than one name, it can be called as "alias." For example, a variable whose idcode is 3, it is called "bus" in scope "top," and "data_bus" in scope "system." But both "bus" and "data_bus" refer to the same variable whose idcode is 3. The application can create an alias to a variable by calling variable creation API with the different name but the same handle. The default scheme of tree creation is idcode scheme. It is highly recommended to create trees by handle scheme, because it is much simple and easy for the application.

An FSDB file may contain none, one, or multiple design hierarchies. The design hierarchy is like a multi-tree. A node is corresponding to a scope of the design hierarchy. A scope can contain scopes and variables. The capability that a scope can contain scopes recursively builds the design hierarchy. In order to traverse the design hierarchy, we must introduce the "current scope" concept. Earlier, we mention that the design hierarchy is like a multi-tree; each node is like a scope. The "current scope" means "which node we are at now." Some of the tree creation APIs can move the "current scope" to another one so that the application can describe and build the design hierarchy. The "current scope" also determines which scope the newly created variables belong to: if a variable is created, then it belongs to the "current scope".

The FSDB Writer supports two schemes for value change creation: signal based scheme and time based scheme. By signal based scheme, the application traverses and creates all the value changes of a variable, then rewinds the time, performs the same steps on another variable. By time based scheme, the application stops at a specific time where has value changes occurred, then it figures out what variables have value changes at that specific time. Finally, it creates the value changes of those variables at that specific time. The same steps repeat until it reaches the maximum time of the simulation run. Although the FSDB Writer supports the above two schemes, we highly recommend the time-based

scheme. The reason is that when the application reads the FSDB file, the partial load functionality does not work for those FSDB files created by signal-based scheme.

For some special cases, the application likes to know the sequence of the value changes occurred at the same time. We call this information as “sequence number” of value change. By default, the FSDB Writer does not record the sequence number of value change. The application must call specific API to notify the FSDB Writer to record this information. When this capability is activated, the FSDB Writer starts a counter to record sequence number. The counter is reset to one when a time is created, and incremented by one when a value change is created. Note that this information will contribute a certain amount of data to FSDB file, hence the size of the FSDB file is possible to be much larger than the original one (the one that is without this information).

Bit Size and Bytes Per Bit

Before we start the details of writing design information, we need to explain the bit size and bytes per bit defined by FSDB. FSDB uses bit size to determine if a variable is a scalar or bus signal. If the bit size is 1, then the variable is a scalar signal. If the bit size is greater than 1, then the variable is a bus signal. FSDB calculates the bit size by the formula: $ABS(lbitnum - rbitnum) + 1$. Where ABS means absolute value, lbitnum means left bit number (the most significant bit index), and rbitnum means right bit number (the least significant bit index). FSDB uses bytes per bit to determine that one bit is represented by how many bytes. If there are less than 257 values in the value set, then we can use one byte to represent one bit since one byte can represent 256 values. If there are less than 65537 but more than 256 values in the value set, then we have to use two bytes to represent one bit.

The bit size and bytes per bit information are very important to value change creation. When the application creates one value change of a variable, the application passes an address indicating where the value is to the FSDB Writer. The FSDB Writer calculates how many bytes it should copy from that address by the formula: bit size * how many bytes to represent one bit. For example, if a variable's bit size is defined as 2, bytes per bit as 4 (one bit is represented by 4 bytes), then the FSDB Writer will copy 8 bytes from that address. Inadequate variable's definition may bring trouble, waste memory and contribute extra size to FSDB file, hence the application should define the variable carefully and properly.

Write Design Information

Idcode Scheme

To write design information by idcode scheme, the application takes the responsibility to make a unique idcode for each unique variable. The idcode is better to start from one so

that the maximum idcode implies how many unique variables there are in the FSDB file. Here is the sample code to demonstrate how to use idcode scheme:

```
struct Signal {
    char *name;
    int lbitnum;
    int rbitnum;
    fsdbVarIdcode idcode;
};
static struct Signal var_arr[2] = { {"cpu_var", 0, 0, 1}, {"ram_var", 7,
0, 2} };
ffwObject *obj;
obj = ffw_Open("test.fsdb", FSDB_FT_VERILOG);
ffw_BeginTree(obj);
ffw_CreateScope(obj, "system", FSDB_ST_VCD_MODULE);
ffw_CreateScope(obj, "cpu", FSDB_ST_VCD_MODULE);
ffw_CreateVarByIdcode(obj, FSDB_VT_VCD_WIRE, FSDB_VD_IMPLICIT,
(void*)FSDB_DT_VERILOG_STANDARD, var_arr[0].lbitnum,
var_arr[0].rbitnum,
var_arr[0].idcode, var_arr[0].name, FSDB_BYTES_PER_BIT_1B);
ffw_CreateUpscope(obj);
ffw_CreateScope(obj, "ram", FSDB_ST_VCD_MODULE);
ffw_CreateVarByIdcode(obj, FSDB_VT_VCD_WIRE, FSDB_VD_IMPLICIT,
(void*)FSDB_DT_VERILOG_STANDARD, var_arr[1].lbitnum,
var_arr[1].rbitnum,
var_arr[1].idcode, var_arr[1].name, FSDB_BYTES_PER_BIT_1B);
ffw_EndTree(obj);
```

The above sample code creates a root scope named "system," which has two sub-scopes named "cpu" and "ram" respectively. Scope "cpu" has a variable named "cpu_var," and its idcode is 1; bit size (bit width) is also 1. Scope "ram" has a variable named "ram_var," and its idcode is 2, bit size is 8.

Handle Scheme

To write design information by handle scheme, the application leaves the making idcode job to the FSDB Writer. What it has to do is to keep variable's handle, that is, a pointer to a specific in-core object. Here is the sample code to demonstrate how to use handle scheme:

```
struct Signal {
    char *name;
    int lbitnum;
    int rbitnum;
};
static struct Signal cpu = { {"cpu_var", 0, 0} };
static struct Signal ram = { {"ram_var", 7, 0} };
ffwObject *obj;
obj = ffw_Open("test.fsdb", FSDB_FT_VERILOG);
ffw_CreateTreeByHandleScheme(obj);
```

```
ffw_BeginTree(obj);  
ffw_CreateScope(obj, "system", FSDB_ST_VCD_MODULE);  
ffw_CreateScope(obj, "cpu", FSDB_ST_VCD_MODULE);  
ffw_CreateVarByHandle(obj, FSDB_VT_VCD_WIRE, FSDB_VD_IMPLICIT,  
    (void*)FSDB_DT_VERILOG_STANDARD, cpu.lbitnum, cpu.rbitnum,  
    (void*) &cpu, cpu.name, FSDB_BYTES_PER_BIT_1B);  
ffw_CreateUpscope(obj);  
ffw_CreateScope(obj, "ram", FSDB_ST_VCD_MODULE);  
ffw_CreateVarByHandle(obj, FSDB_VT_VCD_WIRE, FSDB_VD_IMPLICIT,  
    (void*)FSDB_DT_VERILOG_STANDARD, ram.lbitnum, ram.rbitnum,  
    (void*) &ram, ram.name, FSDB_BYTES_PER_BIT_1B);  
ffw_EndTree(obj);
```

Instead of giving each unique variable an idcode (such as 1 and 2), now you take the address of the variable object as the variable's handle (such as &cpu and &ram) to identify the unique variable. The job for making an idcode for each unique variable is left to the FSDB Writer. Note that the idcode scheme is the default scheme to create trees, so the application must notify the FSDB Writer that it prefers to use handle scheme. Otherwise, some necessary data structures of handle scheme are not allocated and initialized. This notification is done by calling **ffw_CreateTreeByHandleScheme**.

The pros and cons between idcode scheme and handle scheme are:

- Idcode Scheme:

This scheme is faster. However, you must be sure that your minimum idcode is `FSDB_MIN_VAR_IDCODE` and increases by 1 thereafter. In addition, you must know which signal mapped to which idcode. That is why this scheme is easier than handle scheme.

- Handle Scheme:

This scheme is easier to use since you can use the address of a signal as its handle and it does not have to keep track of which signal mapped to which idcode. However, the FSDB Writer does the handle -> idcode mapping and therefore this scheme is less efficient than idcode scheme.

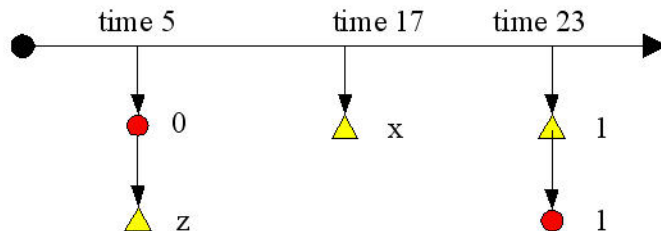
If you care about performance or if you already encode the signals just as your idcode scheme does, you should use idcode scheme. On the other hand, if the performance is not your priority (actually, the impact on performance is generally little) or if you do not want to handle the trouble of encoding the signals as the idcode scheme does, handle scheme should be a better choice.

Write Value Change

In order to explain value change creation clearly, a scenario is depicted in the [Figure 1](#). There are two variables, one is denoted by circle, the other is denoted by triangle. The circle variable has value changes at time 5 and 23 while the triangle variable has value

changes at time 5, 17, and 23. Both variables are scalar type. And each variable's bytes per bit is one byte.

Figure 1 Write Value Change



Based on this figure, signal-based and time-based value change creations are explained in the following sections:

Signal Based Scheme

To create value change by signal based scheme, the application traverses and creates all the value changes of a variable. Then it rewinds the time and performs the same routines on another variable. Here is the sample code to demonstrate signal-based value change creation:

```
struct MySignal _sig;
fsdbTag64 xtag;
unsigned char val;

xtag.H = 0; xtag.L = 5;
ffw_CreateXCoorByHnL(obj, xtag.H, xtag.L);
val = (unsigned char) FSDB_BT_VCD_0;
ffw_CreateVarValueByHandle(obj, (void*)&_sig, &val);
xtag.L = 23;
ffw_CreateXCoorByHnL(obj, xtag.H, xtag.L);
val = (unsigned char) FSDB_BT_VCD_1;
ffw_CreateVarValueByHandle(obj, (void*)&_sig, &val);
ffw_ResetXCoor(obj);

xtag.H = 0; xtag.L = 5;
ffw_CreateXCoorByHnL(obj, xtag.H, xtag.L);
val = (unsigned char) FSDB_BT_VCD_Z;
ffw_CreateVarValueByHandle(obj, (void*)&_sig, &val);
xtag.L = 17;
ffw_CreateXCoorByHnL(obj, xtag.H, xtag.L);
val = (unsigned char) FSDB_BT_VCD_X;
ffw_CreateVarValueByHandle(obj, (void*)&_sig, &val);
xtag.L = 23;
ffw_CreateXCoorByHnL(obj, xtag.H, xtag.L);
```

```
val = (unsigned char) FSDB_BT_VCD_1;  
ffw_CreateVarValueByHandle(obj, (void*)&_sig, &val);
```

The above sample code traverses and creates all the value changes of the circle variable (time 5: value 0, time 32: value 1), then it rewinds the time (by calling **ffw_ResetXCoor**). After that, similar steps are performed on the triangle variable. The application should notice that once all the value changes of a variable have been created, the time must be rewind.

However, if there is no special reason, then we do not recommend using signal-based scheme in the application. One reason is that partial load functionality (FSDB Reader's functionality) does not work for those FSDB files created by signal based scheme.

Time Based Scheme

To create value change by time-based scheme, the application stops at a specific time where value changes occurred. Then it figures out what variables have value changes at that specific time. Finally, it creates the value changes of those variables at that specific time. The same steps repeat until it reaches the maximum time of the simulation run. Here is the sample code to demonstrate time-based value change creation:

```
struct MySignal _sig;  
fsdbTag64 xtag;  
unsigned char val;  
  
xtag.H = 0; xtag.L = 5;  
ffw_CreateXCoorByHnL(obj, xtag.H, xtag.L);  
val = (unsigned char) FSDB_BT_VCD_0;  
ffw_CreateVarValueByHandle(obj, (void*)&_sig, &val);  
val = (unsigned char) FSDB_BT_VCD_Z;  
ffw_CreateVarValueByHandle(obj, (void*)&_sig, &val);  
  
xtag.L = 17;  
ffw_CreateXCoorByHnL(obj, xtag.H, xtag.L);  
val = (unsigned char) FSDB_BT_VCD_X;  
ffw_CreateVarValueByHandle(obj, (void*)&_sig, &val);  
  
xtag.L = 23;  
ffw_CreateXCoorByHnL(obj, xtag.H, xtag.L);  
val = (unsigned char) FSDB_BT_VCD_1;  
ffw_CreateVarValueByHandle(obj, (void*)&_sig, &val);  
ffw_CreateVarValueByHandle(obj, (void*)&_sig, &val);
```

In the above sample code, the application finds out time 5 has value changes, then it creates an xtag at time 5. After that, it figures out that both the circle and triangle variables have value changes at that time, so it creates their value changes. The application goes further to find out the next time where has value changes. Therefore, it finds time 17, creates an xtag at time 17, then it figures out that the triangle variable has one value change, but circle variable does not. So it just creates the triangle variable's value change

at that time. The application keeps going further, creates an xtag at time 23, creates the triangle and circle variables' value changes at that time.

Glitch and Glitch Number

If a variable has more than one different value changes at a specific time, then we say it has glitch at that specific time. In order to reduce the extra data brought by glitch, the FSDB Writer provides two ways to determine which glitch should be stored.

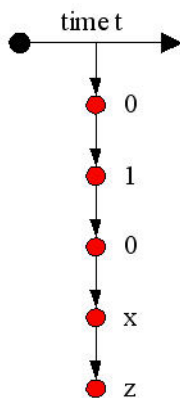
- API: `ffw_SetGlitchNum`
- Environment Variable: `FSDB_ENV_MAX_GLITCH_NUM`

Note:

The maximum glitch number for the FSDB Writer is 255. Another words, the argument "number" in `ffw_SetGlitchNum` (`ffwObject *obj`, `ushort_T number`) cannot exceed 255.

Take [Figure 2](#) as an example. A variable has five value changes: 0, 1, 0, x, and z, at time t. If the application sets glitch number as zero, then all of the five value changes are stored into FSDB. If the application sets glitch number as one, then the last value change is stored into FSDB. If the application sets glitch number as n ($n \geq 2$), then the first (n-1) value change(s) and the last one are stored into FSDB.

Figure 2 Glitch Number



If glitch number is 0, then (0,1,0,x,z) are stored into FSDB.

If glitch number is 1, then (z) is stored into FSDB.

If glitch number is 2, then (0, z) are stored into FSDB.

If glitch number is 3, then (0,1,z) are stored into FSDB.

If glitch number is 4, then (0,1,0,z) are stored into FSDB.

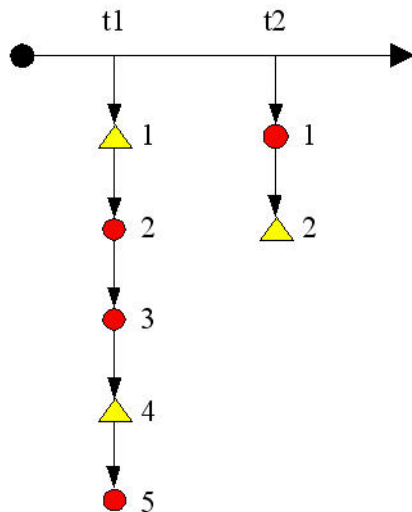
If glitch number is 5, then (0,1,0,x,z) are stored into FSDB.

The FSDB Writer has default value for glitch number, and the default value varies with FSDB version. The current default value can be found in the **ffwAPI.h** header file.

Sequence of Value Changes

In some cases, the application may want to know the sequence of value changes occurred at the same time. Hence the FSDB Writer supports sequence dumping, which records the sequence of value changes occurred at the same time. After the sequence dumping is enabled, the FSDB Writer starts a counter to record sequence number. The counter is set to one when a time (an xtag) is created, it is increment by one after a value change has been created. Take the following figure as an example: there are two signals, denoted by circle and triangle respectively. They all have value changes at time t1 and t2. If sequence dumping is enabled, then sequence of each value change is shown as follows:

Figure 3 Sequence of Value Changes



Since sequence dumping will introduce extra data into file, hence it is not enabled by default. The application must call **ffw_DumpSeqNumTurnOn** to enable sequence dumping.

Duplicate Value Change

For all the value changes of a variable, regardless of the time, as long as the adjacent values are the same, then the second one is a duplicate value change. Duplicate value changes may cascade, so there may be a series of the same value. For most digital signal types, the FSDB Writer automatically filters the duplicate value changes. For analog signal types, the FSDB Writer does not filter duplicate value changes. For example, if the signal is of memory range type or voltage type, the FSDB Writer does not filter its duplicate value changes. If the signal is of digital type, and it has value *z* at time 5, 15, 20, and 25, then the FSDB Writer filters the value at time 15, 20, and 25, and only keeps the value change “*z*” occurred at time 5.

Value Change Memory Limit

When a value change has been created, the FSDB Writer keeps it in memory. If certain amount of value changes has been accumulated, the FSDB Writer flushes all the value changes into a file, and then recycles the memory. This operation is called a “flush session,” while the amount of value changes is the so-called “value change memory limit”. The flush session is usually performed by the FSDB Writer automatically, but in some cases, the application may take control. Calling **ffw_Sync** API forces the FSDB Writer to perform the flush session. The value change memory limit can be altered by calling the **ffw_SetVCMemLimit** API or setting the environment variable *FSDB_ENV_WRITER_MEM_LIMIT*.

Based on the design size and VC count, the writer memory limit will be increased automatically at run time if the run time is not specified. The increase helps to reduce FSDB size and speed up dumping and FSDB reading. A message (for example. “For performance reasons, the Memory Size Limit has been increased to 128M.”) will be shown when the size is changed.

General Writing Steps

- Open an FSDB
- Set scale unit and other information
- Choose tree creation scheme
- Initialize data type creation if necessary
- Create the design hierarchy
- Create the value changes
- Close the FSDB

Open an FSDB

An FSDB is opened by the **ffwOpenFile2** API, which asks the application to give the file name and file type. The file type is to tell the source of the information, and it is defined as `fsdbFileType` in the **fsdbShr.h** header file. After the file is opened successfully, the application can get a pointer to an FSDB Writer object, which is a must parameter for most FSDB Writer APIs.

Set Scale Unit and Other Information

If the simulation does have scale unit, then the application must set it. But if it does not, then the default value is 1ns. The other information, such as simulation date and simulator version, are optional and for reference only. The APIs to set scale unit, simulation date and simulator version are as follows:

- **ffw_SetScaleUnit**
- **ffw_SetSimulationDate**
- **ffw_SetSimulatorVersion**

Choose Tree Creation Scheme

Tree creation scheme is chosen by calling the **ffw_CreateTreeByIdcodeScheme** or **ffw_CreateTreeByHandleScheme** API. The default is idcode scheme if none of them is called.

Initialize Data Type Creation If Necessary

If there is user-defined data type, then the application must call data type creation API to notify the FSDB Writer, so that it can initialize necessary data structures to store data type definition. This can be done by calling the **ffw_GetDataTypeCreationReady** API. For most cases, there is no user-defined data type. Therefore, by default, the FSDB Writer assumes there is no user-defined data type.

Create the Design Hierarchy

The design hierarchy is composed by calling tree creation APIs. The **ffw_BeginTree** API creates a top scope, which has no name. The **ffw_CreateScope2** API creates a scope, which is a child of “the current scope” and then moves “the current scope” down to the newly created one. The **ffw_CreateUpscope** moves “the current scope” up to its parent scope. The **ffwCreateVarByIdcode2** or the **ffwCreateVarByHandle2** API creates a variable, which belongs to “the current scope.” The **ffw_EndTree** API completes

the design hierarchy. Note that an FSDB file may contain none, one, or multiple design hierarchies.

Create the Value Changes

Conceptually, a value change is composed by a pair: a time and a value. The time is created by calling **ffw_CreateXCoorByHnL**, while the value is created by calling the **ffw_CreateVarValByIdcode** or **ffw_CreateVarValByHandle** API.

Close the FSDB

An FSDB is closed by **ffw_Close**, which flushes necessary in-core data and temporary files to the FSDB file. Then it performs some clean up tasks and completes the FSDB file.

Memory Dumping

Memory is a two-dimensional viewpoint of a group of value changes. It comprises two kinds of variables: memory range variable and memory variable. A memory range variable indicates the boundary of memory, which is defined by the name of the variable. A memory variable indicates each element within the memory. For example, to define a block of memory consisting of 4 words of 8 bits each, you could specify the name of the memory range variable as

```
my_mem[0:3]
and the name of the memory variables as
my_mem[0]
my_mem[1]
my_mem[2]
my_mem[3]
```

In the memory range variable `my_mem[0:3]`, `my_mem` means the name of the elements within the memory, 0 denotes the index of the first element, and 3 indicates the index of the last element. In this way, you can tell that `my_mem[0:3]` represents a block of memory which consists of `my_mem[0]`, `my_mem[1]`, `my_mem[2]` and `my_mem[3]`.

After the memory-related variables are created, it's time to create some value changes. While the name of a memory range variable suggests the indexes of the first and last elements, its value change represents the indexes of the first and last elements that has a value change at current xtag. For example, if `my_mem[1]` and `my_mem[3]` each has a value change at time 10 while the values of the other two remain the same, you could dump them using the following code:

```
ffw_CreateXCoorByHnL(ffw_obj, 0, 10);
ffw_CreateMemoryRangeVarValueByHandle(ffw_obj, &my_mem_range, 1, 2);
ffw_CreateVarValueByHandle(ffw_obj, &my_mem1, my_mem1_value);
ffw_CreateVarValueByHandle(ffw_obj, &my_mem2, my_mem2_value);
```

With the above concept in mind, the following is a sample code demonstrating how to dump a block of memory with FSDB Writer APIs:

```
//
// Variable definition
//
struct MemoryRangeSignal {
    char          *name;
    fsdbVarType    type;
};

struct BusSignal {
    char          *name;           // signal name
    fsdbVarType    type;           // signal type
    ushort_T      lbitnum;         // signal left bit number
    ushort_T      rbitnum;         // signal right bit
                                number
    fsdbBytesPerBit bpb;           // signal bytes per bit
};

static MemoryRangeSignal my_mem_range =
    { "my_mem[0:3]", FSDB_VT_VCD_MEMORY_RANGE };

static BusSignal my_mem[] = {
    {"my_mem[0]", FSDB_VT_VCD_MEMORY, 0, 7, FSDB_BYTES_PER_BIT_1B},
    {"my_mem[1]", FSDB_VT_VCD_MEMORY, 0, 7, FSDB_BYTES_PER_BIT_1B},
    {"my_mem[2]", FSDB_VT_VCD_MEMORY, 0, 7, FSDB_BYTES_PER_BIT_1B},
    {"my_mem[3]", FSDB_VT_VCD_MEMORY, 0, 7, FSDB_BYTES_PER_BIT_1B}
};

//
// Variable creation
//
ffw_CreateMemoryRangeVarByHandle(ffw_obj,
    (void*)&my_mem_range, my_mem_range.type, my_mem_range.name);
for (i = 0; i < 4; i++) {
    ffw_CreateVarByHandle(ffw_obj, my_mem[i].type,
        FSDB_VD_INPUT, FSDB_DT_HANDLE_VERILOG_STANDARD,
        my_mem[i].lbitnum, my_mem[i].rbitnum,
        (void*)(my_mem + i), my_mem[i].name, my_mem[i].bpb);
}

//
// Value change creation
//

fsdbTag64    time;
byte_T       value[8];

//
// Value change creation at (0,0)
//
memset((void*)&time, 0, sizeof(fsdbTag64));
```

```
memset((void*)value, FSDB_BT_VCD_X, 8);

ffw_CreateXCoorByHnL(ffw_obj, time.H, time.L);

ffw_CreateMemoryRangeVarValueByHandle(ffw_obj, &my_mem_range, 0, 3);
for (i = 0; i < 4; i++)
    ffw_CreateVarValueByHandle(ffw_obj, (void*)(my_mem + i), value);

//
// Value change creation at (0, 10)
//
memset((void*)value, FSDB_BT_VCD_1, 8);
time.L += 10;
ffw_CreateXCoorByHnL(ffw_obj, time.H, time.L);

ffw_CreateMemoryRangeVarValueByHandle(ffw_obj, &my_mem_range, 1, 3);
ffw_CreateVarValueByHandle(ffw_obj, (void*)(my_mem + 1), value);
ffw_CreateVarValueByHandle(ffw_obj, (void*)(my_mem + 3), value);
```

Transaction Recording

Streams hold transactions. Each transaction consists of a set of attributes and is independent of one another. That is, there is no such concept as "transaction type", as in SCV (SystemC Verification), even if the sets of attributes that constitute two transactions are the same one.

When you create a transaction, you must follow the steps below:

1. create a stream
2. create attributes
3. create transaction

Related APIs are

```
ffw_CreateStream2()
ffw_CreateAttr2()
ffw_BeginTransaction()
ffw_EndTransaction()
ffw_RecordTransactionAttribute()
ffw_GetAttrByHdl()
```

For detailed description, see the Function List chapter.

Note:

The APIs in this section are only for the previous generation waveform-based transaction debug capability (also known as Classic Transaction). For transaction level FSDB dumping, the recommendation is to use the new transaction debug platform that includes new APIs.

Code Examples

The examples in this section are given demonstrating how to use the Writer APIs. First of all, the makefile **ffwTest.makefile** needs to be modified. The following settings in the makefile should be reset before compiling these examples at your environment:

```
include ./COMPILER_DEF
include ./FILE_LIST
INC_DIR = ../../../../ffwAPI
API_LIB = ../../../../lib/libnffw$(POLY_LIB)
```

Suppose these examples run at the Solaris2 platform. The environment settings in the makefile are modified as the following:

```
include <$NOVAS_HOME$>/share/FsdbWriter/SOL2/COMPILER_DEF
include <$NOVAS_HOME$>/share/FsdbWriter/FILE_LIST
INC_DIR = <$NOVAS_HOME$>/share/FsdbWriter
API_LIB = <$NOVAS_HOME$>/share/FsdbWriter/SOL2/libnffw.a
```

stree.cpp

```
//
// Program Name : stree.cpp
//
// Descriptions : Demonstrate how to define a bunch of vars.
//

#ifdef NOVAS_FSDB
#undef NOVAS_FSDB
#endif

#include "stdio.h"
#include "string.h"
#include "stdlib.h"
#include "ffwAPI.h"

static char *scope_name_arr[] = {
    "A", "B", "C", "D", "E", "F", "G", NULL
};

int
main(int argc, char *argv[])
{
    uint_T var_count_per_scope = 10, i, j;
```

```

char *env_ptr, var_name[256];
fsdbVarDir var_dir;
fsdbVarType var_type;
ffwVarMapId vm_id;
ffwObject *fsdb_obj;

env_ptr = getenv("VAR_COUNT_PER_SCOPE");
if (NULL != env_ptr)
    var_count_per_scope = atoi(env_ptr);

sprintf(var_name, "stree%u.fsdb", var_count_per_scope);

//
// Open the fsdb file.
//
fsdb_obj = ffw_Open(var_name, FSDB_FT_VERILOG);
if (NULL == fsdb_obj) {
    fprintf(stdout, "failed to create a fsdb file.\n");
    exit(FSDB_RC_FAILURE);
}

//
// application can add annotation to fsdb, max length of
// annotation is FSDB_ANNOTATION_LEN(512 bytes)
//
ffw_SetAnnotation(fsdb_obj,
"annotation: demonstrate tree creation.\n");

//
// application can tell the version of written fsdb by
// calling ffw_GetFsdbVersion().
//
fprintf(stdout, "fsdb version: %s\n", ffw_GetFsdbVersion());

//
// set the scale unit used for simulation
//
ffw_SetScaleUnit(fsdb_obj, "0.01n");

//
// We highly recommend application to create tree by handle
// scheme instead of idcode shceme. By handle scheme, fsdb
// writer takes care of making an idcode for each unique var,
// hence alleviates the burden of application. What
// application should keep is the var handles, each unique var
// handle used to create a var is corresponding to a unique
// idcode in fsdb, which represents a unique var.
//
// A unique var may have more than one names, like a man may
// have a formal name and a nick name, but formal and nick
// name refer to the same man, not different men.
//

```

```
ffw_CreateTreeByHandleScheme(fsdb_obj);

//
// ffw_BeginTree() tells fsdb writer that we are going to
// define a bunch of vars(signals).
//
ffw_BeginTree(fsdb_obj);

for (i = 0; NULL != scope_name_arr[i]; i++) {

    if (3 == i || 6 == i)
        ffw_CreateUpscope(fsdb_obj);
    //
    // current scope goes up one level
    //
    if (4 == i) {
        ffw_CreateUpscope(fsdb_obj);
        //
        // current scope goes up one level
        //

        ffw_CreateUpscope(fsdb_obj);
        //
        // current scope goes up one level
        //
    }

    if (0 == (i & 0x01))
        ffw_CreateScope(fsdb_obj,
            FSDB_ST_VCD_MODULE, scope_name_arr[i]);
    //
    // create a subscope under current scope, and then
    // current
    // scope goes down one level(goes to the newly created
    // scope level)
    //
    else
        ffw_CreateScope(fsdb_obj,
            FSDB_ST_VCD_TASK, scope_name_arr[i]);
    //
    // create a subscope under current scope, and then
    // current
    // scope goes down one level(goes to the newly created
    // scope level)
    //

    for (j = 0; j < var_count_per_scope; j++) {
        sprintf(var_name, "%s%u", scope_name_arr[i], j);
        var_dir = (fsdbVarDir) (j & 0x03);
        var_type = (fsdbVarType) (j & 0x0f);

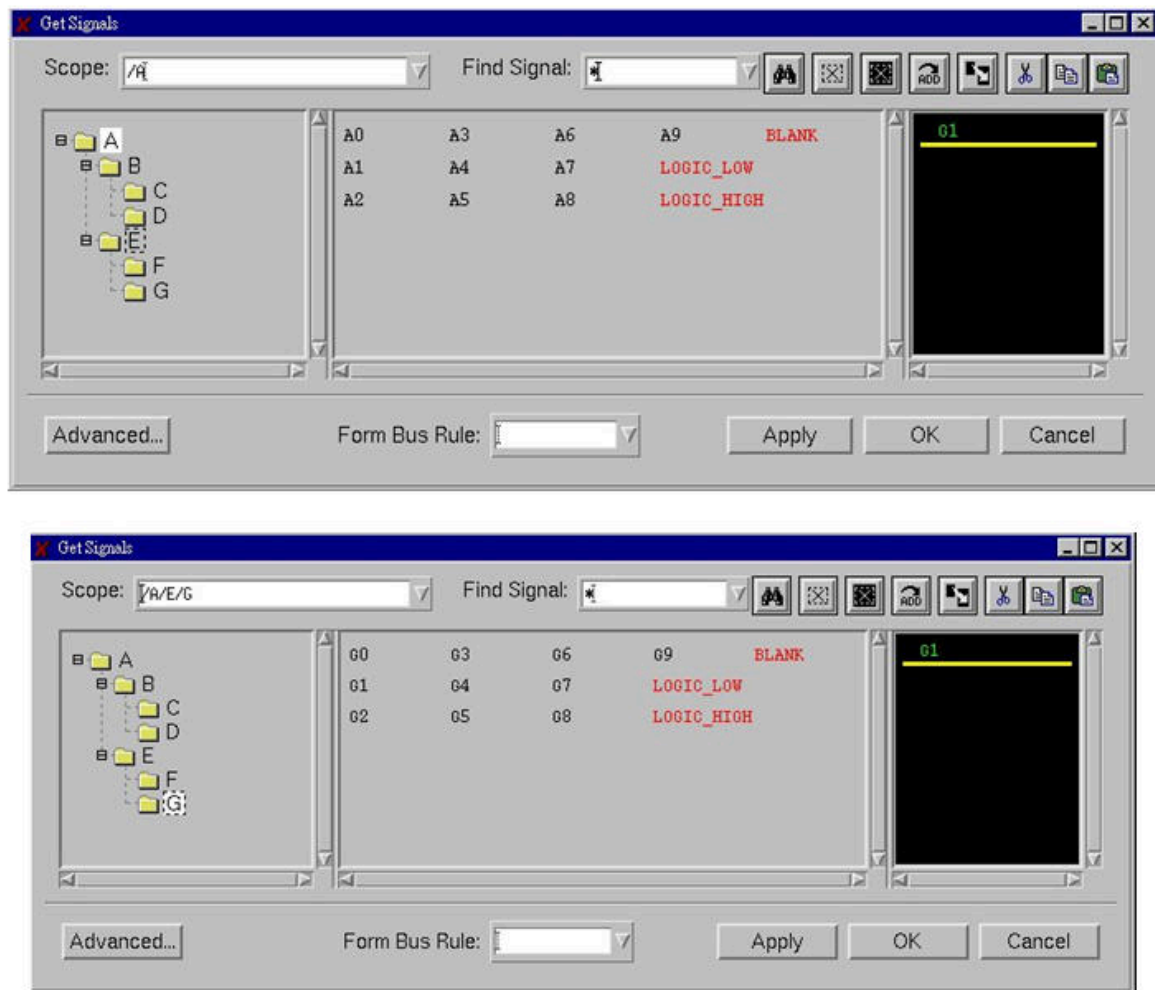
        //
        // (void*)(i*var_count_per_scope + j) is to make sure that
```

```
// each var handle is different, so that each call of
// ffw_CreateVarByHandle() will create a unique var in fsdb.
//
    vm_id = ffw_CreateVarByHandle(fsdb_obj, var_type,
        var_dir, FSDB_DT_HANDLE_VERILOG_STANDARD,
        0, 0, (void*)(i*var_count_per_scope + j),
        var_name, FSDB_BYTES_PER_BIT_1B);
    if (NULL == vm_id) {
        printf("failed to create a var(%s)\n", var_name);
        exit(FSDB_RC_FAILURE);
    }
}

//
// ffw_EndTree() tells fsdb writer that we have finished
// vars' definitions.
//
ffw_EndTree(fsdb_obj);

ffw_Close(fsdb_obj);
return 0;
}
```

Figure 4 The Results of stree.cpp Shown on nWave



vc.cpp

```
//
// Program Name : vc.cpp
//
// Descriptions : Demonstrate how to create value changes.
//

#ifdef NOVAS_FSDB
#undef NOVAS_FSDB
#endif

#include "stdio.h"
#include "string.h"
#include "stdlib.h"
```

```
#include "ffwAPI.h"

struct DigitalSignal {
    char *name; // signal name
    fsdbVarType type; // signal type
    ushort_T lbitnum; // signal left bit number
    ushort_T rbitnum; // signal right bit number
    fsdbBytesPerBit bpb; // signal bytes per bit
    byte_T value; // signal value
};
typedef struct DigitalSignal DigitalSignal;

struct AnalogSignal {
    char *name; // signal name
    fsdbVarType type; // signal type
    ushort_T lbitnum; // signal left bit number
    ushort_T rbitnum; // signal right bit number
    fsdbBytesPerBit bpb;
    byte_T *value; // signal value
};
typedef struct AnalogSignal AnalogSignal;

static DigitalSignal alpha =
{ "alpha", FSDB_VT_NANOSIM_LOGIC, 0, 0, FSDB_BYTES_PER_BIT_1B,
  (byte_T)0};

static DigitalSignal beta =
{ "beta", FSDB_VT_NANOSIM_LOGIC, 0, 0, FSDB_BYTES_PER_BIT_1B, (byte_T)0};

//
// Since the value change of voltage is represented by float value,
// hence the bytes per bit is 4 bytes(FSDB_BYTES_PER_BIT_4B).
//
static AnalogSignal voltage =
{ "voltage(analog)", FSDB_VT_NANOSIM_VOLTAGE, 0, 0,
  FSDB_BYTES_PER_BIT_4B, NULL};

//
// Since the value change of power is represented by double value,
// hence the bytes per bit is 8 bytes(FSDB_BYTES_PER_BIT_8B).
//
static AnalogSignal power =
{ "power(analog)", FSDB_VT_NANOSIM_POWER, 0, 0,
  FSDB_BYTES_PER_BIT_8B, NULL};

#define ARR_DIM(a) (sizeof(a)/sizeof(a[0]))
#define ABS(A) ((A)<(0) ? (-A)):(A))

static char*
AllocateMemory(ushort_T bit_size, fsdbBytesPerBit bpb);

int
```

```
main(int argc, char *argv[])
{
    uint_T vc_count = 36;
    uint_T idx;
    char env_ptr;
    ffwObject *fsdb_obj;
    float oltag_val;
    double power_val;
    char fname[256];

    env_ptr = getenv("VC_COUNT");
    if (NULL != env_ptr)
        vc_count = atoi(env_ptr);

    sprintf(fname, "vc%u.fsdb", vc_count);

    //
    // Open the fsdb file.
    //
    fsdb_obj = ffw_Open(fname, FSDB_FT_NANOSIM);

    //
    // Create tree by handle scheme. Default is idcode scheme.
    //
    // We highly recommend application to create tree by handle
    // scheme instead of idcode scheme. By handle scheme, fsdb
    // writer takes care of making an idcode for each unique var,
    // hence alleviates the burden of application. What
    // application should keep is the var handles, each unique var
    // handle used to create a var is corresponding to a unique
    // idcode in fsdb, which represents a unique var.
    //
    // A unique var may have more than one names, like a man may
    // have a formal name and a nick name, but formal and nick
    // name refer to the same man, not different men.
    //
    ffw_CreateTreeByHandleScheme(fsdb_obj);

    //
    // Set the scale unit for simulation.
    //
    ffw_SetScaleUnit(fsdb_obj, "0.01n");

    //
    // ffw_BeginTree tells fsdb writer that we are going to
    // define a bunch of vars.
    //
    ffw_BeginTree(fsdb_obj); // current scope is root

    //
    // Create a subscope named "Top" under root scope, and then
    // current scope goes down to "Top" scope.
    //
```

```
ffw_CreateScope(fsdb_obj, 0, "Top");

//
// Create a subscope named "Digital" under current
// scope("Top"), and then current scope goes down to
// "Top.Digital".
//
ffw_CreateScope(fsdb_obj, 0, "Digital");

//
// Create a var named "alpha" under current
// scope("Top.Digital").
//
// < About the Var Handle >
//
// var handle is chosen by application, as long as it can be
// distinguished from different vars.
//
// Here we choose the virtual address of object as the var
// handle, such as (void*)&alpha, since different object will
// have different virtual address, it's a good way to take
// this advantage.
//
if (NULL == ffw_CreateVarByHandle(fsdb_obj, alpha.type,
    FSDB_VD_IMPLICIT, FSDB_DT_HANDLE_NANOSIM_DIGITAL,
    alpha.lbitnum, alpha.rbitnum, (void*)&alpha,
    alpha.name, (fsdbBytesPerBit)alpha.bpb)) {
    fprintf(stderr, "failed to create a var(%s)\n",
        alpha.name);
    exit(FSDB_RC_FAILURE);
}

//
// Create a var named "beta" under current scope("Top.Digital").
//
if (NULL == ffw_CreateVarByHandle(fsdb_obj, beta.type,
    FSDB_VD_IMPLICIT, FSDB_DT_HANDLE_NANOSIM_DIGITAL,
    beta.lbitnum, beta.rbitnum, &beta,
    beta.name, (fsdbBytesPerBit)beta.bpb)) {
    fprintf(stderr, "failed to create a var(%s)\n",
        beta.name);
    exit(FSDB_RC_FAILURE);
}

//
// Create an alias name called "my_beta" for the "beta" var.
// Since the var handle is the same as the one in previous
// var creation(both are (void*)&beta), so fsdb writer knows
// these two var creations refer to the same unique var.
//
if (NULL == ffw_CreateVarByHandle(fsdb_obj, beta.type,
    FSDB_VD_IMPLICIT, FSDB_DT_HANDLE_NANOSIM_DIGITAL,
```

```

        beta.lbitnum, beta.rbitnum, &beta,
        "my_beta", (fsdbBytesPerBit)beta.bpb)) {
    fprintf(stderr,
        "failed to create an alias name for the var(%s)\n",
        beta.name);
    exit(FSDB_RC_FAILURE);
}

//
// Make current scope goes up one level, so current scope will be
// "Top".
//
ffw_CreateUpscope(fsdb_obj);

//
// Create a subscope named "Analog" under current
// scope("Top"), and current scope goes to "Top.Analog".
//
ffw_CreateScope(fsdb_obj, 0, "Analog");

//
// Create a var under current scope ("Top.Analog").
//
if (NULL == ffw_CreateVarByHandle(fsdb_obj, voltage.type,
    FSDB_VD_IMPLICIT, FSDB_DT_HANDLE_NANOSIM_ANALOG,
    voltage.lbitnum, voltage.rbitnum, &voltage,
    voltage.name, (fsdbBytesPerBit)voltage.bpb)) {
    fprintf(stderr, "failed to create a var(%s)\n",
        voltage.name);
    exit(FSDB_RC_FAILURE);
}

//
// Allocate enough memory space to hold one value change of
// the var.
//
voltage.value = (byte_T*)
    AllocateMemory(ABS(voltage.lbitnum - voltage.rbitnum) + 1,
        voltage.bpb);

//
// set up the initial value change for the var.
//
voltag_val = 0.0000;
memcpy((void*)voltage.value, (void*)&voltag_val,
    sizeof(float));

//
// Create a var under current scope ("Top.Analog").
//
if (NULL == ffw_CreateVarByHandle(fsdb_obj, power.type,
    FSDB_VD_IMPLICIT, FSDB_DT_HANDLE_NANOSIM_ANALOG,
    power.lbitnum, power.rbitnum, &power,

```

```

        power.name, (fsdbBytesPerBit)power.bpb)) {
        fprintf(stderr, "failed to create a var(%s)\n",
            power.name);
        exit(FSDB_RC_FAILURE);
    }

    //
    // Allocate enough memory space to hold one value change of
    // the var.
    //
    power.value = (byte_T*)
        AllocateMemory(ABS(power.lbitnum - power.rbitnum) + 1,
            power.bpb);
    //
    // set up the initial value change for the var.
    //
    power_val = 0.000000000;
    memcpy((void*)power.value, (void*)&power_val,
        sizeof(double));

    //
    // Make current scope goes up one level, so current scope is
    // "Top".
    //
    ffw_CreateUpscope(fsdb_obj);

    //
    // Make current scope goes up one level, so current scope is
    // root.
    //
    ffw_CreateUpscope(fsdb_obj);

    //
    // ffw_EndTree() tells fsdb writer that we have finished
    // vars' definitions.
    //
    ffw_EndTree(fsdb_obj);

    //
    // value change creation
    //
    // fsdb writer supports time based and signal based value
    // change creation, while we highly recommend application to
    // use time based scheme instead of signal based scheme; this
    // is due to that value changes of signal-based fsdb cannot
    // be partially loaded.
    //
    // Using time based scheme, application creates a time(xtag)
    // first, and then create all the value changes at that time.
    //
    fsdbTag64 time;

```

```

time.H = 0;
time.L = 0;
ffw_CreateXCoorByHnL(fsdb_obj, time.H, time.L);

//
// Create all initial value changes.
//
ffw_CreateVarValueByHandle(fsdb_obj, (fsdbVarHandle)&alpha,
    (byte_T*)&alpha.value);
ffw_CreateVarValueByHandle(fsdb_obj, (fsdbVarHandle)&beta,
    (byte_T*)&beta.value);
ffw_CreateVarValueByHandle(fsdb_obj, (fsdbVarHandle)&voltage,
    (byte_T*)voltage.value);
ffw_CreateVarValueByHandle(fsdb_obj, (fsdbVarHandle)&power,
    (byte_T*)power.value);

for (idx = 1; idx < vc_count; idx++) {
    time.L += idx;
    ffw_CreateXCoorByHnL(fsdb_obj, time.H, time.L);

//
// Make alpha value change be in the range of [0, 7].
//
    alpha.value = idx & 0x07;

//
// Make beta value change be in the range of [1, 8].
//
    beta.value = alpha.value + 1;

    voltag_val += 0.5678;
    memcpy((void*)voltage.value, (void*)&voltag_val,
        sizeof(float));

    power_val += 0.12345678;
    memcpy((void*)power.value, (void*)&power_val,
        sizeof(double));

    ffw_CreateVarValueByHandle(fsdb_obj,
        (fsdbVarHandle)&alpha,
        (byte_T*)&alpha.value);
    ffw_CreateVarValueByHandle(fsdb_obj,
        (fsdbVarHandle)&beta,
        (byte_T*)&beta.value);
    ffw_CreateVarValueByHandle(fsdb_obj,
        (fsdbVarHandle)&voltage,
        (byte_T*)voltage.value);
    ffw_CreateVarValueByHandle(fsdb_obj,
        (fsdbVarHandle)&power,
        (byte_T*)power.value);
}

//

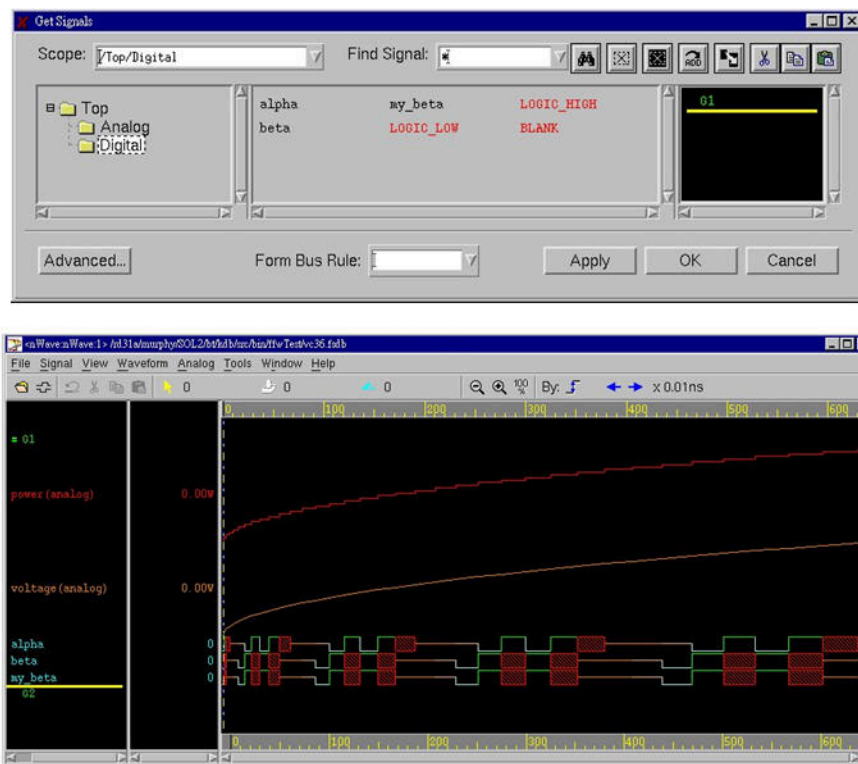
```

```
        // Close the fsdb file.
        //
        ffw_Close(fsdb_obj);
        return 0;
    }

static char*
AllocateMemory(ushort_T bit_size, fsdbBytesPerBit bpb)
{
    uint_T byte_count;
    char    *ptr;

    switch(bpb) {
    case FSDB_BYTES_PER_BIT_1B:
        byte_count = 1*bit_size;
        break;
    case FSDB_BYTES_PER_BIT_2B:
        byte_count = 2*bit_size;
        break;
    case FSDB_BYTES_PER_BIT_4B:
        byte_count = 4*bit_size;
        break;
    case FSDB_BYTES_PER_BIT_8B:
        byte_count = 8*bit_size;
        break;
    default:
        printf("unknown bpb.\n");
        exit(-1);
    }
    ptr = (char*) calloc(1, byte_count);
    if(NULL == ptr) {
        printf("Memory resources exhausted.\n");
        exit(-1);
    }
    return ptr;
}
```

Figure 5 The Results of vc.cpp Shown on nWave



traffic.cpp

```
//
// Program Name: traffic.c
//
// Descriptions : demonstrate how to create user defined data
// type and dump the value changes whose data type is user
// defined.
//

#ifdef NOVAS_FSDB
#undef NOVAS_FSDB
#endif

#include "stdio.h"
#include "string.h"
#include "stdlib.h"
#include "limits.h"
#include "ffwAPI.h"

#define ARR_DIM(a) sizeof(a)/sizeof(a[0])

#define GREEN_LIGHT_DURATION 30
```

```
#define TRIANGLE_LIGHT_DURATION 15
#define CIRCLE_LIGHT_DURATION 30
#define RUNNING_DURATION 180

struct GeneralSignal {
    char *name; // signal name
    fsdbVarType type; // signal type
    ushort_T lbitnum; // signal left bit number
    ushort_T rbitnum; // signal right bit number
    fsdbBytesPerBit bpb; // signal bytes per bit
};
typedef struct GeneralSignal GeneralSignal;

static GeneralSignal traffic_light =
{"traffic_light", FSDB_VT_VHDL_SIGNAL, 0, 0, FSDB_BYTES_PER_BIT_1B};

static GeneralSignal driver =
{"driver", FSDB_VT_VHDL_SIGNAL, 0, 0, FSDB_BYTES_PER_BIT_1B};

static char *traffic_light_literal_array[] =
{ "Green", "Triangle", "Circle"};

enum TrafficLightLiteralIndex {
    TL_GREEN = 0,
    TL_TRIANGLE = 1,
    TL_CIRCLE = 2
};
typedef enum TrafficLightLiteralIndex TrafficLightLiteralIndex;

static char *driver_literal_array[] =
{ "Pass", "Accelerate", "Stop"};

enum DriverLiteralIndex {
    DRVR_PASS = 0,
    DRVR_ACCELERATE = 1,
    DRVR_STOP = 2
};
typedef enum DriverLiteralIndex DriverLiteralIndex;

#define TRAFFIC_LIGHT_DATA_TYPE_HDL (INT_MAX + 1)
#define DRIVER_DATA_TYPE_HDL (INT_MAX + 2)

int
main(int argc, char *argv[])
{
    ffwVarMapId var_map_id;
    ffwObject *fsdb_obj;
    char *env_ptr;

    //
    // open the fsdb file, since we are going to create
    // user defined data type, so we set the file type as
    // FSDB_FT_VHDL. Note that verilog does not have user
```

```
// created data type, fsdb writer assumes verilog to have
// 2 basic data types, one is "byte" for storing 0, 1, x
// and z; the other is "double" for storing real value
// change.
//
fsdb_obj = ffw_Open("traffic.fsdb", FSDB_FT_VHDL);
if (NULL == fsdb_obj) {
    printf("fail to open a fast file object!\n");
    exit(0);
}

//
// For 3rd parties, we encourage and recommend them to
// create tree by handle scheme instead of idcode scheme;
// if using handle scheme, fsdb writer will take of many
// affairs for 3rd parties, such as making the idcode for
// each unique signal.
//
ffw_CreateTreeByHandleScheme(fsdb_obj);

//
// set up the scale unit for simulation.
//
ffw_SetScaleUnit(fsdb_obj, "1s");

//
// If 3rd parties would like to create user defined data
// type, then it must tell fsdb writer by calling the
// following API so that fsdb writer can get some extra data
// strucutres ready to handle user defined data type.
//
ffw_GetDataTypeCreationReady(fsdb_obj);

//
// Tell fsdb writer that we are ready to define a bunch of
// vars.
// Current scope is root.
//
ffw_BeginTree(fsdb_obj);

//
// Create a scope which is a subscope of current scope, then
// current scope goes down to the created scope; hence
// current scope will be "City" after this call.
//
ffw_CreateScope(fsdb_obj, FSDB_ST_VHDL_ARCHITECTURE, "City");

//
// Create a scope which is a subscope of current scope, then
// current scope goes down to the created scope; hence
// current scope will be "City.Intersection" after this call.
//
```

```

ffw_CreateScope(fsdb_obj,
FSDB_ST_VHDL_ARCHITECTURE, "Intersection");

//
// When an enum data type is created, fsdb writer writes the
// data type definition into file and returns a data type
// idcode to the caller, indicating the creation is
// successful or not; if it fails, the return data type
// idcode will be FSDB_INVALID_DATA_TYPE_IDCODE, otherwise,
// the return data type idcode will be greater than
// FSDB_DT_MAX_RESERVED_IDCODE(511).
//
if (FSDB_INVALID_DATA_TYPE_IDCODE ==
    ffw_DTCreateEnum(fsdb_obj,
        (void*)TRAFFIC_LIGHT_DATA_TYPE_HDL,
        ARR_DIM(traffic_light_literal_array),
        traffic_light_literal_array)) {
    fprintf(stderr, "failed to create traffic light data
        type.\n");
    exit(FSDB_RC_FAILURE);
}

//
// Create an enum data type.
//
if (FSDB_INVALID_DATA_TYPE_IDCODE ==
    ffw_DTCreateEnum(fsdb_obj,
        (void*)DRIVER_DATA_TYPE_HDL,
        ARR_DIM(driver_literal_array),
        driver_literal_array)) {
    fprintf(stderr, "failed to create driver data type.\n");
    exit(FSDB_RC_FAILURE);
}

//
// When a var is created, fsdb writer writes the var
// definition
// to file, binds the relationship between the var and its
// data type definition, and makes an idcode associated with
// the var. fsdb writer takes the var handle as an
// identification to tell different vars; if two vars'
// creation calls have the same var handle but different var
// name, then fsdb writer can tell these two var definitions
// are indeed talking about the same var, not different vars;
// like a man may have more than one name(formal name, nick
// name ... etc), and all those names refer to the same man,
// not different men.
//
var_map_id = ffw_CreateVarByHandle(fsdb_obj,
    traffic_light.type, FSDB_VD_IMPLICIT,
    (void*)TRAFFIC_LIGHT_DATA_TYPE_HDL,
    traffic_light.lbitnum, traffic_light.rbitnum,
    (void*) &traffic_light,

```

```

        traffic_light.name, traffic_light.bpb);
if (NULL == var_map_id) {
    fprintf(stderr, "failed to create a var(%s)\n",
        traffic_light.name);
    exit(FSDB_RC_FAILURE);
}

var_map_id = ffw_CreateVarByHandle(fsdb_obj,
    driver.type, FSDB_VD_IMPLICIT,
    (void*)DRIVER_DATA_TYPE_HDL,
    driver.lbitnum, driver.rbitnum,
    (void*) &driver, driver.name, driver.bpb);
if (NULL == var_map_id) {
    fprintf(stderr, "failed to create a var(%s)\n",
        driver.name);
    exit(FSDB_RC_FAILURE);
}

//
// Tell fsdb writer that we have finished the vars'
// definitions.
//
ffw_EndTree(fsdb_obj);

uint_T green_light_duration = GREEN_LIGHT_DURATION;
uint_T triangle_light_duration = TRIANGLE_LIGHT_DURATION;
uint_T circle_light_duration = CIRCLE_LIGHT_DURATION;

env_ptr = getenv("ENV_GREEN_LIGHT_DURATION");
if (NULL != env_ptr)
    green_light_duration = atoi(env_ptr);

env_ptr = getenv("ENV_TRIANGLE_LIGHT_DURATION");
if (NULL != env_ptr)
    triangle_light_duration = atoi(env_ptr);

env_ptr = getenv("ENV_CIRCLE_LIGHT_DURATION");
if (NULL != env_ptr)
    circle_light_duration = atoi(env_ptr);

//
// Begin to create var value.
//
fsdbTag64 time;
TrafficLightLiteralIndex current_light;
DriverLiteralIndex what_driver_should_do;
byte_T vc;

time.H = 0;
time.L = 0;
current_light = TL_GREEN;
what_driver_should_do = DRVR_PASS;

```

```

    for ( ; time.L < RUNNING_DURATION; ) {
        ffw_CreateXCoorByHnL(fsdb_obj, time.H, time.L);

        //
        // We have declared the bytes per bit of "traffic_light"
        // and "driver" as one byte(FSDB_BYTES_PER_BIT_1B), it means
        // that each bit of the value change of them is represented by
        // one byte, hence fsdb writer will based on the bit size and
        // bytes per bit to decide how many bytes it should copy from
        // the byte array presented as a parameter in var value
        // creation API.
        //
        // < Be careful to pass the byte array >
        //
        // ffw_CreateVarValueByHandle(fsdb_obj,
        // (void*)&traffic_light, (byte_T*)&current_light);
        //
        // Due to little endian/big endian, the above code may or
        // may not work; based on the bit size(1) and bytes per bit
        // (1B) of traffic_light, fsdb writer will copy only one byte
        // from the byte array, if the endian type is big and
        // current_light is "1", then fsdb writer will copy the value
        // "0" since the first byte of (byte_T*)&current_light is "0",
        // not the value "1", the value "1" is at the 4th byte of
        // (byte_T*)&current_light.
        //
        // 3rd parties or applications must be careful to fill in
        // values into the byte array correctly.
        //
        vc = (byte_T)current_light;
        ffw_CreateVarValueByHandle(fsdb_obj,
            (void*)&traffic_light, (byte_T*)&vc);
        vc = (byte_T)what_driver_should_do;
        ffw_CreateVarValueByHandle(fsdb_obj,
            (void*)&driver, (byte_T*)&vc);

        switch (current_light) {
        case TL_GREEN:
            time.L += GREEN_LIGHT_DURATION;
            current_light = TL_TRIANGLE;
            what_driver_should_do = DRVR_ACCELERATE;
            break;

        case TL_TRIANGLE:
            time.L += TRIANGLE_LIGHT_DURATION;
            current_light = TL_CIRCLE;
            what_driver_should_do = DRVR_STOP;
            break;

        case TL_CIRCLE:
            time.L += CIRCLE_LIGHT_DURATION;
            current_light = TL_GREEN;

```

```

        what_driver_should_do = DRVR_PASS;
        break;

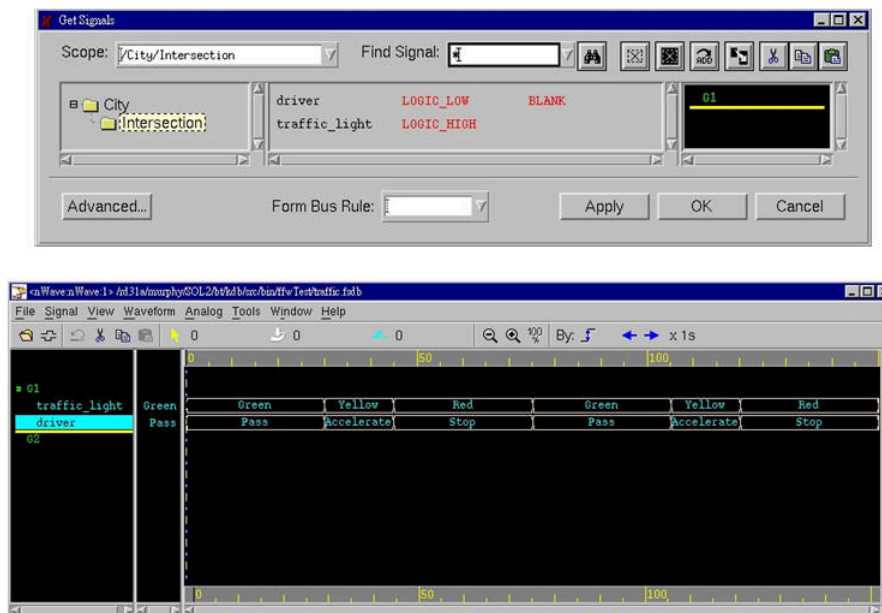
default:
    fprintf(stderr, "should not reach here.\n");
    exit(FSDB_RC_FAILURE);
}

}

//
// close the fsdb file
//
ffw_Close(fsdb_obj);
return 0;
}

```

Figure 6 The Results of traffic.cpp Shown on nWave



transaction_recording.cpp

```

//
// NAME : transaction_recording.cpp
//
// PURPOSE : Demonstrate the use of transaction recording APIs.
//
// DESCRIPTION :
//
// CREATION DATE : 02/12/2004
//

```

```
#ifndef NOVAS_FSDB
    #undef NOVAS_FSDB
#endif

#include "stdio.h"
#include "stdlib.h"
#include "assert.h"
#include "ffwAPI.h"

//
// Constants
//
#ifndef TRUE
    const int TRUE = 1;
#endif

#ifndef FALSE
    const int FALSE = 0;
#endif

//
// Types And Typedefs
//

//
// Global variable definition and declaration
//
fsdbStreamHdl    stream_hdl;
fsdbAttrHdl      attr_hdl1;
fsdbAttrHdl      attr_hdl2;
fsdbAttrHdl      attr_hdl3;
fsdbAttrHdl      attr_hdl4;
fsdbAttrHdl      attr_hdl5;
fsdbAttrHdl      attr_hdl6;
fsdbAttrHdl      attr_hdl7;
fsdbAttrHdl      attr_hdl8;

fsdbVarIdcode    var_idcode = 1;

//
// Function Declaration
//

static void
__TR(void);

static void
__CreateTree(ffwObject* ffw_obj);

static void
__CreateVC(ffwObject* ffw_obj);

static void
```

```

__OpenFsdbFileForWrite(ffwObject **fsdb_obj, char fname[],
    fsdbFileType file_type);

//
// Main Program
//
int
main(int argc, char *argv[])
{
    ffwObject    *ffw_obj = NULL;
    char         fname[] = "transaction.fsdb";

    //
    // Write to the fsdb file
    //
    __OpenFsdbFileForWrite(&ffw_obj, fname, FSDB_FT_VERILOG);
    __CreateTree(ffw_obj);
    __CreateVC(ffw_obj);
    ffw_Close(ffw_obj);

    exit(0);
}

//
// __CreateTree()
//
void
__CreateTree(ffwObject* ffw_obj)
{
    static char var_name[100];

    printf("==== Creating tree =====\n");

    //
    // Call ffw_GetDataTypeCreationReady() and prepare to create
    // transaction attributes.
    //
    ffw_GetDataTypeCreationReady(ffw_obj);

    //
    // Create tree
    //
    ffw_BeginTree(ffw_obj);
    ffw_CreateScope(ffw_obj, FSDB_ST_VCD_MODULE, "Top");

    sprintf(var_name, "stream1");
    stream_hdl = ffw_CreateStream(ffw_obj, var_name);

    attr_hdl1 =
        ffw_CreateAttr(ffw_obj, "logic_attr1",
            FSDB_ATTR_DT_LOGIC, 0, 0, FALSE);

    attr_hdl2 =

```

```

        ffw_CreateAttr(ffw_obj, "bool_attr1", FSDB_ATTR_DT_BOOL,
        0, 1, TRUE);

attr_hdl3 =
    ffw_CreateAttr(ffw_obj, "char_attr1", FSDB_ATTR_DT_CHAR,
    0, 21, FALSE);

attr_hdl4 =
    ffw_CreateAttr(ffw_obj, "string_attr0",
    FSDB_ATTR_DT_STRING, 0, 1, TRUE);

attr_hdl5 =
    ffw_CreateAttr(ffw_obj, "int32_attr0",
    FSDB_ATTR_DT_INT32, 0, 0, FALSE);

attr_hdl6 =
    ffw_CreateAttr(ffw_obj,
    "uint32_attr0", FSDB_ATTR_DT_UINT32, 0, 1, FALSE);

attr_hdl7 =
    ffw_CreateAttr(ffw_obj, "int64_attr0",
    FSDB_ATTR_DT_INT64, 0, 0, TRUE);

attr_hdl8 =
    ffw_CreateAttr(ffw_obj, "uint64_attr0",
    FSDB_ATTR_DT_UINT64, 0, 1, FALSE);

//
// Tree created
//
ffw_EndTree(ffw_obj);
printf("Tree created.\n");
}

//
// __CreateVC()
//
void
__CreateVC(ffwObject* ffw_obj)
{
    fsdbTag64      xtag;
    fsdbXTag       time;
    fsdbAttrHdlVal attr_val0[1];
    fsdbAttrHdlVal attr_val1[2];
    fsdbAttrHdlVal attr_val2[2];
    fsdbAttrHdlVal attr_val3[1];
    fsdbAttrHdlVal attr_val4[1];
    fsdbAttrHdlVal attr_val5[2];
    fsdbAttrHdlVal attr_val6[3];
    uint_T         attr_count[] = {1, 2, 2, 1, 1, 2, 3};

    byte_T         logic_val[] = { FSDB_BT_TRANS_LOGIC_0 };
    byte_T         bool_val[] = {

```

```

                                FSDB_BT_TRANS_BOOL_FALSE,
                                FSDB_BT_TRANS_BOOL_TRUE };
byte_T      char_val[] = {
    'c', 'h', 'a', 'r', ' ', 't', 'e', 's', 't', ' ',
    'e', 'm', 'p', 't', 'y', '-', '>', ' ', 'e', 'n',
    'd', 0};
int          int_val[] = { -100 };
unsigned int  uint_val[] = { 200, 5000 };
char         *str_val[2] = { "string1", "string2" };
long long    int64_val[] = { -35269887898779LL };
unsigned long long uint64_val[3] = { 300,
                                     134759009482493293ULL };

fsdbTransId   trans_id1,trans_id2,trans_id3, trans_id4,
               trans_id5, trans_id6, trans_id7;

attr_val0[0].hdl   = attr_hdl1;
attr_val0[0].value = logic_val;

attr_val1[0].hdl   = attr_hdl2;
attr_val1[0].value = bool_val;
attr_val1[1].hdl   = attr_hdl3;
attr_val1[1].value = char_val;

attr_val2[0].hdl   = attr_hdl4;
attr_val2[0].value = (byte_T*)str_val;
attr_val2[1].hdl   = attr_hdl5;
attr_val2[1].value = (byte_T*)int_val;

attr_val3[0].hdl   = attr_hdl6;
attr_val3[0].value = (byte_T*)uint_val;

attr_val4[0].hdl   = attr_hdl7;
attr_val4[0].value = (byte_T*)int64_val;

attr_val5[0].hdl   = attr_hdl8;
attr_val5[0].value = (byte_T*)uint64_val;
attr_val5[1].hdl   = attr_hdl1;
attr_val5[1].value = logic_val;

attr_val6[0].hdl   = attr_hdl2;
attr_val6[0].value = bool_val;
attr_val6[1].hdl   = attr_hdl3;
attr_val6[1].value = char_val;
attr_val6[2].hdl   = attr_hdl4;
attr_val6[2].value = (byte_T*)str_val;

//
//  10  20          40  50          60  70  80    90    100
//  ----          ----          ----  ----
//  | 1 |          | 3 |          | 4 |    | 6 |
//  ----          ----          ----  ----
//      15      30          55          85

```

```
//      -----      -      -----
//      |  2  |      5      |  7
//      -----      -      -----
//
xtag.H = 0;
xtag.L = 10;
ffw_CreateXCoorByHnL(ffw_obj, xtag.H, xtag.L);
printf("Create xtag at (%d, %d).\n", xtag.H, xtag.L);
time.hltag.H = 0;
time.hltag.L = 10;
trans_id1 = ffw_BeginTransaction(ffw_obj,
    stream_hdl, time, "trans 1",
    NULL, 0);
if (FSDB_INVALID_TRANS_ID == trans_id1) {
    fprintf(stderr, "trans 1 fails during begin!\n");
}

xtag.H = 0;
xtag.L = 20;
ffw_CreateXCoorByHnL(ffw_obj, xtag.H, xtag.L);
printf("Create xtag at (%d, %d).\n", xtag.H, xtag.L);
time.hltag.H = 0;
time.hltag.L = 20;
if (FSDB_RC_SUCCESS !=
    ffw_EndTransaction(ffw_obj, trans_id1, time, attr_val2,
    attr_count[2])) {
    fprintf(stderr, "trans 1 fails during end!\n");
}

//
// Begin transaction in the past
//
time.hltag.H = 0;
time.hltag.L = 15;
trans_id2 = ffw_BeginTransaction(ffw_obj,
    stream_hdl, time, "trans 2",
    attr_val0, attr_count[0]);
if (FSDB_INVALID_TRANS_ID == trans_id2) {
    fprintf(stderr, "trans 2 fails during begin!\n");
}

xtag.H = 0;
xtag.L = 30;
ffw_CreateXCoorByHnL(ffw_obj, xtag.H, xtag.L);
printf("Create xtag at (%d, %d).\n", xtag.H, xtag.L);
time.hltag.H = 0;
time.hltag.L = 30;

if (FSDB_RC_SUCCESS !=
    ffw_EndTransaction(ffw_obj, trans_id2, time, NULL, 0)) {
    fprintf(stderr, "trans 2 fails during end!\n");
}
```

```

xtag.H = 0;
xtag.L = 40;
ffw_CreateXCoorByHnL(ffw_obj, xtag.H, xtag.L);
printf("Create xtag at (%d, %d).\n", xtag.H, xtag.L);
time.hltag.H = 0;
time.hltag.L = 40;
trans_id3 = ffw_BeginTransaction(ffw_obj,
    stream_hdl, time, "trans 3",
    attr_val3, attr_count[3]);
if (FSDB_INVALID_TRANS_ID == trans_id3) {
    fprintf(stderr, "trans 3 fails during begin!\n");
}

xtag.H = 0;
xtag.L = 50;
ffw_CreateXCoorByHnL(ffw_obj, xtag.H, xtag.L);
time.hltag.H = 0;
time.hltag.L = 50;
if (FSDB_RC_SUCCESS !=
    ffw_EndTransaction(ffw_obj, trans_id3, time, NULL, 0)) {
    fprintf(stderr, "trans 2 fails during end!\n");
}

xtag.H = 0;
xtag.L = 60;
ffw_CreateXCoorByHnL(ffw_obj, xtag.H, xtag.L);
printf("Create xtag at (%d, %d).\n", xtag.H, xtag.L);
time.hltag.H = xtag.H;
time.hltag.L = xtag.L;
trans_id4 = ffw_BeginTransaction(ffw_obj,
    stream_hdl, time, "trans 4",
    attr_val5, attr_count[5]);
if (FSDB_INVALID_TRANS_ID == trans_id4) {
    fprintf(stderr, "trans 4 fails during begin!\n");
}

if (FSDB_RC_SUCCESS !=
    ffw_RecordTransactionAttribute(ffw_obj, trans_id4,
    attr_val1, attr_count[1])) {
    fprintf(stderr, "trans 4 fails during special!\n");
}

//
// Begin transaction in the past
//
time.hltag.H = 0;
time.hltag.L = 55;
trans_id5 = ffw_BeginTransaction(ffw_obj,
    stream_hdl, time, "trans 5",
    attr_val3, attr_count[3]);
if (FSDB_INVALID_TRANS_ID == trans_id5) {
    fprintf(stderr, "trans 5 fails during begin!\n");
}

```

```

    if (FSDB_RC_SUCCESS !=
        ffw_EndTransaction(ffw_obj, trans_id5, time, NULL, 0)) {
        fprintf(stderr, "trans 5 fails during end!\n");
    }

    xtag.H = 0;
    xtag.L = 70;
    ffw_CreateXCoorByHnL(ffw_obj, xtag.H, xtag.L);
    printf("Create xtag at (%d, %d).\n", xtag.H, xtag.L);
    time.hltag.H = 0;
    time.hltag.L = 70;
    if (FSDB_RC_SUCCESS !=
        ffw_EndTransaction(ffw_obj, trans_id4, time, NULL, 0)) {
        fprintf(stderr, "trans 5 fails during end!\n");
    }

    xtag.H = 0;
    xtag.L = 80;
    ffw_CreateXCoorByHnL(ffw_obj, xtag.H, xtag.L);
    printf("Create xtag at (%d, %d).\n", xtag.H, xtag.L);
    time.hltag.H = 0;
    time.hltag.L = 80;
    trans_id6 = ffw_BeginTransaction(ffw_obj,
        stream_hdl, time, "trans 6",
        attr_val4, attr_count[4]);
    if (FSDB_INVALID_TRANS_ID == trans_id6) {
        fprintf(stderr, "trans 6 fails during begin!\n");
    }

    xtag.H = 0;
    xtag.L = 100;
    ffw_CreateXCoorByHnL(ffw_obj, xtag.H, xtag.L);
    printf("Create xtag at (%d, %d).\n", xtag.H, xtag.L);
    time.hltag.H = 0;
    time.hltag.L = 90;
    if (FSDB_RC_SUCCESS !=
        ffw_EndTransaction(ffw_obj, trans_id6, time, NULL, 0)) {
        fprintf(stderr, "trans 6 fails during end!\n");
    }

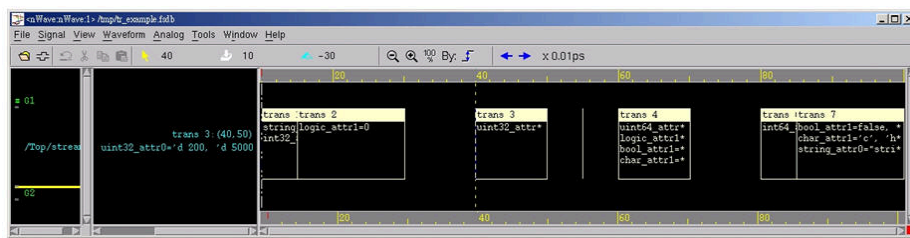
    //
    // Begin transaction in the past
    //
    time.hltag.H = 0;
    time.hltag.L = 85;
    trans_id7 = ffw_BeginTransaction(ffw_obj,
        stream_hdl, time, "trans 7",
        attr_val6, attr_count[6]);
    if (FSDB_INVALID_TRANS_ID == trans_id7) {
        fprintf(stderr, "trans 7 fails during begin!\n");
    }
}

```

```
//
// __OpenFsdbFileForWrite()
//
void
__OpenFsdbFileForWrite(ffwObject **fsdb_obj, char fname[],
    fsdbFileType file_type)
{
    // Open fsdb file
    *fsdb_obj = ffw_Open(fname, file_type);
    if (NULL == *fsdb_obj) {
        printf("Fail to open a fast file object! File name =
            %s.\n", fname);
        exit(FSDB_RC_OBJECT_CREATION_FAILED);
    }

    // Use handle scheme for tree creation
    ffw_CreateTreeByHandleScheme(*fsdb_obj);
    ffw_SetScaleUnit(*fsdb_obj, "0.01p");
}
```

Figure 7 The Results of transaction_recording.cpp Shown on nWave



sv_dt.cpp

```
//
// Program Name : sv_dt.cpp
// Purpose      : Demonstrate System Verilog enum value write/read.
// Descriptions :
// 1. System Verilog enum is modeled as an register instead
//    of an idx that VHDL enum uses.
// 2. An enum value may not have a corresponding literal
//    value in the enum definition.
// 3. Previous API support is not enough
//    ffw_DTCreateEnum2() / ffw_CreateEnum2VarByHandle()
//    ffw_CreateEnum2VarValueByIdcode()
// Do store the enum value in enum table, however its
// vc needs to be first converted to enum idx before saved.
// It also has limitation when value change does not has
// corresponding idx in the enum table.
//
#undef NOVAS_FSDB
```

```
#include "stdio.h"
#include "stdlib.h"
#include "limits.h"
#include "ffwAPI.h"

struct GeneralSignal {
    char          *name;          // signal name
    fsdbVarType    type;          // signal type
    void*         dt;            // signal data type
    ushort_T      lbitnum;       // signal left bit number
    ushort_T      rbitnum;       // signal right bit number
    fsdbVarDir     direction;     // signal direction
    fsdbBytesPerBit bpb;         // signal bytes per bit
};

static GeneralSignal enum_sig = {
    "enum_sig", FSDB_VT_VCD_REG, 0, 0, 3, FSDB_VD_IMPLICIT,
    FSDB_BYTES_PER_BIT_1B
};

#define _ENUM_DT_HANDLE          UINT_MAX

static char *enum_lit_arr[] = { "Red",
                                "Green",
                                "Blue",
                                "Yellow"
                                };

#define _ENUM_VAL_COUNT          7
#define _ENUM_VAL_LEN            4
static unsigned char enum_val_arr[_ENUM_VAL_COUNT][_ENUM_VAL_LEN] = {
    // First 4 value are in the created enum table
    {FSDB_BT_VCD_0, FSDB_BT_VCD_0, FSDB_BT_VCD_0, FSDB_BT_VCD_1},
    // 0001
    {FSDB_BT_VCD_0, FSDB_BT_VCD_0, FSDB_BT_VCD_1, FSDB_BT_VCD_0},
    // 0010
    {FSDB_BT_VCD_0, FSDB_BT_VCD_0, FSDB_BT_VCD_1, FSDB_BT_VCD_1},
    // 0011
    {FSDB_BT_VCD_0, FSDB_BT_VCD_0, FSDB_BT_VCD_1, FSDB_BT_VCD_X},
    // 001X

    // Others value can't be found in the created enum table
    {FSDB_BT_VCD_0, FSDB_BT_VCD_X, FSDB_BT_VCD_X, FSDB_BT_VCD_X},
    // 0XXX
    {FSDB_BT_VCD_0, FSDB_BT_VCD_Z, FSDB_BT_VCD_Z, FSDB_BT_VCD_Z},
    // 0ZZZ
    {FSDB_BT_VCD_1, FSDB_BT_VCD_1, FSDB_BT_VCD_1, FSDB_BT_VCD_1}
    // 1111
};

#define ARR_DIM(a)                (sizeof(a)/sizeof(a[0]))
```

```
static void
__CreateFSDB(ffwObject* ffw_obj);

int
main(int argc, char *argv[])
{
    fsdbRC      rc = FSDB_RC_SUCCESS;
    ffwObject    *obj = NULL;
    char        file_name[] = "enum3.fsdb";

    obj = ffw_Open(file_name, FSDB_FT_VERILOG);
    if (NULL == obj) {
        printf("failed to create ffwObject.\n");
        return FSDB_RC_FAILURE;
    }
    __CreateFSDB(obj);
    ffw_Close(obj);

    return rc;
}

static void
__CreateFSDB(ffwObject *fsdb_obj)
{
    ffwVarMapId  var_map_id;
    ffwVarMap    *ret_vm;

    // Prepare //////////////////////////////////////

    ffw_CreateTreeByHandleScheme(fsdb_obj);
    ffw_SetScaleUnit(fsdb_obj, "1ns");

    ffw_GetDataTypeCreationReady(fsdb_obj);

    ffw_BeginTree(fsdb_obj);

    // Create Data types //////////////////////////////////////
    ffwEnumDTArg  enum_arg;
    enum_arg.size      = sizeof(ffwEnumDTArg);
    enum_arg.data_type_hdl = (void*)_ENUM_DT_HANDLE;
    enum_arg.name       = "enumColor";
    enum_arg.val_type    = FSDB_ENUM_VALUE_TYPE_VERILOG_LOGIC;
    enum_arg.lbitnum     = enum_sig.lbitnum;
    enum_arg.rbitnum     = enum_sig.rbitnum;
    enum_arg.literal_count = ARR_DIM(enum_lit_arr);
    enum_arg.literal_arr = enum_lit_arr;
    enum_arg.val_arr      = &enum_val_arr[0][0];
    if (FSDB_INVALID_DATA_TYPE_IDCODE ==
        ffw_DTCreateEnum3(fsdb_obj, &enum_arg)){
        fprintf(stderr, "failed to create enum data type.\n");
        exit(FSDB_RC_FAILURE);
    }
}
```

```

fprintf(stderr, "Create Signals...\n");

ffw_CreateScope(fsdb_obj, FSDB_ST_VCD_MODULE, "Top");

ffwVarArg2 var_arg;

var_arg.size          = sizeof(ffwVarArg2);
var_arg.type          = enum_sig.type;
var_arg.dir           = enum_sig.direction;
var_arg.data_type_hdl = (void*)_ENUM_DT_HANDLE;
var_arg.lbitnum       = enum_sig.lbitnum;
var_arg.rbitnum       = enum_sig.rbitnum;
var_arg.u_hdl         = (void*) &enum_sig;
var_arg.name          = enum_sig.name;
var_arg.bpb           = enum_sig.bpb;
var_arg.len_descriptor = 0;
var_arg.descriptor     = NULL;
var_arg.client_data_len = 0;
var_arg.client_data    = NULL;

ffwCreateVarByVarHdl2(fsdb_obj, &var_arg, &ret_vm);
if (NULL == ret_vm) {
    fprintf(stderr, "failed to create a var(%s)\n",
            enum_sig.name);
    exit(FSDB_RC_FAILURE);
}

ffw_EndTree(fsdb_obj);

// Create Value ////////////////////////////////////////////

fsdbTag64    time;
char*        tmp_val;
int          byte_count;

byte_count = (abs(enum_sig.lbitnum - enum_sig.rbitnum) + 1) * 1;
tmp_val = (char*) malloc(byte_count);

//
// Create var value (value available in enum table)
//
time.H = 0;
time.L = 0;
ffw_CreateXCoorByHnL(fsdb_obj, time.H, time.L);

time.H = 0;
time.L = 10;
ffw_CreateXCoorByHnL(fsdb_obj, time.H, time.L);
ffw_CreateVarValueByHandle(fsdb_obj, (void*) &enum_sig,
                           &enum_val_arr[0][0]);

time.H = 0;

```

```

time.L = 20;
ffw_CreateXCoorByHnL(fsdb_obj, time.H, time.L);
ffw_CreateVarValueByHandle(fsdb_obj, (void*) &enum_sig,
                           &enum_val_arr[1][0]);

time.H = 0;
time.L = 30;
ffw_CreateXCoorByHnL(fsdb_obj, time.H, time.L);
ffw_CreateVarValueByHandle(fsdb_obj, (void*) &enum_sig,
                           &enum_val_arr[2][0]);

time.H = 0;
time.L = 40;
ffw_CreateXCoorByHnL(fsdb_obj, time.H, time.L);
ffw_CreateVarValueByHandle(fsdb_obj, (void*) &enum_sig,
                           &enum_val_arr[3][0]);

//
// Create var value (value out of enum table)
//
time.H = 0;
time.L = 50;
ffw_CreateXCoorByHnL(fsdb_obj, time.H, time.L);
ffw_CreateVarValueByHandle(fsdb_obj, (void*) &enum_sig,
                           &enum_val_arr[4][0]);

time.H = 0;
time.L = 60;
ffw_CreateXCoorByHnL(fsdb_obj, time.H, time.L);
ffw_CreateVarValueByHandle(fsdb_obj, (void*) &enum_sig,
                           &enum_val_arr[5][0]);

time.H = 0;
time.L = 70;
ffw_CreateXCoorByHnL(fsdb_obj, time.H, time.L);
ffw_CreateVarValueByHandle(fsdb_obj, (void*) &enum_sig,
                           &enum_val_arr[6][0]);
}

```

Figure 8 The Results of sv_dt.cpp Shown on nWave

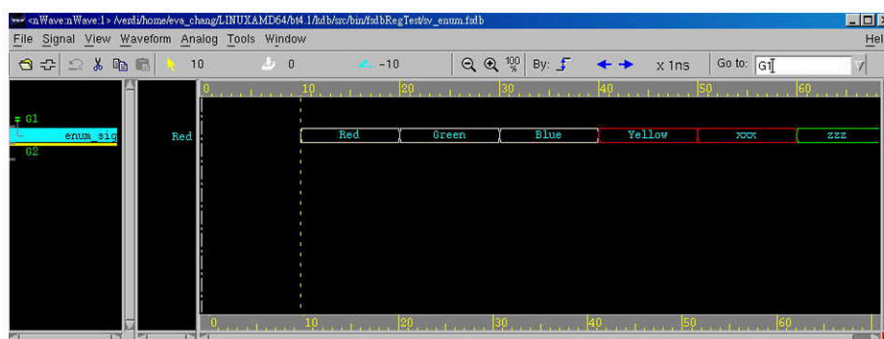
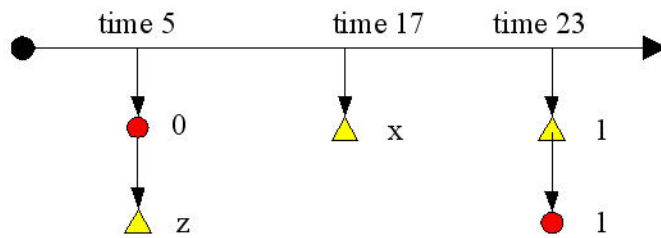
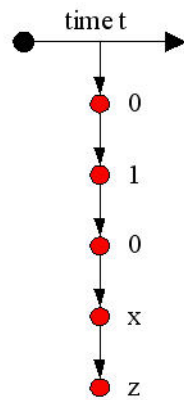


Diagram of Basic Concept



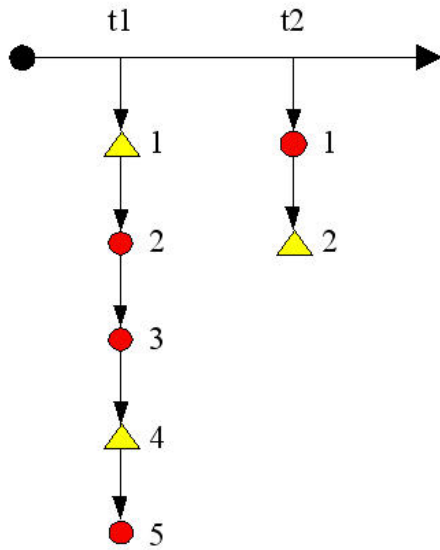
Basic Concept

Diagram of Tree Creation



Tree Creation

Diagram of Value Change Creation



Value Change Creation

2

Function List

This section contains the following topics:

- [ffw_AddBusParameter](#)
- [ffw_AddBusStream](#)
- [ffw_AddRelation](#)
- [ffw_AddSupportInfo](#)
- [ffw_BeginTransaction](#)
- [ffw_BeginTree](#)
- [ffw_Close](#)
- [ffw_CreateABBusValueByHandle](#)
- [ffw_CreateABBusValueByIdcode](#)
- [ffw_CreateArrayBegin](#)
- [ffw_CreateArrayEnd](#)
- [ffw_CreateAttr](#)
- [ffw_CreateAttr2](#)
- [ffw_CreateBus](#)
- [ffw_CreateBusBegin](#)
- [ffw_CreateBusElementByName](#)
- [ffw_CreateBusEnd](#)
- [ffw_CreateEnumAttr](#)
- [ffw_CreateEnumAttr2](#)
- [ffw_CreateFixedPointVarByHandle](#)
- [ffw_CreateFixedPointVarByIdcode](#)

- [ffw_CreateFsdbLog](#)
- [ffw_CreateLoopMarkerVarByVarHdl](#)
- [ffw_CreateLoopMarkerVarByVarIdcode](#)
- [ffw_CreateLoopMarkerVarValueByHandle](#)
- [ffw_CreateLoopMarkerVarValueByIdcode](#)
- [ffw_CreateMDAVarValueByHandle](#)
- [ffw_CreateMDAVarValueByIdcode](#)
- [ffw_CreateMemoryRangeVarByHandle](#)
- [ffw_CreateMemoryRangeVarByIdcode](#)
- [ffw_CreateMemoryRangeVarValueByHandle](#)
- [ffw_CreateMemoryRangeVarValueByIdcode](#)
- [ffw_CreateMessage](#)
- [ffw_CreatePeriodicVarValueByHandle](#)
- [ffw_CreatePeriodicVarValueIdcode](#)
- [ffw_CreatePropVarByHandle](#)
- [ffw_CreatePropVarByIdcode](#)
- [ffw_CreatePropVarValueByHandle](#)
- [ffw_CreatePropVarValueByIdcode](#)
- [ffw_CreateRecordBegin](#)
- [ffw_CreateRecordEnd](#)
- [ffw_CreateRelation](#)
- [ffw_CreateScope](#)
- [ffw_CreateScopeByPath](#)
- [ffw_CreateStream](#)
- [ffw_CreateStructBegin](#)
- [ffw_CreateStructEnd](#)
- [ffw_CreateTreeByHandleScheme](#)

- [ffw_CreateTreeByIdcodeScheme](#)
- [ffw_CreateTriggerPointVarByVarHdl](#)
- [ffw_CreateTriggerPointVarByVarIdcode](#)
- [ffw_CreateTriggerPointVarValueByHandle](#)
- [ffw_CreateTriggerPointVarValueByIdcode](#)
- [ffw_CreateUpscope](#)
- [ffw_CreateVarByHandle](#)
- [ffw_CreateVarByHandleAndPath](#)
- [ffw_CreateVarByIdcode](#)
- [ffw_CreateVarByIdcodeAndPath](#)
- [ffw_CreateVarValueByHandle](#)
- [ffw_CreateVarValueByIdcode](#)
- [ffw_CreateVarValueByVarMapId](#)
- [ffw_CreateXCoorByHnL](#)
- [ffw_Destroy](#)
- [ffw_DisableDumpOff](#)
- [ffw_DisableSyncTree](#)
- [ffw_DTCreateEnum](#)
- [ffw_DTCreateEnum2](#)
- [ffw_DTCreateEnum3](#)
- [ffw_DTCreateFloating](#)
- [ffw_DTCreateInt](#)
- [ffw_DTCreateIntHnL](#)
- [ffw_DTCreatePhysicalBegin](#)
- [ffw_DTCreatePhysicalEnd](#)
- [ffw_DTCreatePhysicalSecondaryUnit](#)
- [ffw_DumpSeqNumTurnOn](#)

- [ffw_DumpStrengthTurnOn](#)
- [ffw_EnableDumpOff](#)
- [ffw_EnableSyncTree](#)
- [ffw_EndTransaction](#)
- [ffw_EndTree](#)
- [ffw_GetActiveObject](#)
- [ffw_GetActiveObjectFileName](#)
- [ffw_GetAttrByHdl](#)
- [ffw_GetDataTypeCreationReady](#)
- [ffw_GetFileName](#)
- [ffw_GetFsdbDumperVersion](#)
- [ffw_GetFsdbVersion](#)
- [ffw_GetGlitchNum](#)
- [ffw_GetIdcodeByHandle](#)
- [ffw_GetIdent](#)
- [ffw_GetMaxVarIdcode](#)
- [ffw_GetMaxXTag](#)
- [ffw_GetMinXTag](#)
- [ffw_GetRefCount](#)
- [ffw_GetStreamByHdl](#)
- [ffw_HasGlitchNumBeenSet](#)
- [ffw_InfoSuppress](#)
- [ffw_Init](#)
- [ffw_IsDumpOff](#)
- [ffw_Open](#)
- [ffw_OpenAutoSwitchFile](#)
- [ffw_RecordTransactionAttribute](#)

- [ffw_RegisterAutoSwitchFileClientData](#)
- [ffw_RegisterDumperCBFunc](#)
- [ffw_RegisterErrorCBFunc](#)
- [ffw_RegisterInfoCBFunc](#)
- [ffw_RegisterTwoGCBFunc](#)
- [ffw_RegisterTwoGCBFuncDontCareLFS](#)
- [ffw_RegisterWarnCBFunc](#)
- [ffw_RegisterWriteErrorCB](#)
- [ffw_ResetXCoor](#)
- [ffw_SetAnnotation](#)
- [ffw_SetAsErrorTransaction](#)
- [ffw_SetBusClock](#)
- [ffw_SetCompressionMethod](#)
- [ffw_SetDesignType](#)
- [ffw_SetDumpLimit](#)
- [ffw_SetGlitchNum](#)
- [ffw_SetLifoDumpLimit](#)
- [ffw_SetLockingScheme](#)
- [ffw_SetMemLimit](#)
- [ffw_SetProducer](#)
- [ffw_SetScaleUnit](#)
- [ffw_SetScopeSeparator](#)
- [ffw_SetSimulationDate](#)
- [ffw_SetSimulatorVersion](#)
- [ffw_SetTransactionColorById](#)
- [ffw_SetTransactionColorByRGB](#)
- [ffw_SetVCMemLimit](#)

- [ffw_SwitchFile](#)
- [ffw_SwitchFileAndCreateInitVal](#)
- [ffw_Sync](#)
- [ffw_SyncControlTurnOff](#)
- [ffw_SyncControlTurnOn](#)
- [ffw_WarnSuppress](#)
- [ffwCreateEvcdVarValByVarHdl](#)
- [ffwCreateEvcdVarValByVarIdcode](#)
- [ffwCreateMDAVarByHandle](#)
- [ffwCreateMDAVarByIdcode](#)
- [ffwCreatePropVarByHandle2](#)
- [ffwCreatePropVarByIdcode2](#)
- [ffwCreateScope](#)
- [ffwCreateScope2](#)
- [ffwCreateScopeByPath](#)
- [ffwCreateStream2](#)
- [ffwCreateStrValByVarHdl](#)
- [ffwCreateStrValByVarIdcode](#)
- [ffwCreateStrVarByVarHdl](#)
- [ffwCreateStrVarByVarIdcode](#)
- [ffwCreateVarByVarHdl](#)
- [ffwCreateVarByVarHdl2](#)
- [ffwCreateVarByVarHdlAndPath](#)
- [ffwCreateVarByVarHdlAndPath2](#)
- [ffwCreateVarByVarIdcode](#)
- [ffwCreateVarByVarIdcode2](#)
- [ffwCreateVarByVarIdcodeAndPath](#)

- [ffwCreateVarByVarIdcodeAndPath2](#)
- [ffwGetVarIdcodeByVarHdl](#)
- [ffwOpenFile](#)
- [ffwOpenFile2](#)

ffw_AddBusParameter

Interface

```
fsdbRC ffw_AddBusParameter (ffwObject *obj, fsdbBusHdl bus, str_T param_name,  
str_T param_value);
```

Purpose

Add a bus parameter to a bus. A bus parameter can be the address width or data width, numbers of masters or slaves, or the endianness of the bus.

Arguments

obj

Pointer to an FSDB Writer object.

bus

Handle of the bus to which the clock is defined.

param_name

Name of the bus parameter.

param_value

Value of the bus parameter.

Return Value

FSDB_RC_SUCCESS upon success. FSDB_RC_FAILURE otherwise.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/bus_tr_recording.cpp
```

See Also

ffw_CreateBus()

ffw_SetBusClock()

ffw_AddBusStream()

ffw_AddBusStream

Interface

```
extern fsdbRC ffw_AddBusStream (ffwObject *obj, fsdbBusHdl bus, fsdbStreamHdl  
stream_hdl);
```

Purpose

Add a stream to the specified bus. A stream contains transactions occurred on the bus. A bus might contain several streams that model levels of abstraction.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`bus`

Handle of the bus to which the clock is defined.

`stream_hdl`

Handle of a stream which holds a set of transactions.

Return Value

FSDB_RC_SUCCESS upon success. FSDB_RC_FAILURE otherwise.

Example

<\$NOVAS_HOME\$>/share/FsdbWriter/example/bus_tr_recording.cpp

See Also

`ffw_CreateBus()`

`ffw_SetBusClock()`

`ffw_AddBusParameter()`

ffw_AddRelation

Interface

```
fsdbRC ffw_AddRelation (ffwObject *obj, fsdbRelationHdl relation_hdl, fsdbTransId  
trans_id1, fsdbTransId trans_id2);
```

Purpose

Add relation between transactions specified by argument `trans_id1` and `trans_id2`.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`relation_hdl`

Handle of a relation which is returned by `ffw_CreateRelation()`.

`trans_id1`

Id of the transaction which is returned by `ffw_BeginTransaction()`

`trans_id2`

Id of the transaction which is returned by `ffw_BeginTransaction()`

Return Value

`FSDB_RC_SUCCESS` upon success. `FSDB_RC_FAILURE` otherwise.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/transaction_recording.cpp
```

See Also

ffw_CreateRelation()

ffw_CreateStream()

ffw_CreateSteam()

ffw_CreateAttr()

ffw_CreateAttr2()

ffw_BeginTransaction()

ffw_EndTransaction()

ffw_AddSupportInfo

Interface

```
extern void ffw_AddSupportInfo (ffwObject *obj, str_T info);
```

Purpose

Add user-defined support information to FSDB file. The information can be any null-terminated string that you need to record in this FSDB file. For example, you can specify your own name in support information so that others can know the creator of this FSDB file when they open it. If you call `ffw_AddSupportInfo()` more than once, new information is appended to the existing, delimiting by '\n', until the total length reaches `FSDB_SUPINFO_USER_DEFINED_LEN`. Further information will be truncated.

Arguments

obj

Pointer to an FSDB Writer object.

info

A null-terminated string that you need to record in this FSDB file.

ffw_BeginTransaction

Interface

```
extern fsdbTransId ffw_BeginTransaction (ffwObject *obj, fsdbStreamHdl  
stream_hdl, fsdbXTag begin_time, str_T label, fsdbAttrHdlVal attr_arr[], ushort_T  
attr_count);
```

Purpose

Create a new transaction with the specified arguments.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`stream_hdl`

Handle of the stream that is used to hold the transaction.

`begin_time`

The begin time of the transaction. It could be current time step or a time step in the past. You cannot create transactions in the future. If you do, you will get an unexpected result.

`label`

The name of the transaction that will be displayed as the title of the transaction in nWave.

`attr_arr`

An array of attributes of the transaction that are to be created during `begin_time`. Each element is a handle/value pair of an attribute. `fsdbAttrHdlVal` is defined as

```
struct fsdbAttrHdlVal {  
    fsdbAttrHdl hdl;  
    byte_T *value;  
};  
typedef struct fsdbAttrHdlVal fsdbAttrHdlVal;
```

`attr_count`

Number of attributes in `attr_arr`.

Return Value

A transaction ID that could be used to refer to the transaction. Its main purpose is to add more attributes to the transaction or end the transaction. `FSDB_INVALID_TRANS_ID` is returned if this API fails to create a transaction.

Limitation

As value change creation, transactions must be created after `ffw_EndTree()` finishes and before the next `ffw_BeginTree()` is called. In other words, you could not create transactions within a tree.

Begin time could be current time step or a time step in the past. You could not create transactions in the future. If you do, you will get an unexpected result.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/transaction_recording.cpp
```

ffw_BeginTree

Interface

```
extern bool_T ffw_BeginTree (ffwObject *obj);
```

Purpose

Begin of tree creation and sets current scope to root scope.

Argument

`*obj`

Pointer to an FSDB Writer object.

Return Value

TRUE if successful; **FALSE** otherwise.

ffw_Close

Interface

```
extern void ffw_Close (ffwObject *obj);
```

Purpose

Close an FSDB file. When the FSDB file is closed, the FSDB Writer flush all in-core value changes to disk file and delete temporary files used by the FSDB Writer.

`ffw_Close()` needs to be called first and then `ffw_Destroy()` to free the memory used by the FSDB Writer.

Argument

`*obj`

Pointer to an FSDB Writer object.

ffw_CreateABBusValueByHandle

Interface

```
extern fsdbGlitchDetectionType ffw_CreateABBusValueByHandle (ffwObject *obj,  
void *var_hdl, fsdbABBusVal *var_val);
```

Purpose

Dump a value change at current xtag, while the value change belongs to the var whose handle is specified by the second parameter, `var_hdl`.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*var_hdl`

Var handle which identifies the var.

`*var_val`

Pointer to a byte array which contains the value change of the var. Refer to `fsdbABBusVal` defined in `fsdbShr.h`.

Return Value

`fsdbGlitchDetectionType` value, which is only referred to by Novas; third parties can ignore the return value.

ffw_CreateABBusValueByldcode

Interface

```
extern fsdbGlitchDetectionType ffw_CreateABBusValueByldcode (ffwObject *obj,  
fsdbVarldcode var_idcode, fsdbABBusVal *var_val);
```

Purpose

Dump a value change at the current xtag, while the value change belongs to the var whose idcode is specified by the second parameter, `var_idcode`.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`var_idcode`

The idcode which represents the var.

`*var_val`

Pointer to a byte array which contains the value change of the var. Refer to **fsdbABBusVal** defined in **fsdbShr.h**.

Return Value

`fsdbGlitchDetectionType` value, which is only referred to by Novas; third parties can ignore the return value.

ffw_CreateArrayBegin

Interface

```
extern void ffw_CreateArrayBegin (ffwObject *obj, str_T array_name, uint_T array_size);
```

Purpose

Start grouping a bunch of signals into array.

Note:

This API is for grouping and does not create any data types.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`array_name`

Pointer to a string that represents array name.

`array_size`

Array size, how many element in this array.

ffw_CreateArrayEnd

Interface

```
extern void ffw_CreateArrayEnd (ffwObject *obj);
```

Purpose

End up grouping a bunch of signals into array.

Argument

`*obj`

Pointer to an FSDB Writer object.

ffw_CreateAttr

Interface

```
extern fsdbAttrHdl ffw_CreateAttr (ffwObject *obj, str_T name, fsdbAttrDataType  
data_type, int larridx, int rarridx, bool_T hidden);
```

Purpose

Create an attribute. Attributes are used to describe properties of a transaction. There are two prerequisites when creating attributes:

1. An attribute must be created between an **ffw_BeginTree()** and its corresponding **ffw_EndTree()** since it is part of a tree
2. **ffw_GetDataTypeCreationReady()** must be called before the first attribute is created. Of course, **ffw_GetDataTypeCreationReady()** needs to be called only once.

Although an attribute is created within a tree, it could be used to create transactions owned by streams in other trees.

This API returns the same attribute handle if and only if all properties of the attribute (including name, data type, larridx, rarridx, and hidden) are the same. If any one of the arguments is different, FSDB Writer returns a new handle even if the name is the same.

An attributes is not bound to any stream or transaction. Once an attribute is created, it could be used to create any transaction of any stream.

An attribute could be an array of elements of data_type. Specifying the arguments larridx and rarridx does this. If it is a scalar, set the two arguments to the same value.

If the attribute is an array of strings, its creation might look like:

```
fsdbAttrHdl attr_hdl;  
fsdbAttrHdlVal attr_val;  
char *str_val[3] = { "string1", "string2", "string3" };  
  
attr_hdl = ffw_CreateAttr(ffw_obj, "string_attr",  
    FSDB_ATTR_DT_STRING, 0, 2, TRUE);  
attr_val.hdl = attr_hdl;  
attr_val.value = (byte_T*) str_val;
```

ffw_CreateAttr2() is recommended, instead of using this API, since this function will not be extended in the future.

Deprecated

This function has been replaced by **ffw_CreateAttr2()**. Although the **ffw_CreateAttr** function still works, it will not be enhanced in the future.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`name`

The name of the attribute.

`data_type`

Data type of the attribute. Refer to the definition of `fsdbAttrDataType` for a full list. It is something like **FSDB_ATTR_DT_LOGIC**.

`larridx`

Left array index of the attribute if it is an array of `data_type`.

`rarridx`

Right array index of the attribute if it is an array of `data_type`.

`hidden`

TRUE if the attribute is not to be displayed in nWave. **FALSE** otherwise. It requires support from nWave and might not be available in earlier versions of nWave.

Return Value

A handle to the attribute that could be used to create transactions.

FSDB_INVALID_ATTR_HDL is returned if this API fails to create an attribute.

Limitation

Currently, the maximum number of attributes that could be created is 65524. Not all data types are supported now. See **fsdbAttrDataType** for a list of supported data types.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/transaction_recording.cpp
```

ffw_CreateAttr2

Interface

```
fsdbAttrHdl ffw_CreateAttr2 (ffwObject *obj, ffwAttrArg *orig_arg);
```

Purpose

Create an attribute. Attributes are used to describe properties of a transaction. There are two prerequisites when creating attributes:

1. An attribute must be created between an **ffw_BeginTree()** and its corresponding **ffw_EndTree()** since it is part of a tree.
2. **ffw_GetDataTypeCreationReady()** must be called before the first attribute is created. Of course, **ffw_GetDataTypeCreationReady()** needs to be called only once.

Although an attribute is created within a tree, it could be used to create transactions owned by streams in other trees.

This API returns the same attribute handle if and only if all properties of the attribute (including name, data type, larridx, rarridx, metainfo and hidden) are the same. If any one of the arguments is different, the FSDB Writer returns a new handle even if the name is the same.

An attribute is not bound to any stream or transaction. Once an attribute is created, it could be used to create any transaction of any stream.

An attribute could be an array of elements of data_type. Specifying the arguments larridx and rarridx does this. If it is a scalar, set the two arguments to the same value.

orig_arg->size to sizeof(ffwAttrArg) must be set to ensure backward compatibility of the FSDB Writer library.

Arguments

*obj

Pointer to an FSDB Writer object.

*orig_arg

A pointer to an attribute argument. Refer to **ffwAttrArg** defined in **ffwAPI.h**.

Return Value

A handle to the attribute that could be used to create transactions.
FSDB_INVALID_ATTR_HDL is returned if this API fails to create an attribute.

Limitation

Currently, the maximum number of attributes that could be created is 65524. Not all data types are supported now. See **fsdbAttrDataType** for a list of supported data types.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/transaction_recording.cpp
```

ffw_CreateBus

Interface

```
extern fsdbAttrHdl ffw_CreateBus (ffwObject *obj, str_T bus_name, str_T protocol_name);
```

Purpose

Create a bus of certain interconnect protocol. A bus is used to hold bus-related information and streams, which hold bus transactions of different levels of abstraction.

Note:

A bus must not be created between tree creations, because it is not part of a tree.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`bus_name`

Name of the bus.

`protocol_name`

Name of the interconnect protocol. There are also predefined protocol names available such as **AHB_PROTOCOL**, **APB_PROTOCOL**, etc. See **fsdbShr.h** for details.

Return Value

A handle to the created bus. **FSDB_INVALID_BUS_HDL** is returned on failure.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/bus_tr_recording.cpp
```

See Also

ffw_CreateBus()

ffw_SetBusClock()

ffw_AddBusStream()

ffw_CreateBusBegin

Interface

```
extern void ffw_CreateBusBegin (ffwObject *obj, str_T bus_name);
```

Purpose

Start grouping a bunch of signals into bus

Note:

This API is for grouping and does not create any data types.

Arguments

obj

Pointer to an FSDB Writer object.

bus_name

Pointer to a string that represents bus name.

ffw_CreateBusElementByName

Interface

```
extern void ffw_CreateBusElementByName (ffwObject *obj, char
*full_path_bus_element_name);
```

Purpose

Add a bus element.

Arguments

**obj*

Pointer to an FSDB Writer object.

**full_path_bus_element_name*

Pointer to string that represents the full path bus element name.

ffw_CreateBusEnd

Interface

```
extern void ffw_CreateBusEnd (ffwObject *obj);
```

Purpose

End up grouping a bunch of signals into bus.

Argument

**obj*

Pointer to an FSDB Writer object.

ffw_CreateEnumAttr

Interface

```
fsdbAttrHdl ffw_CreateEnumAttr (ffwObject *obj, str_T name, int larridx, int rarridx, bool_T hidden, ushort_T literal_count, uint_T val_len, fsdbEnumValueType val_type, str_T literal_arr[], byte_T val_arr[])
```

Purpose

Create an enum type attribute. Attributes are used to describe properties of a transaction. Refer to **ffw_CreateAttr()** for prerequisites and details about attribute creation.

ffw_CreateEnumAttr2() is recommended, instead of using this API, since this function will not be extended in the future.

Deprecated

This function has been replaced by **ffw_CreateEnumAttr2()**. Although the **ffw_CreateEnumAttr** function still works, it will not be enhanced in the future.

Arguments

obj

Pointer to an FSDB Writer object.

name

The name of the attribute.

larridx

Left array index of the attribute if the enum attribute is an array.

rarridx

Right array index of the attribute if the enum attribute is an array.

hidden

TRUE if the attribute is not to be displayed in nWave. **FALSE** otherwise. It requires support from nWave and might not be available in earlier *nWave* versions.

literal_count

The number of literals in the argument `literal_arr`.

`val_len`

Size of an enum value. If `val_type` is **FSDB_ENUM_VALUE_TYPE_INTEGER**, the value is `sizeof(int)`. If `val_type` is **FSDB_ENUM_VALUE_TYPE_LOGIC**, the value is the width of a logic enum value.

`val_type`

Value type of the enum value. Refer to the definition of `fsdbEnumValueType` for a full list. It is something like **FSDB_ENUM_VALUE_TYPE_INTEGER**.

`literal_arr`

Enum literal array.

`literal_arr`

Enum value array.

Return Value

A handle to the attribute that could be used to create transactions.

FSDB_INVALID_ATTR_HDL is returned if this API fails to create an attribute.

Limitation

Currently, the maximum number of attributes that could be created is 65524.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/transaction_recording.cpp
```

ffw_CreateEnumAttr2

Interface

```
extern fsdbAttrHdl ffw_CreateEnumAttr2 (ffwObject *obj, ffwEnumAttrArg *orig_arg)
```

Purpose

Create an enum type attribute. Attributes are used to describe properties of a transaction. Refer to **ffw_CreateAttr2()** for prerequisites and details about attribute creation.

orig_arg->size to sizeof(ffwEnumAttrArg) must be set to ensure backward compatibility of the FSDB Writer library.

Arguments

*obj

Pointer to an FSDB Writer object.

*orig_arg

A pointer to an enum attribute argument. Refer to **ffwEnumAttrArg** defined in **ffwAPI.h**.

Return Value

A handle to the attribute that could be used to create transactions.

FSDB_INVALID_ATTR_HDL is returned if this API fails to create an attribute.

Limitation

Currently, the maximum number of attributes that could be created is 65524.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/transaction_recording.cpp
```

ffw_CreateFixedPointVarByHandle

Interface

```
extern ffwVarMapId ffw_CreateFixedPointVarByHandle(ffwObject *obj,  
ffwFixedPointVarArg *orig_arg);
```

Purpose

Create a fixed point var by handle. The handle is usually a pointer to an object. Because the virtual address of an object is unique, the application can take this advantage to represent a unique var by the virtual address of an object.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*orig_arg`

A pointer to and fixed point var argument. Refer to **ffwFixedPointVarArg** defined in **ffwAPI.h**.

Return Value

Non-NULL if successful; NULL if fail; where **ffwVarMapId** is defined in **ffwAPI.h**.

ffw_CreateFixedPointVarByIdcode

Interface

```
extern fsdbVarIdcode ffw_CreateFixedPointVarByIdcode(ffwObject *obj,  
ffwFixedPointVarArg *orig_var);
```

Purpose

Create a fixed point var (signal) by idcode, while the idcode is given by **ffwFixedPointVarArg**.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*orig_arg`

A pointer to and fixed point var argument. Refer to **ffwFixedPointVarArg** defined in **ffwAPI.h**.

Return Value

A positive integer if successful.

A negative integer (**FSDB_INVALID_VAR_IDCODE**) if fail.

ffw_CreateFsdbLog

Interface

```
extern fsdbFsdbLogId ffw_CreateFsdbLog(ffwObject *obj, fsdbStreamHdl stream_hdl,  
ffwFSDBLog *log);
```

Purpose

Dump a value change at current xtag, while the value change belongs to the stream whose handle is specified by the second parameter, stream_hdl. The value is specified by the **ffwFSDBLog** type log.

Arguments

obj

Pointer to an FSDB Writer object.

stream_hdl

Handle of the stream to identify the var.

log

Pointer to an **ffwFSDBLog** value. Refer to **ffwFSDBLog** defined in **ffwAPI.h**.

Return Value

An fsdbLog ID that can be used to refer to the fsdbLog. Its main purpose is to add more attributes to the fsdbLog or end the fsdbLog. **FSDB_INVALID_TRANS_ID** is returned if this API fails to create an fsdbLog.

ffw_CreateLoopMarkerVarByVarHdl

Interface

```
extern ffwVarMapId ffw_CreateLoopMarkerVarByVarHdl (ffwObject *obj, str_T var_name,  
void *var_hdl);
```

Purpose

Create a loop marker var by handle. The handle is usually a pointer to an object. Since the virtual address of an object is unique. The application can take this advantage to represent a unique var by the virtual address of an object. The var type of a string var is **FSDB_VT_LOOP_MARKER**. The maximum number of such variables user could create in an FSDB file is **FSDB_MAX_LOOP_MARKER_COUNT**, which is defined in **fsdbShr.h**.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`var_name`

Pointer to a string that represents the var name.

`*var_hdl`

Var handle. Usually a virtual address of an object which can identify a unique var.

Return Value

Non-NULL if successful; NULL otherwise, where **ffwVarMapId** is defined in **ffwAPI.h**.

ffw_CreateLoopMarkerVarByVarIdcode

Interface

```
extern fsdbVarIdcode ffw_CreateLoopMarkerVarByVarIdcode (ffwObject *obj, str_T  
var_name, fsdbVarIdcode var_idcode);
```

Purpose

Create a var by idcode, while the idcode is given by the application. The var type of a loop marker var is **FSDB_VT_LOOP_MARKER**. The maximum number of such variables user could create in an FSDB file is **FSDB_MAX_LOOP_MARKER_COUNT**, which is defined in **fsdbShr.h**.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`var_name`

Pointer to a string that represents the var name.

`var_idcode`

The idcode which represents the var.

Return Value

A positive integer if successful. A negative integer (**FSDB_INVALID_VAR_IDCODE**) if fail.

ffw_CreateLoopMarkerVarValueByHandle

Interface

```
extern fsdbGlitchDetectionType ffw_CreateLoopMarkerVarValueByHandle (ffwObject *obj,  
void *var_hdl, fsdbXTag *begin_time);
```

Purpose

Create a loop for the loop marker var whose handle is specified by the second parameter, `var_hdl`. A loop consists of a begin time and an end time. The begin time is specified by the third parameter `begin_time`. The end time is represented by current time step.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*var_hdl`

Var handle which identifies the var.

`*begin_time`

The begin time of the loop to be created.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

ffw_CreateLoopMarkerVarValueByIdcode

Interface

```
extern fsdbGlitchDetectionType ffw_CreateLoopMarkerVarValueByIdcode (ffwObject *obj,  
fsdbVarIdcode var_idcode, fsdbXTag *begin_time);
```

Purpose

Create a loop for the loop marker var whose idcode is specified by the second parameter, `var_idcode`. A loop consists of a begin time and an end time. The begin time is specified by the third parameter `begin_time`. The end time is represented by current time step.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`var_idcode`

The idcode which represents the var.

`*begin_time`

The begin time of the loop to be created.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

ffw_CreateMDAVarValueByHandle

Interface

```
extern fsdbGlitchDetectionType ffw_CreateMDAVarValueByHandle (ffwObject *obj, void  
*var_hdl, int address, byte_T *val);
```

Purpose

Dump a value change at current xtag. The value change belongs to the address of the MDA var whose handle is specified by the second parameter *var_hdl*.

The application does not need to create initial value for each address of the MDA var since it is already given in **ffw_CreateMDAVarByIdcode** or **ffw_CreateMDAVarByHandle**.

Deprecated

This function has been replaced by **ffwCreateMDAVarValueByHandle()**. Although the **ffw_CreateMDAVarValueByHandle** function still works, it will not be enhanced in the future.

Arguments

**obj*

Pointer to an FSDB Writer object.

**var_hdl*

Var handle which represents the MDA var.

address

The address of the MDA var we want to dump the value change for.

**val*

Pointer to a byte array which contains the value change of the address of the var.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

See Also

ffw_CreateMDAVarValueByIdcode

Limitation

See Limitation of **ffw_CreateMDAVarValueByIdcode**

ffw_CreateMDAVarValueByIdcode

Interface

```
extern fsdbGlitchDetectionType ffw_CreateMDAVarValueByIdcode (ffwObject *obj,  
fsdbVarIdcode var_idcode, int address, byte_T *val);
```

Purpose

Dump a value change at current xtag. The value change belongs to the address of the MDA var whose idcode is specified by the second parameter *var_idcode*.

The address of an MDA var is calculated in row-major order. For example, the address `mem[a1][a2][a3]` of the following signal:

```
mem[0:r1-1][0:r2-1][0:r3-1]
```

is

```
a1 * (r2*r3) + a2 * r3 + a3
```

The application does not need to create an initial value for each address of the MDA var. It's already given in **ffw_CreateMDAVarByIdcode** or **ffw_CreateMDAVarByHandle**.

Deprecated

This function has been replaced by **ffwCreateMDAVarValueByIdcode()**. Although the **ffw_CreateMDAVarValueByIdcode** function still works, it will not be enhanced in the future.

Arguments

**obj*

Pointer to an FSDB Writer object.

var_idcode

The idcode which represents the MDA var.

address

The address of the MDA var we want to dump the value change for.

**val*

Pointer to a byte array which contains the value change of the address of the var.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

See Also

ffw_CreateMDAVarValueByHandle

Limitation

The FSDB Writer does not do glitch filtering in this API. The FSDB Writer does not detect duplicate value changes in this API.

ffw_CreateMemoryRangeVarByHandle

Interface

```
extern fsdbGlitchDetectionType ffw_CreateMemoryRangeVarByHandle (ffwObject *obj,  
void *var_hdl, fsdbVarType var_type, str_T var_name);
```

Purpose

ffw_CreateMemoryRangeVarByHandle creates a memory range var by handle. The handle is usually a pointer to an object since the virtual address of an object is unique. The application can take this advantage to represent a unique var by the virtual address of an object. For the details about how to define a memory range, see **ffw_CreateMemoryRangeVarByldcode**.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*var_hdl`

Var handle. Usually a virtual address of an object which can identify a unique var.

`var_type`

Var type. Refer to **fsdbVarType** defined in **fsdbShr.h**. Its value should be **FSDB_VT_VCD_MEMORY_RANGE** or **FSDB_VT_VHDL_MEMORY_RANGE**.

`var_name`

Pointer to a string that represents the var name.

Return Value

A positive integer if successful, a negative integer (**FSDB_INVALID_VAR_IDCODE**) if fail.

ffw_CreateMemoryRangeVarByIdcode

Interface

```
extern fsdbVarIdcode ffw_CreateMemoryRangeVarByIdcode (ffwObject *obj,  
fsdbVarIdcode var_idcode, fsdbVarType var_type, str_T var_name);
```

Purpose

ffw_CreateMemoryRangeVarByIdcode creates a memory range var by idcode, while the idcode is given by the application. A block of memory is a two-dimensional matrix of values and consists of a memory range variable and a group of memory variables. A memory range variable indicates the boundary of the block of memory, which is defined by the name of the variable. For example, if the block of memory consists of 4 words of 8 bits each, it can be defined as:

```
my_mem[0:3]  
my_mem[0]  
my_mem[1]  
my_mem[2]  
my_mem[3]
```

The memory range variable `my_mem[0:3]` indicates that the block of memory consists of 4 memory variables and their names are of the format `my_mem[i]`. `i` indicates the index of the memory variables and starts from 0 to 3.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`var_idcode`

ffw_CreateMemoryRangeVarValueByHandle

The idcode which represents the var.

`var_type`

Var type. Refer to **fsdbVarType** defined in **fsdbShr.h**. Its value should be **FSDB_VT_VCD_MEMORY_RANGE** or **FSDB_VT_VHDL_MEMORY_RANGE**.

`var_name`

pointer to a string that represents the var name.

Return Value

A positive integer if successful, a negative integer (**FSDB_INVALID_VAR_IDCODE**) if fail.

ffw_CreateMemoryRangeVarValueByHandle

Interface

```
extern fsdbGlitchDetectionType ffw_CreateMemoryRangeVarValueByHandle (ffwObject
*obj, void *var_hdl, int start_mem_idx, int close_mem_idx);
```

Purpose

ffw_CreateMemoryRangeVarValueByHandle dumps a memory range value change (start_mem_idx and close_mem_idx) at current xtag, while the value change belongs to the var whose handle is specified by the second parameter, var_hdl. For the details about how to specify start_mem_idx and close_mem_idx, see **ffw_CreateMemoryRangeVarValueByIdcode**.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*var_hdl`

Var handle which identifies the var.

`start_mem_idx`

Indicates the start of all memory variable indexes that have value changes at current xtag.

`close_mem_idx`

Indicates the end of all memory variable indexes that have value changes at current xtag.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

ffw_CreateMemoryRangeVarValueByldcode

Interface

```
extern fsdbGlitchDetectionType ffw_CreateMemoryRangeVarValueByldcode (ffwObject  
*obj, fsdbVarldcode var_idcode, int start_mem_idx, int close_mem_idx);
```

Purpose

ffw_CreateMemoryRangeVarValueByldcode dumps a memory range value change (start_mem_idx and close_mem_idx) at current xtag, while the value change belongs to the var whose idcode is specified by the second parameter, var_idcode. start_mem_idx and close_mem_idx are used to surround the starting and closing indexes of memory variable value changes at current xtag. For example, if the name of the memory range variable is my_mem[1:10] and memory variables my_mem[3], my_mem[5], my_mem[6], my_mem[9] have value changes at current xtag, you should specify start_mem_idx as 3 and close_mem_idx as 9 so that the four value changes can be fully surrounded. start_mem_idx does not have to be smaller than close_mem_idx. In the above example, if the name of the memory range variable is my_mem[10:1], then start_mem_idx should be specified as 9 and the close_mem_idx as 3, depending on the direction of indexes in the memory range variable name.

Arguments

**obj*

Pointer to an FSDB Writer object.

var_idcode

var idcode which identifies the memory range var.

start_mem_idx

Indicates the start of all memory variable indexes that have value changes at current xtag.

close_mem_idx

Indicates the end of all memory variable indexes that have value changes at current xtag.

Return Value

fsdbGlitchDetectionType value is only referred to by Novas; third parties can ignore the return value.

ffw_CreateMessage

Interface

```
extern FsdbMessageld ffw_CreateMessage (ffwObject *obj, fsdbStreamHdl stream_hdl,  
fsdbXTag time, str_T label, fsdbAttrHdlVal attr_arr[], ushort_T attr_count, fsdbColorId  
color_id)
```

Purpose

Create a new message with the specified arguments.

Arguments

obj

A pointer to an FSDB Writer object.

stream_hdl

Handle of the stream which must be of the type **FSDB_STREAM_TYPE_MESSAGE** and is used to hold the messages.

time

The time of the message. It can be a current time step or a time step in the past. Messages cannot be created in the future. If they are, unexpected results will occur.

label

The name of the message displayed as the title of the message in *nWave*.

attr_arr

An array of attributes belonging to the message. Each element is a handle/value pair of an attribute. **fsdbAttrHdlVal** is defined as struct **fsdbAttrHdlVal** {fsdbAttrHdl hdl; byte_T *value;}; typedef struct fsdbAttrHdlVal fsdbAttrHdlVal;

`attr_count`

Number of attributes in `attr_arr`.

`color_id`

The color of the message label. The **fsdbColorId** definition can be found in **ffShr.h**.

Return Value

The message ID can be used to refer to the message.

FSDB_INVALID_MESSAGE_ID is returned if this API fails to create a message.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/message.cpp
```

ffw_CreatePeriodicVarValueByHandle

Interface

```
extern fsdbGlitchDetectionType ffw_CreatePeriodicVarValueByHandle (ffwObject *obj, void  
*var_hdl, ffwClkArg *arg);
```

Purpose

Dump periodic value changes from the current xtag to the file end, when the value changes belong to the *var* whose handle is specified by the second parameter, *var_hdl*.

Note:

Both **ffw_CreateVarValueByHandle** and **ffw_CreatePeriodicVarValueByHandle** cannot be called with the same var handle.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*var_hdl`

Var handle to identify the var.

*arg

Refer to struct **ffwClkArg** defined in **ffwAPI.h**.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

Example

```
//  
// A part of code to dump periodic value changes as follows:  
// time      : 200, 300, 400, 500, ...  
// value change : 0    1    0    1    ...  
//  
  
ffwClkArg arg;  
arg.size = sizeof(ffwClkArg);  
arg.duration.hltag.L = 100;  
arg.duration.hltag.H = 0;  
arg.pattern = FFW_CLK_PATTERN_ONE_ZERO_CYCLE;  
fsdbXTag time;  
time.H = 0; time.L = 200  
  
ffw_CreateXCoorByHnL(obj, time.H, time.L);  
ffw_CreatePeriodicVarValueByHandle(obj, var_hdl, &arg);  
//  
// We cannot dump any other value changes for this var_hdl  
//
```

ffw_CreatePeriodicVarValueIdcode

Interface

```
extern fsdbGlitchDetectionType ffw_CreatePeriodicVarValueByIdcode (ffwObject *obj,  
fsdbVarIdcode var_idcode, ffwClkArg *arg);
```

Purpose

Dump periodic value changes from the current xtag to the file end, when the value changes belong to the *var* whose idcode is specified by the second parameter, *var_idcode*.

Note:

Both **ffw_CreateVarValueByIdcode** and **ffw_CreatePeriodicVarValueIdcode** cannot be called with the same var idcode.

Arguments

**obj*

Pointer to an FSDB Writer object.

var_idcode

Var idcode to represent the var.

**arg*

Refer to struct **ffwClkArg** defined in **ffwAPI.h**.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

ffw_CreatePropVarByHandle

Interface

```
fsdbVarIdcode ffw_CreatePropVarByHandle (ffwObject *obj, ffwPropVarArg *arg);
```

Purpose

Create a property var by handle. The handle is usually a pointer to an object since the virtual address of an object is unique, the application can take this advantage to represent a unique var by the virtual address of an object.

ffwCreatePropVarByHandle2() is recommended, instead of using this API, since this function will not be extended in the future.

Deprecated

This function has been replaced by **ffwCreatePropVarByHandle()**. Although the **ffw_CreatePropVarByHandle** function still works, it will not be enhanced in the future.

Arguments

*obj

Pointer to an FSDB Writer object.

*arg

A pointer pointing to a property variable argument, **ffwPropVarArg**. Refer to **ffwAPI.h** for details. A brief description of the argument is as follows.

```
// Argument passed to the API format
struct ffwPropVarArg {
ffwIdcodeOrHdl u; /* var idcode or var handle */
str_T name; /* var name */
byte_T type; /* var type */
byte_T not_used[3];
str_T assoc_clk_name; /* associated clock name */
char **assoc_var_name; /* associated variable name */
uint_T count; /* count of assoc_var_name */
str_T text_description;
};
typedef struct ffwPropVarArg ffwPropVarArg;
//Detailed description of the fields above
ffwIdcodeOrHdl: An unsigned integer or a pointer which points to a void
type object.
var_name: A string pointer representing full hierarchy name or plain
name.
var_type: For a property variable, it's variable type can be either one
of the following:
FSDB_VT_PROP_ASSUME /* Property assume(similiar to
** assert but shown in different way) */
FSDB_VT_PROP_RESTRICT /* Property input constraint */
FSDB_VT_PROP_COVER /* Property cover clause */
FSDB_VT_PROP_LOCAL_MEMORY /* Property local memory clause */
FSDB_VT_PROP_LOCAL /* Property local clause */
FSDB_VT_PROP_EVENT /* Property event clause */
FSDB_VT_PROP_BOOL /* Property bool clause */
FSDB_VT_PROP_ASSERT_FORBID /* Property assert forbid clause */
FSDB_VT_PROP_ASSERT_CHECK /* Property assert check clause */
FSDB_VT_PROP_ASSERT /* for backward compatible */
For details, refer to fsdbVarType defined in fsdbShr.h.
assoc_clk_name: Pointer to a string representing associated clock name.
The assoc_clk_name can be a slice of a vector signal.
(Format: assoc_clk_name_string[lbitnum, rbitnum] )
**assoc_var_name[]: An array of pointers pointing to associated variable
names.
The assoc_var_name can be a slice of a vector variable.
(Format: assoc_var_name_string[lbitnum, rbitnum] )
count: the number of assoc_var_name strings.
text_description: Pointer to a string describing the property var.
```

With predefined formats, the text_description string can be used for special purposes.

[Known special uses of text_description]

Associate "failure message" with a property variable.

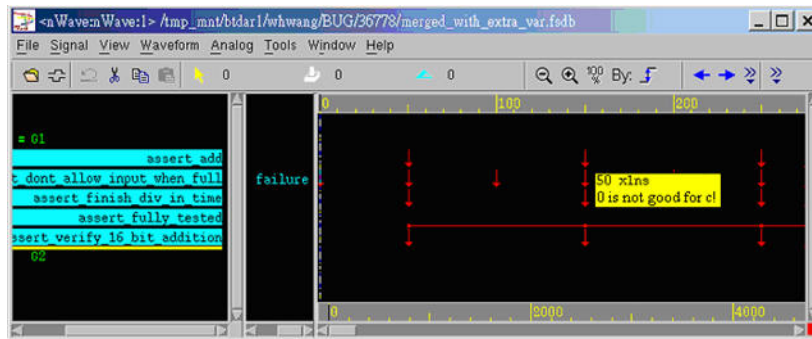
[Format]

%__DESC__% normal descriptions %__FAIL_MSG__% failure message.

^^^^^^^^^^^^^^^^delimiter ^^^^^^^^^^^^^^^^^delimiter

When the associated variable has assertion failures, failure message (string behind delimiter

"%__FAIL_MSG__%") will be shown on nWave like the graph below.



Return Value

A positive integer if successful. A negative integer (**FSDB_INVALID_VAR_IDCODE**) if fail.

Example

[OVA code]

```
assert assert_his_event: forbid (his_event);
clock posedge his_clk {
    event his_event: #[3..6] data != ready;
}
```

//

// An example of possible corresponding codes to dump this property var to FSDB.

//

```
char *prop_sig_assoc_var_name[] = {
    "data", "ready"
};
```

```
struct PropSignal {
    str_T      name;                /* var name */
    byte_T     type;                /* var type */
    str_T      assoc_clk_name;      /* associated clock name */
    char       **assoc_var_name;    /* associated variable name */
    uint_T     count;              /* count of assoc_var_name */
}
```

```
        str_T        text_description;
};
typedef struct PropSignal PropSignal;

PropSignal prop_sig1 = {
    "assert_his_event", FSDB_VT_PROP_ASSERT_FORBID, "his_clk",
    prop_sig1_assoc_var_name, 2,
    "This is a property signal "
};

ffwPropVarArg arg,
memset((void*)&arg, 0, sizeof(ffwPropVarArg));
arg.u.hdl= (void*) prop_sig1;
arg.name= prop_sig1.name;
arg.type= prop_sig1.type;
arg.assoc_clk_name= prop_sig1.assoc_clk_name;
arg.assoc_var_name= prop_sig1.assoc_var_name;
arg.count= prop_sig1.count;
arg.text_description= prop_sig1.text_description;
ffw_CreatePropVarByHandle(ffw_obj, &arg);
```

ffw_CreatePropVarByIdcode

Interface

fsdbVarIdcode ffw_CreatePropVarByIdcode (ffwObject *obj, ffwPropVarArg *arg);

Purpose

Create a property var by idcode, while the idcode is given by the application.
ffwCreatePropVarByIdcode2() is recommended, instead of using this API, since this function will not be extended in the future.

Deprecated

This function has been replaced by **ffwCreatePropVarByIdcode()**. Although the **ffw_CreatePropVarByIdcode** function still works, it will not be enhanced in the future.

Arguments

*obj

Pointer to an FSDB Writer object.

*arg

A pointer to a property variable argument, **ffwPropVarArg**. Refer to **ffwAPI.h** and **ffw_CreatePropVarByHandle()** for details.

Return Value

A positive integer if successful. A negative integer (**FSDB_INVALID_VAR_IDCODE**) if fail.

ffw_CreatePropVarValueByHandle

Interface

```
fsdbGlitchDetectionType ffw_CreatePropVarValueByHandle (ffwObject *obj, void *var_hdl,
fsdbXtag *begin_time, str_T result_str)
```

Purpose

Dump an assertion evaluation result at current xtag, while the result belongs to the property var whose handle is specified by the second parameter, var_hdl.

Arguments

**obj*

Pointer to an FSDB Writer object.

**var_hdl*

Var handle which identifies the var.

**begin_time*

A pointer to a 64-bit time structure, fsdbXtag, whose value is the time step where the evaluation threads starts.

result_str

A pointer to a string representing an evaluation result. Some standard strings are used for special purposes, such as “failure”, “success” and “unknown”. When result_str are filled with those standard strings, nWave displays a corresponding graph according to their var type. See the table below for details. If result_str is not a standard string, it will be displayed as a string on nWave.

	result_str
--	------------

Property var type	failure	success	unknown
FSDB_VT_PROP_EVENT	↓		X
FSDB_VT_PROP_BOOL	↓		X
FSDB_VT_PROP_ASSERT_FOR BID	⚡	↑	X
FSDB_VT_PROP_ASSERT_CHECK	↓	↑	X
FSDB_VT_PROP_ASSERT	↓	↑	X

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/prop.cpp
```

See Also

ffwCreatePropVarByIdcode2()
ffw_CreatePropVarByIdcode()
ffwCreatePropVarByHandle2()
ffw_CreatePropVarByHandle()
ffw_CreatePropVarValueByIdcode()

ffw_CreatePropVarValueByIdcode

Interface

fsdbGlitchDetectionType ffw_CreatePropVarValueByIdcode (ffwObject *obj, fsdbVarIdcode var_idcode, fsdbXtag *begin_time, str_T result_str)

Purpose

Dump an assertion evaluation result at current xtag, while the result belongs to the property var whose idcode is specified by the second parameter, var_idcode.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`var_idcode`

Var idcode which identifies the var.

`*begin_time`

A pointer pointing to a 64-bit time structure, fsdbXtag, whose value is the time step where the evaluation threads starts.

`result_str`

A pointer pointing to a string representing an evaluation result. Refer to **ffw_CreatePropVarValueByHandle()** for details.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/prop.cpp
```

See Also

`ffwCreatePropVarByIdcode2()`

`ffw_CreatePropVarByIdcode()`

`ffwCreatePropVarByHandle2()`

`ffw_CreatePropVarByHandle()`

`ffw_CreatePropVarValueByHandle()`

ffw_CreateRecordBegin

Interface

```
ffw_CreateRecordBegin (ffwObject *obj, str_T record_name, uint_T field_count);
```

Purpose

Start grouping a bunch of signals into record.

Note:

This API is for grouping and does not create any data types.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`record_name`

Pointer to a string that represents record name.

`field_count`

Field count, how many fields in this record.

ffw_CreateRecordEnd

Interface

```
extern void ffw_CreateRecordEnd (ffwObject *obj);
```

Purpose

End up grouping a bunch of signals into record.

Argument

`*obj`

Pointer to an FSDB Writer object.

ffw_CreateRelation

Interface

```
fsdbRelationHdl ffw_CreateRelation (ffwObject *obj, str_T name);
```

Purpose

Create a relation instance used for building relations between transactions.

Arguments

obj

Pointer to an FSDB Writer object.

name

Name of the relation.

Return Value

A handle to the relation instance that could be used to add relation between transactions. **FSDB_INVALID_RELATION_HDL** is returned if this API fails to create an attribute.

See Also

`ffw_AddRelation()`

`ffw_BeginTransction()`

`ffw_EndTransaction()`

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/transaction_recording.cpp
```

ffw_CreateScope

Interface

```
extern void ffw_CreateScope (ffwObject *obj, byte_T type, str_T name);
```

Purpose

Create a sub-scope under current scope and then current scope goes to the newly created sub-scope.

Deprecated

This function has been replaced by **ffwCreateScope2()**. Although the function **ffwCreateScope** still works, it will not be enhanced in the future.

Arguments

obj

Pointer to an FSDB Writer object.

type

Scope type, refer to **fsdbScopeType** defined in **fsdbShr.h**.

name

Pointer to a string that represents scope name.

ffw_CreateScopeByPath

Interface

```
extern fsdbRC ffw_CreateScopeByPath (ffwObject *obj, byte_T scope_type, str_T  
full_path_scope_name, str_T scope_separator);
```

Purpose

Create a scope hierarchy by parsing the parameter *full_path_scope_name*. If the application provides a full path scope name to this API, API will parse the hierarchical

scope name; divide it into tokens using the *scope_separator* provided. Scope tree is then created according to those tokens and their precedence.

Deprecated

This function has been replaced by **ffwCreateScopeByPath2()**. Although the **ffw_CreateScopeByPath** function still works, it will not be enhanced in the future.

Arguments

**obj*

Pointer to an FSDB Writer object.

scope_type

Scope type. Refer to **fsdbScopeType** defined in **fsdbShr.h**.

full_path_scope_name

Pointer to a string that represents the full path scope name.

scope_separator

Pointer to a string that represents the scope separator.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_POINTER_IS_NULL if *full_path_scope_name* is NULL.

FSDB_RC_SCOPE_TYPE_IS_OUT_OF_RANGE if *scope_type* is invalid.

FSDB_RC_MEMORY_IS_EXHAUSTED if memory is exhausted.

ffw_CreateStream

Interface

```
extern fsdbStreamHdl ffw_CreateStream (ffwObject *obj, str_T name);
```

Purpose

Create a stream to hold transactions. A stream must be created between an **ffw_BeginTree()** and its corresponding **ffw_EndTree()** since it is part of a tree.

ffwCreateStream2() is recommended, instead of using this API, since this function will not be extended in the future.

Deprecated

This function has been replaced by **ffwCreateStream2()**. Although the **ffw_CreateStream** function still works, it will not be enhanced in the future.

Arguments

**obj*

Pointer to an FSDB Writer object.

name

The name of the stream.

Return Value

A handle to the stream that could be used to hold transactions.

FSDB_INVALID_STREAM_HDL is returned if this API fails to create a stream.

ffw_CreateStructBegin

Interface

```
extern void ffw_CreateStructBegin (ffwObject *obj, str_T struct_name, fsdbStructType struct_type, uint_T field_count);
```

Purpose

Start grouping a bunch of signals into a struct. This API is for grouping, it does not create any data type. This API is similar to **ffw_CreateRecordBegin()**. The difference is that this API must be supplied with a struct type.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`struct_name`

Pointer to a string that represents the struct name.

`struct_type`

Struct type. Refer to **fsdbStructType** defined in **fsdbShr.h**.

```
enum fsdbStructType {
    FSDB_STRUCT_TYPE_VHDL_RECORD      = 0,
    FSDB_STRUCT_TYPE_HVL_DL           = 1,    /* HVL driver/load */

    FSDB_STRUCT_TYPE_UNPACKED_STRUCT = 10,    /* SV unpacked struct */
    FSDB_STRUCT_TYPE_PACKED_STRUCT   = 11,    /* SV packed struct */
                                           /* 12-14 reserved */
    FSDB_STRUCT_TYPE_UNPACKED_UNION  = 15,    /* SV unpacked union */
    FSDB_STRUCT_TYPE_PACKED_UNION    = 16,    /* SV packed union */
                                           /* 17-20 reserved */
};
typedef enum fsdbStructType fsdbStructType;
```

`field_count`

The number of fields in this struct.

ffw_CreateStructEnd

Interface

```
extern void ffw_CreateStructEnd (ffwObject *obj);
```

Purpose

End grouping a bunch of signals into struct.

Argument

`*obj`

Pointer to an FSDB Writer object.

ffw_CreateTreeByHandleScheme

Interface

```
extern void ffw_CreateTreeByHandleScheme (ffwObject *obj);
```

Purpose

Set tree creation scheme as handle scheme, which means the application does not need to worry about making idcode for each signal, it leaves that job to the FSDB Writer. **ffw_CreateTreeByHandleScheme** also enables the FSDB Writer to set up necessary data structures for subsequent var creation, such as hashing table, etc.

Argument

*obj

Pointer to an FSDB Writer object.

ffw_CreateTreeByIdcodeScheme

Interface

```
extern void ffw_CreateTreeByIdcodeScheme (ffwObject *obj);
```

Purpose

Set tree creation scheme as idcode scheme, which means the application takes the responsibility to make an idcode for each signal, and it must guarantee each idcode is positive integer, and all idcodes are continuous from the smallest to the biggest one. The default scheme for tree creation is idcode scheme.

Argument

*obj

Pointer to an FSDB Writer object.

ffw_CreateTriggerPointVarByVarHdl

Interface

```
extern ffwVarMapId ffw_CreateTriggerPointVarByVarHdl (ffwObject *obj, str_T var_name,  
void *var_hdl);
```

Purpose

Create a trigger point signal. This is a special kind of signal only used by some third party vendors for marking trigger time of a simulation.

Refer to **ffw_CreateTriggerPointVarByVarIdcode()** for the detail of its usage and constraints.

Arguments

**obj*

Pointer to an FSDB Writer object.

var_name

Name of the trigger point var.

**var_hdl*

Pointer to a string that represents the full path scope name.

Return Value

A positive integer if successful, a negative integer (**FSDB_INVALID_VAR_IDCODE**) if fail.

ffw_CreateTriggerPointVarByVarIdcode

Interface

```
ffwVarMapId ffw_CreateTriggerPointVarByVarIdcode (ffwObject *obj, str_T var_name,  
fsdbVarIdcode var_idcode);
```

Purpose

Create a trigger point signal. This is a special kind of signal only used by some third party vendors for marking trigger time of a simulation.

- The values of trigger time are stored in a trigger point signal.
- An FSDB file could contain at most 1 trigger point signal. If more than one is created, only the first one is visible in nWave.
- Basically, trigger times are value changes of the trigger point signal. Therefore, no limitation is imposed on the number of trigger times.
- Trigger points are only visible in Debussy.qds.

Procedures of creating trigger point signal and trigger time:

Creating a trigger point signal is like creating ordinary variables. Only the API name is different.

```
ffwObject *obj;  
obj = ffw_Open("test.fsdb", FSDB_FT_VERILOG);  
ffw_BeginTree(obj);  
ffw_CreateTriggerPointVarByVarIdcode(ffw_obj, "trigger_point",  
    trigger_point_idcode);  
ffw_EndTree(obj);  
A trigger point signal might contain several trigger times during  
simulation. A trigger time should be recorded at the time step it  
happens. For example, if a trigger time is 300,  
ffw_CreateXCoorByHnL(ffw_obj, 0, 300);  
ffw_CreateTriggerPointVarValueByIdcode(ffw_obj, trigger_point_idcode);
```

Arguments

*obj

Pointer to an FSDB Writer object.

var_name

Name of the trigger point var.

var_idcode

Var idcode which identifies the var.

Return Value

A positive integer if successful, a negative integer (**FSDB_INVALID_VAR_IDCODE**) if fail.

ffw_CreateTriggerPointVarValueByHandle

Interface

```
fsdbGlitchDetectionType ffw_CreateTriggerPointVarValueByHandle (ffwObject *obj, void *var_hdl);
```

Purpose

Create a trigger point. Trigger point is a special kind of value only used by some third party vendors for marking trigger time of a simulation.

Refer to **ffw_CreateTriggerPointVarByVarIdcode()** for details.

Arguments

**obj*

Pointer to an FSDB Writer object.

**var_hdl*

Pointer to a string that represents the full path scope name.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas so that third parties can ignore the return value.

ffw_CreateTriggerPointVarValueByIdcode

Interface

```
fsdbGlitchDetectionType ffw_CreateTriggerPointVarValueByIdcode (ffwObject *obj, fsdbVarIdcode var_idcode);
```

Purpose

Create a trigger point. Trigger point is a special kind of value only used by some third party vendors for marking trigger time of a simulation.

Refer to **ffw_CreateTriggerPointVarByVarIdcode()** for the detail of its usage and constraints.

Arguments

**obj*

Pointer to an FSDB Writer object.

var_idcode

Var idcode which identifies the var.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

ffw_CreateUpscope

Interface

```
extern void ffw_CreateUpscope (ffwObject *obj);
```

Purpose

Make current scope go to its parent scope.

Argument

**obj*

Pointer to an FSDB Writer object.

ffw_CreateVarByHandle

Interface

```
extern ffwVarMapId ffw_CreateVarByHandle (ffwObject *obj, fsdbVarType var_type,  
fsdbVarDir var_dir, void *data_type_hdl, int lbitnum, int rbitnum, void *var_hdl, str_T  
var_name, fsdbBytesPerBit bpb);
```

Purpose

Create a var by handle. The handle is usually a pointer to an object. Since the virtual address of an object is unique, the application can take advantage of this to represent a unique var by the virtual address of an object.

Deprecated

This function has been replaced by **ffwCreateVarByVarHdl2()**. Although the **ffw_CreateVarByHandle** function still works, it will not be enhanced in the future.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`Var_type`

Var type. Refer to **fsdbVarType** defined in **fsdbShr.h**.

`Var_dir`

Var direction. Refer to **fsdbVarDir** defined in **fsdbShr.h**; if the direction is unknown, then specify it as implicit.

`*data_type_hdl`

Data type handle is a void pointer type cast on **fsdbDataType** or a handle given by the application. Refer to **fsdbDataType** defined in **fsdbShr.h**.

`lbitnum`

Left bit number.

`rbitnum`

Right bit number.

`*var_hdl`

Var handle, usually a virtual address of an object which can identify a unique var.

`var_name`

Pointer to a string that represents the var name.

`bpb`

An enumeration value which means how many bytes it takes to represent a bit of the var.

Return Value

Non-NULL if successful; NULL if fail; where **ffwVarMapId** is defined in **ffwAPI.h**.

Note:

If users would like to put '/' in signal names for files in Epic format, '\' (backslash) must be added as the leading character of the signal name. The '\' works as an "escaped character" and won't be shown in the *Get Signals* form and the signal pane of the *nWave* window.

ffw_CreateVarByHandleAndPath

Interface

```
extern ffwVarMapId ffw_CreateVarByHandleAndPath (ffwObject *obj, fsdbVarType
var_type, fsdbVarDir var_dir, void *data_type_hdl, int lbitnum, int rbitnum, void
*var_hdl, str_T var_name, fsdbBytesPerBit bpb, str_T full_path_scope_name, str_T
scope_separator)
```

Purpose

Create a var by handle, full path scope name and scope separator. The handle is usually a pointer to an object. Since the virtual address of an object is unique, the application can take advantage of this to represent a unique var by the virtual address of an object.

Deprecated

This function has been replaced by **ffwCreateVarByVarHdlAndPath2()**. Although the **ffw_CreateVarByHandleAndPath** function still works, it will not be enhanced in the future.

Arguments

**obj*

Pointer to an FSDB Writer object.

var_type

Var type. Refer to **fsdbVarType** defined in **fsdbShr.h**.

var_dir

Var direction. Refer to **fsdbVarDir** defined in **fsdbShr.h**; if the direction is unknown, then specify it as implicit.

`*data_type_handle`

Data type handle is a void pointer type cast on **fsdbDataType** or a handle given by the application. Refer to **fsdbDataType** defined in **fsdbShr.h**.

`lbitnum`

Left bit number.

`rbitnum`

Right bit number.

`*var_hdl`

Var handle, usually a virtual address of an object which can identify a unique var.

`var_name`

Pointer to a string that represents the var name.

`bpb`

An enumeration value which means how many bytes it takes to represent a bit of the var.

`full_path_scope_name`

Pointer to a string that is a full path scope name where the var belongs.

`scope_separator`

Pointer to a string that represents the scope separator.

Return Value

Non-NULL if successful; NULL if fail; where **ffwVarMapId** is defined in **ffwAPI.h**.

Limitation

This API cannot be used between

`ffw_CreateArrayBegin()`

`/ffw_CreateArrayEnd()`

`ffw_CreateRecordBegin()/ffw_CreateRecordEnd()`

`ffw_CreateStructBegin()/ffw_CreateStructEnd()`

It might create scopes inside the pair and cause the wrong hierarchy.

ffw_CreateVarByldcode

Interface

```
extern fsdbVarldcode ffw_CreateVarByldcode (ffwObject *obj, fsdbVarType var_type,  
fsdbVarDir var_dir, void *data_type_handle, int lbitnum, int rbitnum, fsdbVarldcode idcode,  
str_T var_name, fsdbBytesPerBit bpb);
```

Purpose

Create a var (signal) by idcode, while the idcode is given by the application.

Deprecated

This function has been replaced by **ffwCreateVarByVarldcode2()**. Although the **ffw_CreateVarByldcode** function still works, it will not be enhanced in the future.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`var_type`

Var type. Refer to **fsdbVarType** defined in **fsdbShr.h**.

`var_dir`

Var direction. Refer to **fsdbVarDir** defined in **fsdbShr.h**; if the direction is unknown, then specify it as implicit.

`*data_type_handle`

Data type handle is a void pointer type cast on **fsdbDataType** or a handle given by the application. Refer to **fsdbDataType** defined in **fsdbShr.h**.

`lbitnum`

Left bit number.

`rbitnum`

Right bit number.

`idcode`

The idcode which represents the var.

`var_name`

Pointer to a string that represents the var name.

`bpb`

An enumeration value which means how many bytes it takes to represent a bit of the var.

Return Value

A positive integer on success; a negative integer (**FSDB_INVALID_VAR_IDCODE**) on failure.

Note:

If users would like to put '/' in signal names for files in Epic format, '\ ' (backslash) must be added as the leading character of the signal name. The '\ ' works as an "escaped character" and won't be shown in the *Get Signals* form and the signal pane of the *nWave* window.

ffw_CreateVarByIdcodeAndPath

Interface

```
extern fsdbVarIdcode ffw_CreateVarByIdcodeAndPath (ffwObject *obj,
fsdbVarType var_type, fsdbVarDir var_dir, void *data_type_handle, int lbitnum,
int rbitnum, fsdbVarIdcode idcode, str_T var_name, fsdbBytesPerBit bpb, str_T
full_path_scope_name, str_T scope_separator);
```

Purpose

Create a var by idcode, full path scope name and scope separator. When the FSDB Writer parses the full path var name, once a scope is figured out, it creates a sub-scope under current scope, based on the newly figured out scope, and then current scope goes to the newly figured out scope.

Deprecated

This function has been replaced by **ffwCreateVarByVarIdcodeAndPath2()**. Although the **ffw_CreateVarByIdcodeAndPath** function still works, it will not be enhanced in the future.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`var_type`

Var type. Refer to **fsdbVarType** defined in **fsdbShr.h**.

`var_dir`

Var direction. Refer to **fsdbVarDir** defined in **fsdbShr.h**; if the direction is unknown, then specify it as implicit.

`*data_type_handle`

Data type handle is a void pointer type cast on **fsdbDataType** or a handle given by the application. Refer to **fsdbDataType** defined in **fsdbShr.h**.

`lbitnum`

Left bit number.

`rbitnum`

Right bit number.

`idcode`

The idcode which represents the var.

`var_name`

Pointer to a string that represents the var name.

`bpb`

An enumeration value which means how many bytes it has to take to represent a bit of the var.

`full_path_scope_name`

A pointer to a string that is a full path scope name where the var belongs.

`scope_separator`

A pointer to a string that represents the scope separator.

Return Value

A positive integer if success; a negative integer (**FSDB_INVALID_VAR_IDCODE**) if fail.

Limitation

This API cannot be used between

`ffw_CreateArrayBegin()/ffw_CreateArrayEnd()`

`ffw_CreateRecordBegin()/ffw_CreateRecordEnd()`

`ffw_CreateStructBegin()/ffw_CreateStructEnd()`

It might create scopes inside the pair and cause the hierarchy wrong.

ffw_CreateVarValueByHandle

Interface

```
extern fsdbGlitchDetectionType ffw_CreateVarValueByHandle (ffwObject *obj, void
*var_hdl, byte_T *var_val);
```

Purpose

Dump a value change at current xtag, while the value change belongs to the var whose handle is specified by the second parameter, `var_hdl`.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*var_hdl`

Var handle which identifies the var.

`*var_val`

Pointer to a byte array which contains the value change of the var.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

ffw_CreateVarValueByIdcode

Interface

```
extern fsdbGlitchDetectionType ffw_CreateVarValueByIdcode (ffwObject *obj,  
fsdbVarIdcode var_idcode, byte_T *var_val);
```

Purpose

Dump a value change at current xtag, while the value change belongs to the var whose idcode is specified by the second parameter, var_idcode.

Arguments

*obj

Pointer to an FSDB Writer object.

var_idcode

The idcode which represents the var.

*var_val

Pointer to a byte array which contains the value change of the var.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

ffw_CreateVarValueByVarMapId

Interface

```
fsdbGlitchDetectionType ffw_CreateVarValueByVarMapId (ffwObject *obj, ffwVarMapId  
var_map_id, byte_T *var_val);
```

Purpose

Dump a value change at current xtag, while the value change belongs to the var whose **VarMapId** is specified by the second parameter, var_map_id.

Arguments

**obj*

Pointer to an FSDB Writer object.

var_map_id

VarMapId which identifies the var. It is returned by some “ByHandle” APIs such as **ffw_CreateVarByHandle()**.

**var_val*

Pointer to a byte array which contains the value change of the var.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

ffw_CreateXCoorByHnL

Interface

```
extern void ffw_CreateXCoorByHnL (ffwObject *obj, fsdbTag32 H, fsdbTag32 L);
```

Purpose

Create a xtag where has value changes occurred. The FSDB Writer splits 64-bit integer into high and low 32-bit unsigned integers, the high 32-bit unsigned integer is denoted as H, the low 32-bit unsigned integer is denoted as L; it takes H and L to act as 64-bit unsigned integer.

Arguments

**obj*

Pointer to an FSDB Writer object.

H

High 32-bit unsigned integer.

L

Low 32-bit unsigned integer.

ffw_Destroy

Interface

```
extern void ffw_Destroy (ffwObject *obj);
```

Purpose

Destroy all the data structures of an FSDB Writer object, and deletes FSDB file.

Argument

**obj*

Pointer to an FSDB Writer object.

ffw_DisableDumpOff

Interface

```
extern void ffw_DisableDumpOff (ffwObject *obj);
```

Purpose

Disable dumpoff (enable dumpon) from now on.

Argument

**obj*

Pointer to an FSDB Writer object.

ffw_DisableSyncTree

Interface

```
extern void ffw_DisableSyncTree (ffwObject *obj);
```

Purpose

Prevent the FSDB Writer from writing the newly created tree to disk when **ffw_EndTree()** is called. This could reduce a lot of disk I/O overhead when creating a lot of new hierarchies/ signals on the fly when running in post-processing mode. Ensure that you enable it when all design trees are created. Otherwise, some data might not be written to disk and lost. It is enabled by default to ensure data integrity.

Argument

`*obj`

Pointer to an FSDB Writer object.

ffw_DTCreateEnum

Interface

```
extern fsdbDataTypeIdcode ffw_DTCreateEnum (ffwObject *obj, void *data_type_hdl,  
ushort_T literal_count, str_T literal_arr[]);
```

Purpose

Create an enumeration data type. The FSDB Writer will keep the enumeration definition and data type handle, and then will return a data type idcode to identify this enumeration definition. The application should keep the data type handle which can guide the FSDB Writer to locate the enumeration definition.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*data_type_hdl`

Data type handle, given by the application.

`literal_count`

Number of literal in literal array.

`literal_arr`

Literal array.

Return Value

Positive data type idcode if successful; negative data type idcode if fail.

ffw_DTCreateEnum2

Interface

```
fsdbDataTypeIdcode ffw_DTCreateEnum2 (ffwObject *obj, void *data_type_hdl, str_T  
name, ushort_T literal_count, uint_T val_len, fsdbEnumValueType val_type, str_T  
literal_arr[], byte_T val_arr[]);
```

Purpose

Create an enumeration data type. The FSDB Writer will keep the enumeration definition and data type handle, and then will return a data type idcode to identify this enumeration definition. The application should keep the data type handle which can guide the FSDB Writer to locate the enumeration definition.

Deprecated

This function has been replaced by **ffw_DTCreateEnum3()**. Although the **ffw_DTCreateEnum2** function still works, it will not be enhanced in the future.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*data_type_hdl`

Data type handle, given by the application.

`name`

Data type name.

`literal_count`

The number of literals in the argument `literal_arr`.

`val_len`

Size of an enum value.

`val_type`

Value type of the enum value. Refer to the definition of `fsdbEnumValueType` for a full list. It is something like **FSDB_ENUM_VALUE_TYPE_INTEGER**.

`literal_arr`

Enum literal array

`val_arr`

Enum value array

Return Value

Positive data type idcode if successful; negative data type idcode if fail.

fw_DTCreateEnum3

Interface

```
extern fsdbDataTypeIdcode ffw_DTCreateEnum3(ffwObject *obj, ffwEnumDTArg
*enum_arg);
```

Purpose

Create an enumeration data type. The FSDB Writer will keep the enumeration definition and data type handle, and then will return a data type idcode to identify this enumeration definition. The application should keep the data type handle which can guide the FSDB Writer to locate the enumeration definition.

- System Verilog enum is modeled as a register instead of an idx that VHDL enum uses.
- An enum value may not have a corresponding literal value in the enum definition.
- **ffw_DTCreateEnum3** removes the limitation of **ffw_DTCreateEnum2()**.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*enum_arg`

Pointer to enum data type argument. Refer to **ffwEnumDTArg** defined in **ffwAPI.h**.

Return Value

Positive data type idcode if successful; negative data type idcode if fail.

ffw_DTCreateFloating

Interface

```
extern fsdbDataTypeIdcode ffw_DTCreateFloating (ffwObject *obj, void *data_type_hdl,  
double lbound, double rbound);
```

Purpose

Create floating data type, its minimum and maximum value are denoted by lbound and rbound respectively.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*data_type_hdl`

Data type handle, which identifies the 64-bit integer definition.

`lbound`

Left bound of floating data type.

`rbound`

Right bound of floating data type.

Return Value

Positive data type idcode if successful; negative data type idcode if fail.

ffw_DTCreateInt

Interface

```
extern fsdbDataTypeIdcode ffw_DTCreateInt (ffwObject *obj, void *data_type_hdl, uint_T lbound, uint_T rbound);
```

Purpose

Create an integer data type. The FSDB Writer will keep the integer definition and data type handle, assign a data type idcode to identify this integer definition; what the application should keep is the data type handle, which can guide The FSDB Writer to locate the integer definition.

Arguments

obj

Pointer to an FSDB Writer object.

data_type_hdl

Integer definition.

lbound

The left bound of this integer definition.

rbound

The right bound of this integer definition.

Return Value

Positive data type idcode if successful; negative data type idcode if fail.

ffw_DTCreateIntHnL

Interface

```
extern fsdbDataTypeIdcode ffw_DTCreateIntHnL (ffwObject *obj, void *data_type_hdl);
```

Purpose

Create 64-bit integer data type, while the high 32-bit is denoted by H, low 32-bit is denoted by L.

Arguments

**obj*

Pointer to an FSDB Writer object.

**data_type_hdl*

Data type handle, which identifies the 64-bit integer definition.

Return Value

Positive data type idcode if successful; negative data type idcode if fail.

ffw_DTCreatePhysicalBegin

Interface

```
extern fsdbDataTypeIdcode ffw_DTCreatePhysicalBegin (ffwObject *obj, void  
*data_type_hdl, double lbound, double rbound, str_T base_unit);
```

Purpose

Create physical begin data type, this API is corresponding to the physical begin definition in VHDL, which defines the left bound, right bound and the base unit of a physical data type.

Arguments

**obj*

Pointer to an FSDB Writer object.

**data_type_hdl*

Data type handle, which identifies the physical begin definition.

lbound

Left bound of physical begin definition.

`rbound`

Right bound of physical begin definition.

`base_unit`

Pointer to a string that represents the base unit.

Return Value

Positive data type idcode if successful; negative data type idcode if fail.

ffw_DTCreatePhysicalEnd

Interface

```
extern void ffw_DTCreatePhysicalEnd (ffwObject *obj);
```

Purpose

End up physical definition.

Argument

`*obj`

Pointer to an FSDB Writer object.

ffw_DTCreatePhysicalSecondaryUnit

Interface

```
extern void ffw_DTCreatePhysicalSecondaryUnit (ffwObject *obj, double multiple, str_T  
identifier);
```

Purpose

Specify the secondary unit if any.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`multiple`

Double value which represents multiple.

`identifier`

Pointer to a string that represents identifier.

ffw_DumpSeqNumTurnOn

Interface

```
extern void ffw_DumpSeqNumTurnOn (ffwObject *obj);
```

Purpose

Turn *on* sequence number dumping of value change so that the application can tell the sequence of value changes happened at the same time. The FSDB Writer has a sequence counter to record the sequence number of each value change happened at the same time; the sequence counter is incremented by one once there comes a value change; the sequence counter is reset when the FSDB Writer moves to the next time where has value changes happened.

Argument

`*obj`

Pointer to an FSDB Writer object.

ffw_DumpStrengthTurnOn

Interface

```
extern void ffw_DumpStrengthTurnOn (ffwObject *obj);
```

Purpose

Tell the FSDB Writer that value change contains strength information. This is for Verilog only. The FSDB Writer assumes value change does not contain strength information by default.

Argument

**obj*

Pointer to an FSDB Writer object.

ffw_EnableDumpOff

Interface

```
extern void ffw_EnableDumpOff (ffwObject *obj);
```

Purpose

Enable dumpoff from now on.

Argument

**obj*

Pointer to an FSDB Writer object.

ffw_EnableSyncTree

Interface

```
extern void ffw_EnableSyncTree (ffwObject *obj);
```

Purpose

Enable tree synchronization that was previously disabled by **ffw_DisableSyncTree()**. See **ffw_DisableSyncTree()**.

Argument

`*obj`

Pointer to an FSDB Writer object.

ffw_EndTransaction

Interface

```
extern fsdbRC ffw_EndTransaction (ffwObject *obj, fsdbTransId trans_id, fsdbXTag  
end_time, fsdbAttrHdlVal attr_arr[], uint_T attr_count);
```

Purpose

End a transaction that is previously created by **ffw_BeginTransaction()**.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`trans_id`

ID of the transaction that is to be ended.

`end_time`

The end time of the transaction. It could be current time step or a time step in the past or future. If `end_time` of any transaction is in the future and greater than the FSDB file's max time, the file's max time will be adjusted to the value of `end_time` automatically.

`attr_arr`

An array of attributes of the transaction that are to be created during `end_time`. Each element is a handle/value pair of an attribute. `fsdbAttrHdlVal` is defined as

```
struct fsdbAttrHdlVal {  
    fsdbAttrHdl hdl;  
    byte_T *value;  
};  
typedef struct fsdbAttrHdlVal fsdbAttrHdlVal;
```

`attr_count`

Number of attributes in `attr_arr`.

Return Value

FSDB_RC_SUCCESS upon success.

FSDB_INVALID_TRANS_ID if the id is **FSDB_INVALID_TRANS_ID** or has not been created yet.

FSDB_RC_FAILURE otherwise.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/transaction_recording.cpp
```

ffw_EndTree

Interface

```
extern void ffw_EndTree (ffwObject *obj);
```

Purpose

End of tree creation. It closes a tree creation. The FSDB Writer will calculate how many signals have been created in this tree when **ffw_EndTree** is called.

Argument

*obj

Pointer to an FSDB Writer object.

ffw_GetActiveObject

Interface

```
extern ffwObject* ffw_GetActiveObject(void);
```

Purpose

This API returns the active writer object, which is the object that is last created.

Return Value

A pointer to the active FSDB Writer object.

Example

```
ffw_obj_1 = ffw_Open("fname_1", FSDB_VT_VERILOG);  
ffw_obj_2 = ffw_Open("fname_2", FSDB_VT_VERILOG);  
ffw_obj_3 = ffw_Open("fname_3", FSDB_VT_VERILOG); // <-Active Object  
  
ffwObject* ptr_to_active_writer_object;  
ptr_to_active_writer_object = ffw_GetActiveObject(); // will return  
ffw_obj_3
```

ffw_GetActiveObjectName

Interface

```
extern str_T ffw_GetActiveObjectName(void);
```

Purpose

This API returns the file name of the active writer object, which is the object that is last created.

Return Value

A string pointer to the file name of the active file object.

ffw_GetAttrByHdl

Interface

```
extern ffwAttr* ffw_GetAttrByHdl(ffwObject *obj, fsdbAttrHdl hdl);
```

Purpose

Get the properties of an already-created attribute by its attribute handle. The properties of an attribute can be retrieved as long as after it was created by **ffw_CreateAttr()**.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`hdl`

Handle of the attribute that is to be retrieved.

Return Value

Non-Null if successful; NULL otherwise.

ffw_GetDataTypeCreationReady

Interface

```
extern void ffw_GetDataTypeCreationReady (ffwObject *obj);
```

Purpose

Initialize necessary data structures for data type creation. By default, the FSDB Writer does not initialize the data structures for data type creation, so if the application would like to dump user-defined data type, then it must call this API.

Argument

`*obj`

Pointer to an FSDB Writer object.

ffw_GetFileName

Interface

```
extern str_T ffw_GetFileName(ffwObject *obj);
```

Purpose

Get an FSDB file name.

Argument

*obj

Pointer to an FSDB Writer object.

Return Value

A string pointer to the file name of the file object.

ffw_GetFsdbDumperVersion

Interface

```
extern str_T ffw_GetFsdbDumperVersion(ffwObject *obj);
```

Purpose

Get the FSDB dumper version.

Argument

*obj

Pointer to an FSDB Writer object.

Return Value

Pointer to a string that represents the FSDB dumper version.

ffw_GetFsdbVersion

Interface

```
extern str_T ffw_GetFsdbVersion();
```

Purpose

Get current FSDB version. Debussy has its own release version, while FSDB has its own release version too; FSDB version is used to identify the format change of FSDB so that the FSDB Reader can know how to read the FSDB file based on its version.

Return Value

Pointer to a string that represents the FSDB version.

ffw_GetGlitchNum

Interface

```
extern ushort_T ffw_GetGlitchNum (ffwObject *obj);
```

Purpose

Get current setting of maximum glitch number.

Argument

**obj*

Pointer to an FSDB Writer object.

Return Value

The max glitch number has been set by **ffw_SetGlitchNmber** or **FSDB_MAX_GLITCH_NUM**.

ffw_GetIdcodeByHandle

Interface

```
extern fsdbVarIdcode ffw_GetIdcodeByHandle (ffwObject *obj, void *handle);
```

Purpose

Return var idcode by providing the var handle.

Arguments

**obj*

Pointer to an FSDB Writer object.

**handle*

Var handle, usually a virtual address of an object which can identify a unique var.

Return Value

The idcode, which represents the var specified by handle.

ffw_GetIdent

Interface

```
extern str_T ffw_GetIdent();
```

Purpose

Get identification string defined by Novas, which can be told what date the FSDB Writer library was compiled, what version it is, and so on.

Return Value

Pointer to a string that contains useful information for Novas.

ffw_GetMaxVarIdcode

Interface

```
extern fsdbVarIdcode ffw_GetMaxVarIdcode (ffwObject *obj);
```

Purpose

Get current maximum var idcode, this value is equivalent to how many unique signals defined in the FSDB file.

Argument

**obj*

Pointer to an FSDB Writer object.

Return Value

Positive var idcode if successful; negative var idcode if fail.

ffw_GetMaxXTag

Interface

```
extern fsdbRC ffw_GetMaxXTag (ffwObject *obj, fsdbXTag *max_xtag);
```

Purpose

Get the maximum xtag (i.e. the last xtag) created in the FSDB file. If **ffw_ResetXCoor()** is called, maximum xtag will be reset to 0 and the recording of maximum xtag will restart.

Arguments

**obj*

Pointer to an FSDB Writer object.

**max_xtag*

An fsdbXTag variable the caller passes in for the FSDB Writer to return current maximum xtag in the FSDB file.

Return Value

FSDB_RC_SUCCESS if successful. **FSDB_RC_FAILURE** otherwise.

ffw_GetMinXTag

Interface

```
extern fsdbRC ffw_GetMinXTag (ffwObject *obj, fsdbXTag *min_xtag);
```

Purpose

Get the minimum xtag (i.e. the first xtag) created in the FSDB file.

Arguments

**obj*

Pointer to an FSDB Writer object.

**min_xtag*

An fsdbXTag variable the caller passes in for the FSDB Writer to return current minimum xtag in the FSDB file.

Return Value

FSDB_RC_SUCCESS if successful. **FSDB_RC_FAILURE** otherwise.

ffw_GetRefCount

Interface

```
extern unit_T ffw_GetRefCount (ffwObject *obj);
```

Purpose

There could be more than one pointer pointing to the same writer object. This API returns how many pointers are currently pointing to the specified writer object.

Reference count of a writer object will be decremented by one when the **ffw_Close()** is called. The FSDB file will not be closed until its reference count equals to 0.

Argument

**obj*

Pointer to an FSDB Writer object.

Return Value

Number of pointers pointing to the specified writer object.

Example

```
ptr1_to_the_same_writer = ffw_Open("same_writer_fname", FSDB_VT_VERILOG);  
ptr2_to_the_same_writer = ffw_Open("same_writer_fname", FSDB_VT_VERILOG);  
ptr3_to_the_same_writer = ffw_Open("same_writer_fname", FSDB_VT_VERILOG);  
printf("%d pointers are currently referencing to the FSDB file(%s)",  
ffw_GetRefCount(ptr1_to_the_same_writer), "same_writer_fname" );
```

>3 pointers are currently referencing to the FSDB file

ffw_GetStreamByHdl

Interface

```
extern ffwStreamInfo* ffw_GetStreamByHdl (ffwObject *obj, fsdbStreamHdl hdl);
```

Purpose

Get the properties of an already-created stream by its stream handle. The properties of a stream can be retrieved as long as after it's created by **ffw_CreateStream()**.

Arguments

**obj*

Pointer to an FSDB Writer object.

hdl

Handle of the stream that is to be retrieved.

Return Value

Non-null if successful; NULL otherwise.

ffw_HasGlitchNumBeenSet

Interface

```
extern bool_T ffw_HasGlitchNumBeenSet (ffwObject *obj);
```

Purpose

Check if the glitch number is already set via void ffw_SetGlitchNum (ffwObject *obj, ushort_T number).

Arguments

*obj

A pointer to an FSDB Writer object.

Return Value

TRUE if **ffw_SetGlitchNum** has been called; FALSE otherwise.

ffw_InfoSuppress

Interface

```
void ffw_InfoSuppress (bool_T bSuppress);
```

Purpose

By calling this function with bSuppress set to TRUE, all informative messages are suppressed and not printed on console.

This change takes effect for all writer objects in the same process.

Arguments

`bSuppress`

TRUE when all informative messages need to be suppressed, **FALSE** otherwise.

See Also

`ffw_RegisterInfoCBFunc()`

ffw_Init

Interface

```
extern void ffw_Init();
```

Purpose

Initialize necessary data structures for the FSDB Writer, such as data structures for compression. This API is obsolete from Debussy 5.0 and it is kept for backward compatibility.

ffw_IsDumpOff

Interface

```
extern bool_T ffw_IsDumpOff(ffwObject *obj);
```

Purpose

Check whether dumpoff is enabled.

Argument

`*obj`

Pointer to an FSDB Writer object.

Return Value

True if dump off is enabled; else false.

ffw_Open

Interface

```
extern ffwObject* ffw_Open (str_T fsdb_fname, fsdbFileType file_type);
```

Purpose

Open an FSDB file and returns a pointer to an FSDB Writer object; the returned FSDB Writer object is very important since most API take it as the first parameter.

Deprecated

This function has been replaced by **ffwOpenFile2()**. Although the **ffw_Open** function still works, it will not be enhanced in the future.

Arguments

fsdb_fname

File name of the FSDB file.

file_type

File type of the FSDB file, defined as fsdbFileType in **fsdbShr.h**.

Return Value

Non-null if successful; NULL otherwise.

ffw_OpenAutoSwitchFile

Interface

```
extern ffwObject* ffw_OpenAutoSwitchFile (ffwAutoSwitchFileRec *file_rec);
```

Purpose

Open an auto-switch FSDB file. When the application would like to open an auto-switch FSDB file, it must specify the file size limit which is the criterion to determine when the FSDB Writer should switch current FSDB file to the next one; moreover, the application has to register the callback function which will be called when file size limit is hit, along with a client data which will be passed via the registered callback function. When a xtag is created, the FSDB Writer checks the file size to see if the file size limit is hit or not; if hit, then the registered callback function will be called, which tells the application that it should switch current FSDB file to the next one. In the callback function, the application has to close current FSDB file, open another FSDB file (switched FSDB file), and then dumps the value change of all signals at that moment, which are the initial value changes of the switched FSDB file.

Argument

`*file_rec`

Pointer to auto-switch file object.

Return Value

Non-null if successful; NULL otherwise.

ffw_RecordTransactionAttribute

Interface

```
extern fsdbRC ffw_RecordTransactionAttribute (ffwObject *obj, fsdbTransId trans_id,
fsdbAttrHdlVal attr_arr[], uint_T attr_count);
```

Purpose

Add more attributes to a transaction during transaction evaluation. For a transaction, **ffw_RecordTransactionAttribute()** could be called for many times as long as the transaction has not been ended yet. As long as **ffw_RecordTransactionAttribute()** is called between corresponding **ffw_BeginTransaction()** and **ffw_EndTransaction()**, it is not important at which time step it is called. A transaction only has begin_time and end_time. FSDB will not record at which time step the API is called.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`trans_id`

ID of the transaction that is to be ended.

`attr_arr`

An array of attributes of the transaction that are to be added. Each element is a handle/value pair of an attribute. `fsdbAttrHdlVal` is defined as:

```
struct fsdbAttrHdlVal {  
    fsdbAttrHdl hdl;  
    byte_T *value;  
};  
typedef struct fsdbAttrHdlVal fsdbAttrHdlVal;
```

`attr_count`

Number of attributes in `attr_arr`.

Return Value

FSDB_RC_SUCCESS upon success.

FSDB_INVALID_TRANS_ID if the id is **FSDB_INVALID_TRANS_ID** or has not been created yet.

FSDB_RC_FAILURE otherwise.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/transaction_recording.cpp
```

ffw_RegisterAutoSwitchFileClientData

Interface

```
extern void ffw_RegisterAutoSwitchFileClientData (ffwObject *obj, void *client_data);
```

Purpose

Register an auto-switch client data that will be passed back to the application in auto-switch callback function.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*client_data`

Pointer to auto-switch client data.

ffw_RegisterDumperCBFunc

Interface

```
extern void ffw_RegisterDumperCBFunc (ffwObject *obj, ffwDumperCBFunc cb);
```

Purpose

Provide a callback scheme that is used to notify dumpers when some events of the FSDB Writer object happen (Dump On / Dump Off / Switch File).

To get such event callbacks, dumper needs to register dumper callback function using this API, and its usage follows:

1. Dumper should create a callback function with the prototype defined in **ffwAPI.h**

```
void (*ffwDumperCBFunc)(ffwDumperCBType dumper_cb_type, void  
*dumper_cb_data);
```

In the callback function, dumper can define how it plans to deal with different call back event according to the “dumper_cb_type”.

2. Register the aforementioned callback to the FSDB Writer object using

```
void ffw_RegisterDumperCBFunc(ffwObject *obj, ffwDumperCBFunc cb);
```

Note:

When one of the many dumpers call **ffw_EnableDumpOff()** or **ffw_DisableDumpOn()**, the FSDB Writer invokes the call back functions with call back type (**FFW_DUMPER_CBT_DUMPON /**

FFW_DUMPER_CBT_DUMPOFF) that "ALL" the dumpers registered, including the dumper who actually call **ffw_EnableDumpoff()** or **ffw_DisableDumpOn()** **FFW_DUMPER_CBT_SWITCHFILE** Registered CB functions will be invoked with this call back type when one of the dumper call **ffw_SwitchFile()** and the FSDB Writer ACTUALLY FINISHED the switching process.

Arguments

**obj*

Pointer to an FSDB Writer object.

cb

User-defined callback function that will handle the error message.

ffw_RegisterErrorCBFunc

Interface

```
extern void ffw_RegisterErrorCBFunc (fsdbMsgCBFunc cb)
```

Purpose

Redirect error messages issued by the FSDB Writer to the user-defined callback function *cb*. In this way, you will not see messages like:

ERROR This is an error message

Instead, the user-defined callback function can decide how to display or handle this error message. After the error message is handled, the FSDB Writer will terminate the process since the "error" here means a fatal error and the program could not continue execution any more.

If *cb* is set to NULL, the FSDB Writer will regain the control to display the error message in the default format.

Note:

This change will take effect for all writer objects in the same process.

Argument

cb

User-defined callback function that will handle the error message.

ffw_RegisterInfoCBFunc

Interface

```
extern void ffw_RegisterInfoCBFunc (fsdbMsgCBFunc cb);
```

Purpose

Redirect all informative messages issued by the FSDB Writer to the user-defined callback function *cb*. In this way, you will not see messages like

This FSDB file (xxx.fsdb) is close to 2G. Registered 2G callback function is called.

Instead, the user-defined callback function can decide how to display or handle informative message.

If *cb* is set to NULL, the FSDB Writer will regain the control to display the informative message in the default format.

Note:

This change will take effect for all writer objects in the same process.

Arguments

cb

User-defined callback function that will handle the informative messages.

See Also

ffw_InfoSuppress()

ffw_RegisterTwoGCBFunc

Interface

```
extern void ffw_RegisterTwoGCBFunc (ffwObject *obj, ffwTwoGCBFunc cb);
```

Purpose

Register a callback function to the writer object specified by the first parameter *obj*. The registered callback function will be called when the size of the FSDB file is about to exceed two gigabytes.

Note:

This application will not generate a callback on systems with Large File Support (LFS).

Arguments

**obj*

Pointer to an FSDB Writer object.

cb

A callback function given by the application.

ffw_RegisterTwoGCBFuncDontCareLFS

Interface

```
void ffw_RegisterTwoGCBFuncDontCareLFS (ffwObject *obj, ffwTwoGCBFunc cb);
```

Purpose

Register a callback function for the writer object specified by the first parameter *obj*. The registered callback function will be called when the FSDB file size exceeds two gigabytes. The application can choose to process or ignore the callback when it occurs.

Unlike the callback registered with **ffw_RegisterTwoGCBFunc**, this callback will be generated even on systems with Large File Support (LFS).

Arguments

**obj*

Pointer to an FSDB Writer object.

cb

A callback function given by the application.

ffw_RegisterWarnCBFunc

Interface

```
extern void ffw_RegisterWarnCBFunc (fsdbMsgCBFunc cb)
```

Purpose

Redirect warning messages issued by the FSDB Writer to the user-defined callback function *cb*. In this way, you will not see messages like:

WARN This is a warning message

Instead, the user-defined callback function can decide how to display or handle this warning message.

If *cb* is set to NULL, the FSDB Writer will regain the control to display the warning message in the default format.

Note:

This change will take effect for all writer objects in the same process.

Argument

cb

User-defined callback function that will handle the warning message.

ffw_RegisterWriteErrorCB

Interface

```
extern void ffw_RegisterWriteErrorCB (ffwWriteErrorCB write_error_cb);
```

Purpose

Register a write error callback function which will be called when write system call fails. The FSDB Writer has pre-defined write error callback function by default, and it can be overridden by the application defined one, via the call of this API.

Argument

`write_error_cb`

Application defined write error callback function.

ffw_ResetXCoor

Interface

```
extern void ffw_ResetXCoor (ffwObject *obj);
```

Purpose

Rewind current xtag to the initial one. This API is used to create value changes via signal based scheme, which means the application dumps all value changes of one signal, rewinds the xtag, and then followed by another signal.

Argument

`*obj`

Pointer to an FSDB Writer object.

ffw_SetAnnotation

Interface

```
extern void ffw_SetAnnotation (ffwObject *obj, str_T annotation);
```

Purpose

Write annotation to FSDB file. The application can write annotation to FSDB file via this API, and the annotation is written to FSDB file only when FSDB file is going to be closed; if FSDB file is still in processing, then annotation is kept in memory and not written yet. The maximum length of annotation is defined as **FSDB_ANNOTATION_LEN** in **fsdbShr.h** header file.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`annotation`

Pointer to a string that is the annotation.

ffw_SetAsErrorTransaction

Interface

```
fsdbRC ffw_SetAsErrorTransaction (ffwObject *obj, fsdbTransId trans_id);
```

Purpose

Set a transaction as an error transaction.

Note:

This API must only be called after the target error transaction has **begun(ffw_BeginTransaction())** and before it has **ended(ffw_EndTransaction())**.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`trans_id`

Id of the transaction that is to be marked as error transaction. This id is returned by **ffw_BeginTransaction()**.

Return Value

FSDB_RC_SUCCESS upon success.

FSDB_INVALID_TRANS_ID if the id is **FSDB_INVALID_TRANS_ID** or has not been created yet.

FSDB_RC_FAILURE otherwise.

ffw_SetBusClock

Interface

```
extern fsdbRC ffw_SetBusClock (ffwObject *obj, fsdbBusHdl bus, str_T clk_path,  
fsdbClockType clk_type);
```

Purpose

Set the clock of a bus. This defines when data is sampled on a bus.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`bus`

Handle of the bus to which the clock is defined.

`clk_path`

Full hierarchy path of the clock.

`clk_type`

Specify the clock edge type. Refer to the definition of **fsdbClockType**.

Return Value

FSDB_RC_SUCCESS upon success. **FSDB_RC_FAILURE** otherwise.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/bus_tr_recording.cpp
```

See Also

`ffw_CreateBus()`

`ffw_SetBusParameter()`

`ffw_AddBusStream()`

ffw_SetCompressionMethod

Interface

```
extern void ffw_SetCompressionMethod (ffwObject *obj, int method);
```

Purpose

Select compression method for compressing value changes. In order to resolve some linking problems with third parties, this API is obsolete from Debussy 5.0 and kept for backward compatibility.

Arguments

obj

Pointer to an FSDB Writer object.

method

Compression method.

ffw_SetDesignType

Interface

```
extern void ffw_SetDesignType (ffwObject *obj, fsdbDesignType design_type);
```

Purpose

Specify the design type of the FSDB file.

Arguments

obj

Pointer to an FSDB Writer object.

design_type

Design type. Refer to **fsdbDesignType** in **fsdbShr.h**.

ffw_SetDumpLimit

Interface

```
extern void ffw_SetDumpLimit (ffwObject *obj, uint_T size);
```

Purpose

Set dump limit of an FSDB file in mega bytes. Once dump limit is turned for an FSDB file, you cannot turn it off. When dump limit is turned on and the size of the FSDB file reaches the specified limit, the FSDB Writer will recycle the space from the beginning of this file and the value changes in the recycled space will no longer exist. Because dump limit is only for value changes, the file size will not be exactly the same with the specified size. It may be slightly larger or smaller. The minimum value of size is 32 on all platforms. The maximum value of size is 1800 on 32-bit platforms because a file cannot exceed 2GB on these machines. As for 64-bit platforms, there is no such limitation. You should avoid using dynamic trees (i.e. trees created after any value change is created) when dump limit is turned on. Otherwise, the FSDB file will immediately grow to the specified dump limit size because such trees are placed in the end of the file, even if there are no value changes in between. Notice that when you use dump limit, there will be slight performance penalty and you will also lose recycled value changes.

Arguments

obj

Pointer to an FSDB Writer object.

size

Dump limit size in mega bytes.

ffw_SetGlitchNum

Interface

```
extern void ffw_SetGlitchNum (ffwObject *obj, ushort_T number);
```

Purpose

Write maximum glitch number to the FSDB file. If the maximum glitch number is n , then the first $(n - 1)$ glitches and the last one will be recorded in the FSDB file, while setting **FSDB_ENV_MAX_GLITCH_NUM** environment variable to value n has the same effect.

Arguments

**obj*

Pointer to an FSDB Writer object.

number

Maximum glitch number.

ffw_SetLifoDumpLimit

Interface

```
extern void ffw_SetLifoDumpLimit (ffwObject *obj, uint_T size);
```

Purpose

Set LIFO (last in first out) dump limit of an FSDB file in mega bytes. The main difference between this API and its counterpart **ffw_SetDumpLimit()** is - When FSDB file reaches the specified limit, **ffw_SetLifoDumpLimit()** leads the FSDB Writer to discard the following value changes (LIFO) while **ffw_SetDumpLimit()** leads the FSDB Writer to recycle space of the early value changes for the newly created value changes (FIFO).

Once dump limit is turned on for an FSDB file, you cannot turn it off, and LIFO dump limit cannot be used with **ffw_SetDumpLimit()** and auto switch file at the same time.

When LIFO dump limit is turned on and the size of the FSDB file reaches the specified limit, the FSDB Writer will NOT start to discard value changes immediately. Instead, it will first write all the values currently stored in the memory buffer (size of the memory buffer can be set by **ffw_SetMemLimit**, **ffw_SetVCMemLimit** and environment variable **FSDB_ENV_WRITER_MEM_LIMIT**). Likewise, if the LIFO dump limit is reached while writing a dynamic tree, the FSDB Writer will write the whole dynamic tree into FSDB, then start to discard the following values. As a result, file size will not be exactly the same with the specified size. It may be slightly larger or smaller.

The minimum value of size is 32 on all platforms. The maximum value of size is 1800 on 32-bit platforms because a file cannot exceed 2GB on these machines. As for 64-bit platforms, there is no such limitation.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`size`

Dump limit size in mega bytes.

ffw_SetLockingScheme

Interface

```
extern void ffw_SetLockingScheme(ffwLockingScheme l);
```

Purpose

Specify the locking scheme of the FSDB file.

Argument

1

The value of lock scheme. Refer to **ffwLockingScheme** defined in **ffwAPI.h**.

ffw_SetMemLimit

Interface

```
extern void ffw_SetMemLimit (uint_T mb);
```

Purpose

Specify the memory limit which is the criterion for the FSDB Writer to determine when to flush in-core value changes to disk file. Due to support of multiple FSDB Writer objects, this API is obsolete from Debussy 5.0 and kept for backward compatibility.

Argument

`mb`

The memory limit, unit is mega bytes.

ffw_SetProducer

Interface

```
extern void ffw_SetProducer (ffwObject *obj, str_T producer);
```

Purpose

Record the producer of this FSDB file. A producer can be any null-terminated string that contains the full path name of the application that creates this file. For example, it can be `/home/guest/bin/add_producer` or `/usr/bin/simulator/bin/verilog`. With this information recorded, we can easily tell which program creates this FSDB file. If you call **ffw_SetProducer()** more than once, previous producer information will be overwritten by the later. That is, only once producer is allowed.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`producer`

A null-terminated string that contains the full path name of the application that creates this FSDB file.

ffw_SetScaleUnit

Interface

```
extern void ffw_SetScaleUnit (ffwObject *obj, str_T unit);
```

Purpose

Write scale unit to the FSDB file. The maximum length of unit is defined as **FSDB_LEN_SCALE_UNIT** in **fsdbShr.h** header file.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`unit`

Pointer to a string that represents unit. It is the measured unit of X coordinate, such as 10ns, 10ps etc.

ffw_SetScopeSeparator

Interface

```
extern void ffw_SetScopeSeparator (ffwObject *obj, str_T separator);
```

Purpose

Write scope separator to the FSDB file. The maximum length of separator is defined as **FSDB_LEN_SCOPE_SEPARATOR** in **fsdbShr.h** header file.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`separator`

Pointer to a string that represents scope separator.

ffw_SetSimulationDate

Interface

```
extern void ffw_SetSimulationDate (ffwObject *obj, str_T date);
```

Purpose

Write simulator date to the FSDB file. The maximum length of date is defined as **FSDB_LEN_SIMULATOR_DATE** in **fsdbShr.h** header file.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`date`

Pointer to a string that represents simulation date.

ffw_SetSimulatorVersion

Interface

```
extern void ffw_SetSimulatorVersion (ffwObject *obj, str_T version);
```

Purpose

Write simulator version to the FSDB file. The maximum length of version is defined as **FSDB_LEN_SIMULATOR_VERSION** in **fsdbShr.h** header file.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`version`

Pointer to a string that represents simulator version.

ffw_SetTransactionColorById

Interface

```
extern fsdbRC ffw_SetTransactionColorById (ffwObject *obj, fsdbTransId trans_id,  
fsdbColorId color_id);
```

Purpose

Set the color of a transaction by giving its color id.

Normally, a transaction is shown in its default color in nWave. The API could override the default setting and let nWave display the transaction in the specified color.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`trans_id`

Id of the transaction to set the color to.

`color_id`

Id of the color. It's defined as **fsdbColorID**. Available colors can be found in **fsdbShr.h** and are also listed below:

```
FSDB_COLOR_ID_WHITE
FSDB_COLOR_ID_LAVENDER
FSDB_COLOR_ID_LIGHTYELLOW2
FSDB_COLOR_ID_MISTYROSE
FSDB_COLOR_ID_GRAY
FSDB_COLOR_ID_SLATEGRAY
FSDB_COLOR_ID_RED
FSDB_COLOR_ID_ORANGERED
FSDB_COLOR_ID_TOMATO
FSDB_COLOR_ID_RED3
FSDB_COLOR_ID_ORANGE
FSDB_COLOR_ID_DARKORANGE
FSDB_COLOR_ID_YELLOW
FSDB_COLOR_ID_OLIVEDRAB1
FSDB_COLOR_ID_DARKOLIVEGREEN2
FSDB_COLOR_ID_GOLD
FSDB_COLOR_ID_LIGHTGOLDENROD
FSDB_COLOR_ID_YELLOWGREEN
FSDB_COLOR_ID_GREEN
FSDB_COLOR_ID_MEDIUMSPRINGGREEN
FSDB_COLOR_ID_SPRINGGREEN2
FSDB_COLOR_ID_SEAGREEN3
FSDB_COLOR_ID_LIMEGREEN
FSDB_COLOR_ID_FORESTGREEN
FSDB_COLOR_ID_BLUE
FSDB_COLOR_ID_CORNFLOWERBLUE
FSDB_COLOR_ID_MEDIUMSLATEBLUE
FSDB_COLOR_ID_DEEPSKYBLUE
FSDB_COLOR_ID_LIGHTSKYBLUE
```

```
FSDB_COLOR_ID_CADETBLUE2
FSDB_COLOR_ID_CYAN
FSDB_COLOR_ID_PALETURQUOISE
FSDB_COLOR_ID_DARKTURQUOISE
FSDB_COLOR_ID_MEDIUMAQUAMARINE
FSDB_COLOR_ID_AQUAMARINE
FSDB_COLOR_ID_PURPLE
FSDB_COLOR_ID_DARKKHAKI
FSDB_COLOR_ID_SIENNA
FSDB_COLOR_ID_PERU
FSDB_COLOR_ID_BURLYWOOD
FSDB_COLOR_ID_SANDYBROWN
FSDB_COLOR_ID_CHOCOLATE
FSDB_COLOR_ID_BROWN
FSDB_COLOR_ID_SALMON
FSDB_COLOR_ID_HOTPINK
FSDB_COLOR_ID_DEEPPINK
FSDB_COLOR_ID_LIGHTPINK
FSDB_COLOR_ID_WHEAT2
FSDB_COLOR_ID_MAGENTA
FSDB_COLOR_ID_VIOLET
FSDB_COLOR_ID_AQUAMARINE1
FSDB_COLOR_ID_CHARTREUSE2
FSDB_COLOR_ID_PINK2
FSDB_COLOR_ID_GRAY80
FSDB_COLOR_ID_BLACK
FSDB_COLOR_ID_SPRINGGREEN
FSDB_COLOR_ID_OLIVEDRAB
FSDB_COLOR_ID_DARKSEAGREEN
FSDB_COLOR_ID_DARKSEAGREEN2
FSDB_COLOR_ID_SEAGREEN
FSDB_COLOR_ID_MEDIUMSEAGREEN
FSDB_COLOR_ID_LIGHTSEAGREEN
FSDB_COLOR_ID_PALEGREEN
FSDB_COLOR_ID_PINK
FSDB_COLOR_ID_WHEAT
```

Return Value

FSDB_RC_SUCCESS upon success.

FSDB_INVALID_TRANS_ID if the id is invalid or has not been created yet.

FSDB_RC_FAILURE otherwise.

See Also

`ffw_SetTransactionColorByRGB()`

ffw_SetTransactionColorByRGB

Interface

```
extern fsdbRC ffw_SetTransactionColorByRGB (ffwObject *obj, fsdbTransId trans_id, int red, int green, int blue);
```

Purpose

Set the color of a transaction by giving its red, green, and blue values.

Normally, a transaction is shown in its default color in nWave. The API could override the default setting and let nWave display the transaction in the specified color.

Arguments

obj

Pointer to an FSDB Writer object.

trans_id

Id of the transaction to set the color to.

red

Red value of the color. It could be 0 to 255.

green

Green value of the color. It could be 0 to 255.

blue

Blue value of the color. It could be 0 to 255.

Return Value

FSDB_RC_SUCCESS upon success.

FSDB_INVALID_TRANS_ID if the id is invalid or has not been created yet.

FSDB_RC_FAILURE otherwise.

See Also

ffw_SetTransactionColorById()

ffw_SetVCMemLimit

Interface

```
extern void ffw_SetVCMemLimit (ffwObject *obj, uint_T mem_limit)
```

Purpose

Specify the memory limit which the FSDB Writer uses to determine when to flush in-core value changes to disk file. If the total memory size of in-core value changes exceeds the memory limit, the FSDB Writer will flush all in-core value changes to disk file so the FSDB Writer can recycle the memory. Multiple FSDB Writer objects are supported so the application can write multiple FSDB files simultaneously; this is why this API has a parameter pointing to the FSDB Writer object.

Arguments

obj

Pointer to an FSDB Writer object.

mem_limit

The memory limit, unit is mega bytes. Valid values are from 4 to 2048. The default writer memory limit follows:

64MB if *var_count* < 5M,

128MB if 5M ≤ *var_count* < 10M,

256MB if 10M ≤ *var_count*

ffw_SwitchFile

Interface

```
extern ffwObject* ffw_SwitchFile (ffwObject *old_obj, str_T new_fsdb_fname);
```

Purpose

Switch the current FSDB file to the next one.

Arguments

`*old_obj`

Pointer to an FSDB Writer object that will be closed.

`new_fsdb_fname`

Pointer to a string that represents the file name of the newly switched FSDB file.

Return Value

Non-null if successful; NULL otherwise.

ffw_SwitchFileAndCreateInitVal

Interface

```
extern ffwObject* ffw_SwitchFileAndCreateInitVal (ffwObject *old_obj, str_T  
new_fsdb_fname);
```

Purpose

Switch the current FSDB file to a new file, and insert an initial value to the new file.

Arguments

`*old_obj`

Pointer to an FSDB Writer object that will be closed.

`new_fsdb_fname`

Pointer to a string that represents the file name of the newly switched FSDB file.

Return Value

Non-null if successful; NULL otherwise.

ffw_Sync

Interface

```
extern void ffw_Sync (ffwObject *obj);
```

Purpose

Flush in-core value changes to FSDB even though the memory limit has not been hit. The FSDB Writer has memory limit for in-core value changes, it is the criterion to determine when the in-core value changes have to be flushed to disk file. If the application would like to force the FSDB Writer to flush in-core value changes to disk file, no matter the memory limit has been hit or not, then just call this API.

Argument

*obj

Pointer to an FSDB Writer object.

ffw_SyncControlTurnOff

Interface

```
extern void ffw_SyncControlTurnOff (ffwObject *obj);
```

Purpose

Turn *off* synchronous control between writer and reader, so the application cannot access the value changes of an unfinished FSDB file.

Argument

*obj

Pointer to an FSDB Writer object.

ffw_SyncControlTurnOn

Interface

```
extern void ffw_SyncControlTurnOn (ffwObject *obj);
```

Purpose

Turn on synchronous control between writer and reader so that the application can access the value changes of an FSDB file even if it's not finished yet.

Argument

*obj

Pointer to an FSDB Writer object.

ffw_WarnSuppress

Interface

```
extern void ffw_WarnSuppress (bool_T bSuppress);
```

Purpose

Show for hide warning messages from the FSDB Writer.

Argument

*obj

Pointer to an FSDB Writer object.

ffwCreateEvcdVarValByVarHdl

Interface

```
extern fsdbGlitchDetectionType ffwCreateEvcdVarValByVarHdl (ffwObject *obj, void  
*var_hdl, ffwEvcdBitRec *var_val);
```

Purpose

Dump a value change at current xtag, while the value change belongs to the var whose handle is specified by the second parameter, `var_hdl`.

The value change is in extended VCD format, i.e., with both port value and strength.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*var_hdl`

Var handle, usually a virtual address of an object which can identify a unique var.

`*var_val`

Pointer to **ffwEvcdBitRec** array which contains the value change and strength of the var. Refer to **ffwEvcdBitRec** defined in **ffwAPI.h**.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/evcd.cpp
```

ffwCreateEvcdVarValByVarIdcode

Interface

```
extern fsdbGlitchDetectionType ffwCreateEvcdVarValByVarIdcode (ffwObject *obj,  
fsdbVarIdcode var_idcode, ffwEvcdBitRec *var_val);
```

Purpose

Dump a value change at current xtag, while the value change belongs to the var whose idcode is specified by the second parameter, `var_idcode`.

The value change is in extended VCD format, i.e., with both port value and strength.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`var_idcode`

Var idcode which identifies the var.

`*var_val`

Pointer to **ffwEvcdBitRec** array which contains the value change and strength of the var. Refer to **ffwEvcdBitRec** defined in **ffwAPI.h**.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/evcd.cpp
```

ffwCreateMDAVarByHandle

Interface

```
extern ffwVarMapId ffwCreateMDAVarByHandle (ffwObject *obj, ffwMDAVarArg *arg);
```

Purpose

Create an MDA (Multiple Dimension Array) var by handle. The handle is usually a pointer to an object since the virtual address of an object is unique. The application can take this advantage to represent a unique var by the virtual address of an object. Var handle is set by `arg->u.hdl = your_var_handle`.

`arg->size` to `sizeof(ffwMDAVarArg)` must be set to ensure backward compatibility of the FSDB Writer library.

For details, see **ffwCreateMDAVarByIdcode**.

Arguments

**obj*

Pointer to an FSDB Writer object.

**arg*

A pointer to a MDA var argument. Refer to **ffwMDAVarArg** in **ffwAPI.h**.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_FAILURE on failure.

FSDB_RC_INVALID_VAR_TYPE if `var->type` is invalid.

FSDB_RC_BIT_SIZE_IS_OUT_OF_RANGE if the bit size of the signal is larger than **FSDB_MAX_BIT_SIZE**.

FSDB_RC_VAR_IDCODE_IS_NOT_CONTINUOUS if `var->u.idcode` is not continuous.

FSDB_RC_MEMORY_IS_EXHAUSTED if memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or `var->size` is not correctly set.

See Also

`ffwCreateMDAVarByIdcode`

ffwCreateMDAVarByIdcode

Interface

```
extern fsdbVarIdcode ffwCreateMDAVarByIdcode (ffwObject *obj, ffwMDAVarArg *arg);
```

Purpose

Create an MDA (Multiple Dimension Array) var by idcode with the idcode given by the application. The idcode is set by `arg->u.idcode = your_var_idcode`.

MDA uses the concept of sparse arrays to store memory signals. Compared with **ffw_CreateMemoryRangeVarByldcode**, this API has several advantages:

1. It has better scalability. To create

```
reg [0:7] mem[0:1023]
```

ffw_CreateMemoryRangeVarByldcode needs to use 1024 signals to represent the memory while the API only needs 1 signal. This saves a lot of memory when creating an MDA with a huge number of cells.

2. It supports multiple dimensional array signals when **ffw_CreateMemoryRangeVarByldcode** only supports 2D signals.

ffwMDAVarArg has several fields that need special attention:

- **dim**: Dimension of the MDA signal.
- **packed_arr_begin_pos**: The begin position of the packed array for the MDA signal, starting from 0. SystemVerilog refers to the dimensions declared before signal name by `packed_array`. See below for an example.
- **left_idx_arr**: An array of the lbitnum of all dimensions.
- **right_idx_arr**: An array of the rbitnum of all dimensions.
- **init_val_a_bit**: Initial value of the signal. Usually **FSDB_BT_VCD_X** for Verilog signals. Therefore, if **FSDB_BT_VCD_X** is given to `mem[0:1023][0:7]`, initial values of all 1024 cells of the MDA signal will be `XXXX_XXXX`.

Take the signal below as an example:

```
reg [0:4][0:5] mem5 [0:1][0:2][0:3]
```

it should be created with the argument settings below:

```
name: mem5[0:1][0:2][0:3][0:4][0:5]
dim: 5
packed_arr_begin_pos: 3, since [0:4] is the 3rd dimension starting from 0
left_idx_arr: since the lbitnum of all 5 dimensions are 0,

left_idx_arr[0] = 0;
left_idx_arr[1] = 0;
left_idx_arr[2] = 0;
left_idx_arr[3] = 0;
left_idx_arr[4] = 0;

right_idx_arr:

right_idx_arr[0] = 1;
right_idx_arr[1] = 2;
right_idx_arr[2] = 3;
```

```
right_idx_arr[3] = 4;  
right_idx_arr[4] = 5;  
init_val_a_bit: FSDB_BT_VCD_X
```

it should be created with the argument settings below:

arg->size to sizeof(ffwMDAVarArg) must be set to ensure backward compatibility of the FSDB Writer library.

Arguments

*obj

Pointer to an FSDB Writer object.

*arg

A pointer to a MDA var argument. Refer to **ffwMDAVarArg** in **ffwAPI.h**.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_FAILURE on failure.

FSDB_RC_INVALID_VAR_TYPE if var->type is invalid.

FSDB_RC_BIT_SIZE_IS_OUT_OF_RANGE if the bit size of the signal is larger than **FSDB_MAX_BIT_SIZE**.

FSDB_RC_VAR_IDCODE_IS_NOT_CONTINUOUS if var->u.idcode is not continuous.

FSDB_RC_MEMORY_IS_EXHAUSTED if memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or var->size is not correctly set.

See Also

ffwCreateMDAVarByHandle

ffwCreatePropVarByHandle2

Interface

```
fsdbVarIdcode ffwCreatePropVarByHandle2 (ffwObject *obj, ffwPropVarArg2 *arg);
```

Purpose

Create a property var by handle. The handle is usually a pointer to an object since the virtual address of an object is unique, the application can take this advantage to represent a unique var by the virtual address of an object. **ffw_CreatePropVarByHandle** takes **ffwPropVarArg** and **ffw_CreatePropVarByHandle2** takes **ffwPropVarArg2**. As this API is an extended version of **ffw_CreatePropVarByHandle**, **ffwPropVarArg2** can be extended in the future.

Arguments

**obj*

Pointer to an FSDB Writer object.

**arg*

A pointer pointing to a property variable argument, **ffwPropVarArg2**. Refer to **ffwAPI.h** for details. **ffwPropVarArg2** is an extension of **ffwPropVarArg**, there are a lot of identical fields between these two arguments. Refer to **ffw_CreatePropVarByHandle()** for the detailed description of those fields. Below are some descriptions of the extended fields added to **ffwPropArg2**.

```
/* These two fields are used for computed property signal */ uint_T len_descriptor; /* length  
of descriptor. usually 0 */ byte_T *descriptor; /* usually NULL */
```

Return Value

A positive integer if successful. A negative integer (**FSDB_INVALID_VAR_IDCODE**) if fail.

See Also

ffw_CreatePropVarByIdcode()

ffw_CreatePropVarByHandle()

ffw_CreatePropVarValueByIdcode()

ffw_CreatePropVarValueByHandle()

ffwCreatePropVarByIdcode2()

ffwCreatePropVarByIdcode2

Interface

```
fsdbVarIdcode ffwCreatePropVarByIdcode2 (ffwObject *obj, ffwPropVarArg2 *arg);
```

Purpose

Create a property var by idcode, while the idcode is given by the application.

Arguments

**obj*

Pointer to an FSDB Writer object.

**arg*

A pointer pointing to a property variable argument, **ffwPropVarArg2**. Refer to **ffwAPI.h** and **ffwCreatePropVarByHandle2()** for details.

Return Value

A positive integer representing assigned idcode if successful. A negative integer (**FSDB_INVALID_VAR_IDCODE**) if fail.

See Also

ffw_CreatePropVarByIdcode()

ffw_CreatePropVarByHandle()

ffw_CreatePropVarValueByIdcode()

ffw_CreatePropVarValueByHandle()

ffwCreatePropVarByHandle2()

ffwCreateScope

Interface

```
extern fsdbRC ffwCreateScope (ffwObject *obj, ffwScopeArg *scope);
```

Purpose

Create a sub-scope under current scope and then current scope goes to the newly created sub-scope.

ffwCreateScope2() is recommended, instead of using this API, since this function will not be extended in the future.

Deprecated

This function has been replaced by **ffwCreateScope2()**. Although the **ffwCreateScope** function still works, it will not be enhanced in the future.

Arguments

**obj*

Pointer to an FSDB Writer object.

**scope*

A pointer to a scope argument. Refer to **ffwScopeArg2** defined in **ffwAPI.h**.

Note:

case_insensitive in **ffwScopeArg2** must be specified when creating a scope. When set as true, the scope name is case insensitive; when set as false, the scope name is case sensitive. The scope name is case sensitive by default.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_POINTER_IS_NULL if rec->name is NULL.

FSDB_RC_SCOPE_TYPE_IS_OUT_OF_RANGE if rec->type is invalid.

FSDB_RC_MEMORY_IS_EXHAUSTED if memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or `rec->size` is not correctly set.

ffwCreateScope2

Interface

```
extern fsdbRC ffwCreateScope2 (ffwObject *obj, ffwScopeArg2 *scope);
```

Purpose

Create a sub-scope under current scope and then current scope goes to the newly created sub-scope.

`scope->size` to `sizeof(ffwScopeArg2)` must be set to ensure backward compatibility of the FSDB Writer library.

Arguments

**obj*

Pointer to an FSDB Writer object.

**scope*

A pointer to a scope argument. Refer to **ffwScopeArg2** defined in **ffwAPI.h**.

Note:

`case_insensitive` in **ffwScopeArg2** must be specified when creating a scope. When set as true, the scope name is case insensitive; when set as false, the scope name is case sensitive. The scope name is case sensitive by default.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_POINTER_IS_NULL if `rec->name` is NULL.

FSDB_RC_SCOPE_TYPE_IS_OUT_OF_RANGE if `rec->type` is invalid.

FSDB_RC_MEMORY_IS_EXHAUSTED if memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or `rec->size` is not correctly set.

ffwCreateScopeByPath

Interface

```
extern fsdbRC ffwCreateScopeByPath (ffwObject *obj, ffwScopeArg *scope, str_T  
scope_separator);
```

Purpose

Create a scope hierarchy by parsing the name field of the scope argument which is provided by the second parameter **scope*. If the application provides a full path scope name to this API, API will parse the hierarchical scope name, divide it into tokens using the `scope_separator` provided. Scope tree is then created according to those tokens and their precedence.

The module name, if specified within the parameter **scope*, is only associated with the leaf of the scope path.

Arguments

**obj*

Pointer to an FSDB Writer object.

**scope*

A pointer to a scope argument. Refer to **ffwScopeArg** defined in **ffwAPI.h**.

scope_separator

A pointer to a string that represents the scope separator.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_POINTER_IS_NULL if `rec->name` is NULL.

FSDB_RC_SCOPE_TYPE_IS_OUT_OF_RANGE if `rec->type` is invalid.

FSDB_RC_MEMORY_IS_EXHAUSTED if memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or rec->size is not correctly set.

Example

```
// Tree
ffw_BeginTree(fsdb_obj);
ffwScopeArg scope_arg;

memset((void*)&scope_arg, 0, sizeof(ffwScopeArg));
scope_arg.name = "top.scope1.scope2.scope3.scope4";
scope_arg.module = "TEST_BENCH"; // module name only associated with
    leaf scope

                                // in this case scope4
scope_arg.type = FSDB_ST_VCD_MODULE;
ffwCreateScopeByPath(fsdb_obj, &scope_arg, ".");

memset((void*)&scope_arg, 0, sizeof(ffwScopeArg));
scope_arg.name = "scope5.scope6.scope7";
scope_arg.type = FSDB_ST_VCD_MODULE;
ffwCreateScopeByPath(fsdb_obj, &scope_arg, ".");
```

The corresponding FSDB Tree generated

```
Begin Tree: (0 0) or (0)
Scope: vcd_module top NULL NULL (0 0 or 0) Has_Not_mem_var case_Sensitive
Scope: vcd_module scope1 NULL NULL (0 0 or 0) Has_Not_mem_var
    case_Sensitive
Scope: vcd_module scope2 NULL NULL (0 0 or 0) Has_Not_mem_var
    case_Sensitive
Scope: vcd_module scope3 NULL NULL (0 0 or 0) Has_Not_mem_var
    case_Sensitive
Scope: vcd_module scope4 TEST_BENCH NULL (0 0 or 0) Has_Not_mem_var
    case_Sensitive
Scope: vcd_module scope5 NULL NULL (0 0 or 0) Has_Not_mem_var
    case_Sensitive
Scope: vcd_module scope6 NULL NULL (0 0 or 0) Has_Not_mem_var
    case_Sensitive
Scope: vcd_module scope7 NULL NULL (0 0 or 0) Has_Not_mem_var
    case_Sensitive
End Tree:
```

ffwCreateStream2

Interface

```
fsdbStreamHdl ffwCreateStream2 (ffwObject *obj, ffwStreamArg *arg);
```

Purpose

Create a stream to hold transactions. A stream must be created between an **ffw_BeginTree()** and its corresponding **ffw_EndTree()** since it is part of a tree.

`arg->size` to `sizeof(ffwStreamArg)` must be set to ensure backward compatibility of the FSDB Writer library.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*arg`

A pointer to a stream argument. Refer to **ffwStreamArg** defined in **ffwAPI.h**.

Return Value

A handle to the stream that could be used to hold transactions.

FSDB_INVALID_STREAM_HDL is returned if this API fails to create a stream.

Example

```
<$NOVAS_HOME$>/share/FsdbWriter/example/transaction_recording.cpp
```

ffwCreateStrValByVarHdl

Interface

```
extern fsdbGlitchDetectionType ffwCreateStrValByVarHdl (ffwObject *obj, void *var_hdl,  
str_T str, str_T *ret_ffw_str);
```

Purpose

Dump a value change of a string var, specified in `str`, at current xtag. The value change belongs to the var whose handle is specified by the second parameter, `var_hdl`.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*var_hdl`

The handle which represents the var.

`str`

The string value change to be created.

`*ret_ffw_str`

For Novas internal use only.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

ffwCreateStrValByVarIdcode

Interface

```
extern fsdbGlitchDetectionType ffwCreateStrValByVarIdcode (ffwObject *obj,  
fsdbVarIdcode var_idcode, str_T str, str_T *ret_ffw_str);
```

Purpose

Dump a value change of a string var, specified in `str`, at current xtag. The value change belongs to the var whose idcode is specified by the second parameter, `var_idcode`.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`var_idcode`

The idcode which represents the var.

`str`

The string value change to be created.

`*ret_ffw_str`

For Novas internal use only.

Return Value

fsdbGlitchDetectionType value, which is only referred to by Novas; third parties can ignore the return value.

ffwCreateStrVarByVarHdl

Interface

```
extern ffwVarMapId ffwCreateStrVarByVarHdl (ffwObject *obj, str_T var_name, void
*var_hdl);
```

Purpose

Create a string var by handle. The handle is usually a pointer to an object since the virtual address of an object is unique, the application can take this advantage to represent a unique var by the virtual address of an object. The var type of a string var is **FSDB_VT_STRING**.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`var_name`

Pointer to the string that represents the var name.

`*var_hdl`

Var handle. Usually a virtual address of an object which can identify a unique var.

Return Value

Non-NULL if successful, NULL otherwise, where **ffwVarMapId** is defined in **ffwAPI.h**.

ffwCreateStrVarByVarIdcode

Interface

```
extern fsdbVarIdcode ffwCreateStrVarByVarIdcode (ffwObject *obj, str_T var_name,  
fsdbVarIdcode var_idcode);
```

Purpose

Create a string var by idcode, while the idcode is given by the application. The var type of a string var is **FSDB_VT_STRING**.

Arguments

obj

Pointer to an FSDB Writer object.

var_name

Pointer to a string that represents the var name.

var_idcode

The idcode which represents the var.

Return Value

A positive integer if successful. A negative integer (**FSDB_INVALID_VAR_IDCODE**) if fail.

ffwCreateVarByVarHdl

Interface

```
extern fsdbRC ffwCreateVarByVarHdl (ffwObject *obj, ffwVarArg *arg, ffwVarMap  
**ret_vm);
```

Purpose

Create a var by handle. The handle is usually a pointer to an object since the virtual address of an object is unique. The application can take this advantage to represent a unique var by the virtual address of an object. **ffwCreateVarByVarHdl2()** is recommended, instead of using this API since this function will not be extended in the future.

Deprecated

This function has been replaced by **ffwCreateVarByVarHdl2()**. Although the **ffwCreateVarByVarHdl** function still works, it will not be enhanced in the future.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*arg`

A pointer to a var argument. Refer to **ffwVarArg** in **ffwAPI.h**.

`**ret_vm`

Returned var map. Refer to **ffwVarMap** in **ffwAPI.h**.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_FAILURE on failure.

FSDB_RC_INVALID_VAR_TYPE if var->type is invalid.

FSDB_RC_BIT_SIZE_IS_OUT_OF_RANGE if the bit size of the signal is larger than **FSDB_MAX_BIT_SIZE**.

FSDB_RC_VAR_IDCODE_IS_NOT_CONTINUOUS if var->u.idcode is not continuous.

FSDB_RC_MEMORY_IS_EXHAUSTED is memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or var->size is not correctly set.

ffwCreateVarByVarHdl2

Interface

```
extern fsdbRC ffwCreateVarByVarHdl2 (ffwObject *obj, ffwVarArg2 *arg, ffwVarMap  
**ret_vm);
```

Purpose

Create a var by handle. The handle is usually a pointer to an object since the virtual address of an object is unique, the application can take this advantage to represent a unique var by the virtual address of an object.

arg->size to sizeof(ffwVarArg2) must be set to ensure backward compatibility of the FSDB Writer library.

Arguments

*obj

Pointer to an FSDB Writer object.

*arg

A pointer to a var argument. Refer to **ffwVarArg2** in **ffwAPI.h**.

**ret_vm

Returned var map. Refer to **ffwVarMap** in **ffwAPI.h**.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_FAILURE on failure.

FSDB_RC_INVALID_VAR_TYPE if var->type is invalid.

FSDB_RC_BIT_SIZE_IS_OUT_OF_RANGE if the bit size of the signal is larger than **FSDB_MAX_BIT_SIZE**.

FSDB_RC_VAR_IDCODE_IS_NOT_CONTINUOUS if var->u.idcode is not continuous.

FSDB_RC_MEMORY_IS_EXHAUSTED is memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or `var->size` is not correctly set.

ffwCreateVarByVarHdlAndPath

Interface

```
extern fsdbRC ffwCreateVarByVarHdlAndPath (ffwObject *obj, ffwVarArg *arg, str_T  
full_path_scope_name, str_T scope_separator, ffwVarMap **ret_vm);
```

Purpose

Create a var by handle, full path scope name and scope separator. The handle is usually a pointer to an object since the virtual address of an object is unique, the application can take this advantage to represent a unique var by the virtual address of an object.

ffwCreateVarByVarHdlAndPath2() is recommended, instead of using this API, since this function will not be extended in the future.

Deprecated

This function has been replaced by **ffwCreateVarByVarHdlAndPath2()**. Although the **ffwCreateVarByVarHdlAndPath** function still works, it will not be enhanced in the future.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*arg`

A pointer to a var argument. Refer to `ffwVarArg` in **ffwAPI.h**.

`full_path_scope_name`

A pointer to a string that is a full path scope name where the var belongs.

`scope_separator`

A pointer to a string that represents the scope separator.

`**ret_vm`

Returned var map. Refer to ffwVarMap in **ffwAPI.h**.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_FAILURE on failure.

FSDB_RC_INVALID_VAR_TYPE if var->type is invalid.

FSDB_RC_BIT_SIZE_IS_OUT_OF_RANGE if the bit size of the signal is larger than **FSDB_MAX_BIT_SIZE**. **FSDB_RC_VAR_IDCODE_IS_NOT_CONTINUOUS** if var->u.idcode is not continuous.

FSDB_RC_MEMORY_IS_EXHAUSTED is memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or var->size is not correctly set.

Limitation

This API cannot be used between

ffw_CreateArrayBegin()/ffw_CreateArrayEnd()

ffw_CreateRecordBegin()/ffw_CreateRecordEnd()

ffw_CreateStructBegin()/ffw_CreateStructEnd()

It might create scopes inside the pair and cause the hierarchy wrong.

ffwCreateVarByVarHdlAndPath2

Interface

```
extern fsdbRC ffwCreateVarByVarHdlAndPath2 (ffwObject *obj, ffwVarArg2 *arg, str_T  
full_path_scope_name, str_T scope_separator, ffwVarMap **ret_vm);
```

Purpose

Create a var by handle, full path scope name and scope separator. The handle is usually a pointer to an object since the virtual address of an object is unique, the application can take this advantage to represent a unique var by the virtual address of an object.

arg->size to sizeof(ffwVarArg2) must be set to ensure backward compatibility of the FSDB Writer library.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*arg`

A pointer to a var argument. Refer to **ffwVarArg2** in **ffwAPI.h**.

`full_path_scope_name`

A pointer to a string that is a full path scope name where the var belongs.

`scope_separator`

A pointer to a string that represents the scope separator.

`**ret_vm`

Returned var map. Refer to **ffwVarMap** in **ffwAPI.h**.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_FAILURE on failure.

FSDB_RC_INVALID_VAR_TYPE if var->type is invalid.

FSDB_RC_BIT_SIZE_IS_OUT_OF_RANGE if the bit size of the signal is larger than **FSDB_MAX_BIT_SIZE**.

FSDB_RC_VAR_IDCODE_IS_NOT_CONTINUOUS if var->u.idcode is not continuous.

FSDB_RC_MEMORY_IS_EXHAUSTED if memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or var->size is not correctly set.

Limitation

This API cannot be used between `ffw_CreateArrayBegin()/ffw_CreateArrayEnd()`
`ffw_CreateRecordBegin()/ffw_CreateRecordEnd()` `ffw_CreateStructBegin()/ffw_CreateStructEnd()`

It might create scopes inside the pair and cause the hierarchy wrong.

ffwCreateVarByVarIdcode

Interface

```
extern fsdbRC ffwCreateVarByVarIdcode (ffwObject *obj, ffwVarArg *var);
```

Purpose

Create a var (signal) by idcode, while the idcode is given by the application.

ffwCreateVarByVarIdcode2() is recommended, instead of using this API, since this function will not be extended in the future.

Deprecated

This function has been replaced by **ffwCreateVarByVarIdcode2()**. Although the **ffwCreateVarByVarIdcode** function still works, it will not be enhanced in the future.

Arguments

**obj*

Pointer to an FSDB Writer object.

**var*

A pointer to a var argument. Refer to **ffwVarArg** in **ffwAPI.h**.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_FAILURE on failure.

FSDB_RC_INVALID_VAR_TYPE if `var->type` is invalid.

FSDB_RC_BIT_SIZE_IS_OUT_OF_RANGE if the bit size of the signal is larger than **FSDB_MAX_BIT_SIZE**.

FSDB_RC_VAR_IDCODE_IS_NOT_CONTINUOUS if `var->u.idcode` is not continuous.

FSDB_RC_MEMORY_IS_EXHAUSTED if memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or var->size is not correctly set.

ffwCreateVarByVarIdcode2

Interface

```
extern fsdbRC ffwCreateVarByVarIdcode2 (ffwObject *obj, ffwVarArg2 *var);
```

Purpose

Create a var(signal) by idcode, while the idcode is given by the application.

var->size to sizeof(ffwVarArg2) must be set to ensure backward compatibility of the FSDB Writer library.

Arguments

*obj

Pointer to an FSDB Writer object.

*var

A pointer to a var argument. Refer to **ffwVarArg2** in **ffwAPI.h**.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_FAILURE on failure.

FSDB_RC_INVALID_VAR_TYPE if var->type is invalid.

FSDB_RC_BIT_SIZE_IS_OUT_OF_RANGE if the bit size of the signal is larger than **FSDB_MAX_BIT_SIZE**.

FSDB_RC_VAR_IDCODE_IS_NOT_CONTINUOUS if var->u.idcode is not continuous.

FSDB_RC_MEMORY_IS_EXHAUSTED is memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or var->size is not correctly set.

ffwCreateVarByVarIdcodeAndPath

Interface

```
extern fsdbRC ffwCreateVarByVarIdcodeAndPath (ffwObject *obj, ffwVarArg *arg, str_T  
full_path_scope_name, str_T scope_separator);
```

Purpose

Create a var by idcode, full path scope name and scope separator. When the FSDB Writer parses the full path var name, once a scope is figured out, it creates a sub-scope under current scope, based on the newly figured out scope, and then current scope goes to the newly figured out scope.

ffwCreateVarByVarIdcodeAndPath2() is recommended, instead of using this API since this function will not be extended in the future.

Deprecated

This function has been replaced by **ffwCreateVarByVarIdcodeAndPath2()**. Although the **ffwCreateVarByVarIdcodeAndPath** function still works, it will not be enhanced in the future.

Arguments

**obj*

Pointer to an FSDB Writer object.

**arg*

A pointer to a var argument. Refer to **ffwVarArg** in **ffwAPI.h**.

full_path_scope_name

A pointer to a string that is a full path scope name where the var belongs.

scope_separator

A pointer to a string that represents the scope separator.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_FAILURE on failure.

FSDB_RC_INVALID_VAR_TYPE if var->type is invalid.

FSDB_RC_BIT_SIZE_IS_OUT_OF_RANGE if the bit size of the signal is larger than **FSDB_MAX_BIT_SIZE**.

FSDB_RC_VAR_IDCODE_IS_NOT_CONTINUOUS if var->u.idcode is not continuous.

FSDB_RC_MEMORY_IS_EXHAUSTED if memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or var->size is not correctly set.

Limitation

This API cannot be used between

ffw_CreateArrayBegin()/ffw_CreateArrayEnd()

ffw_CreateRecordBegin()/ffw_CreateRecordEnd()

ffw_CreateStructBegin()/ffw_CreateStructEnd()

It might create scopes inside the pair and cause the hierarchy wrong.

ffwCreateVarByVarIdcodeAndPath2

Interface

```
extern fsdbRC ffwCreateVarByVarIdcodeAndPath2 (ffwObject *obj, ffwVarArg2 *arg, str_T  
full_path_scope_name, str_T scope_separator);
```

Purpose

Create a var by idcode, full path scope name and scope separator. When the FSDB Writer parses the full path var name, once a scope is figured out, it creates a sub-scope under current scope, based on the newly figured out scope, and then current scope goes to the newly figured out scope.

arg->size to sizeof(ffwVarArg2) must be set to ensure backward compatibility of the FSDB Writer library.

Arguments

`*obj`

Pointer to an FSDB Writer object.

`*arg`

A pointer to a var argument. Refer to **ffwVarArg2** in **ffwAPI.h**.

`full_path_scope_name`

A pointer to a string that is a full path scope name where the var belongs.

`scope_separator`

A pointer to a string that represents the scope separator.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_FAILURE on failure.

FSDB_RC_INVALID_VAR_TYPE if `var->type` is invalid.

FSDB_RC_BIT_SIZE_IS_OUT_OF_RANGE if the bit size of the signal is larger than **FSDB_MAX_BIT_SIZE**.

FSDB_RC_VAR_IDCODE_IS_NOT_CONTINUOUS if `var->u.idcode` is not continuous.

FSDB_RC_MEMORY_IS_EXHAUSTED if memory is exhausted.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or `var->size` is not correctly set.

Limitation

This API cannot be used between `ffw_CreateArrayBegin()/ffw_CreateArrayEnd()`
`ffw_CreateRecordBegin()/ffw_CreateRecordEnd()` `ffw_CreateStructBegin()/ffw_CreateStructEnd()`

It might create scopes inside the pair and cause the hierarchy wrong.

ffwGetVarIdcodeByVarHdl

Interface

```
extern fsdbRC ffwGetVarIdcodeByVarHdl (ffwObject *obj, void *var_hdl, fsdbVarIdcode *ret_var_idcode);
```

Purpose

Get var idcode by providing the var handle.

Arguments

**obj*

Pointer to an FSDB Writer object.

**var_hdl*

Var handle, usually a virtual address of an object which can identify a unique var.

**ret_var_idcode*

A pointer to the returned var_idcode.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_FAILURE if the var handle does not exist.

ffwOpenFile

Interface

```
extern fsdbRC ffwOpenFile (ffwFileArg *rec, ffwObject **ret_obj);
```

Purpose

Open an FSDB file and returns a pointer to an FSDB Writer object via parameter *ret_obj*; the returned FSDB Writer object is very important since most API take it as the first parameter.

ffwOpenFile2() is recommended, instead of using this API, since this function will not be extended in the future.

Deprecated

This function has been replaced by **ffwOpenFile2()**. Although the **ffwOpenFile** function still works, it will not be enhanced in the future.

Arguments

`*rec`

A pointer to a file argument. Refer to **ffwFileArg** defined in **ffwAPI.h**.

`**ret_obj`

A pointer to the FSDB Writer object that is created.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_POINTER_IS_NULL if `rec->fsdb_fname` is NULL.

FSDB_RC_INVALID_FILE_TYPE if `rec->file_type` is invalid.

FSDB_RC_OBJECT_CREATION_FAILED if writer object creation fails.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or `rec->size` is not correctly set.

ffwOpenFile2

Interface

```
extern fsdbRC ffwOpenFile2 (ffwFileArg2 *rec, ffwObject **ret_obj);
```

Purpose

Open an FSDB file and returns a pointer to an FSDB Writer object via parameter `ret_obj`; the returned FSDB Writer object is very important since most API take it as the first parameter.

rec->size to sizeof(ffwFileArg2) must be set to ensure backward compatibility of the FSDB Writer library.

Arguments

`*rec`

A pointer to a file argument. Refer to **ffwFileArg2** defined in ffwAPI.h.

`**ret_obj`

A pointer to the FSDB Writer object that is created.

Return Value

FSDB_RC_SUCCESS if successful.

FSDB_RC_POINTER_IS_NULL if rec->fsdb_fname is NULL.

FSDB_RC_INVALID_FILE_TYPE if rec->file_type is invalid.

FSDB_RC_OBJECT_CREATION_FAILED if writer object creation fails.

FSDB_RC_LIBRARY_INCOMPATIBLE if the FSDB Writer header file your code includes is incompatible with the FSDB Writer library your code links with or rec->size is not correctly set.

3

FAQs

As the combined usage of APIs is complex, special usages are collected in this chapter for reference.

Create a 2-D Memory Signal or an MDA Signal

Question: How should a 2-dimensional memory signal or an MDA signal be created?

Answer: Two methods are available: 1. With memory depth and memory type signals -> for 2-dimensional memory only, and 2. With array begin and array end -> for general multiple dimensional arrays.

With Memory Depth and Memory Type Signals

For example, to create a 2D memory signal:

```
reg [0:7] macroram [0:15];
```

One of the following two methods, creating signals or creating values, can be used.

A memory signal acts as a memory cell. A memory depth signal acts as a bounding box to include all memory cells that have value changes at a time step. For better viewing performance, do not include redundant cells that do not have value changes.

```
// Create signals
//
ffw_CreateMemoryRangeVarByHandle(ffw_obj,
(void*)&mem_range, FSDB_VT_VCD_MEMORY_RANGE, "macroram[0:15]");
ffw_CreateVarByHandle(ffw_obj, FSDB_VT_VCD_MEMORY,
FSDB_VD_INPUT, FSDB_DT_HANDLE_VERILOG_STANDARD, 0, 7, (void*)&mem_cell_0,
"macroram[0]", FSDB_BYTES_PER_BIT_1B);
ffw_CreateVarByHandle(ffw_obj, FSDB_VT_VCD_MEMORY,
FSDB_VD_INPUT, FSDB_DT_HANDLE_VERILOG_STANDARD, 0, 7, (void*)&mem_cell_1,
"macroram[1]", FSDB_BYTES_PER_BIT_1B);
...
ffw_CreateVarByHandle(ffw_obj, FSDB_VT_VCD_MEMORY,
FSDB_VD_INPUT, FSDB_DT_HANDLE_VERILOG_STANDARD, 0, 7,
(void*)&mem_cell_14,
"macroram[14]", FSDB_BYTES_PER_BIT_1B);
ffw_CreateVarByHandle(ffw_obj, FSDB_VT_VCD_MEMORY,
FSDB_VD_INPUT, FSDB_DT_HANDLE_VERILOG_STANDARD, 0, 7,
(void*)&mem_cell_15,
```

Create a 2-D Memory Signal or an MDA Signal

```

"macroram[15]", FSDB_BYTES_PER_BIT_1B);
// Create values
//
// time 10: cell_1 and cell_2 have value changes.
// Set start_mem_idx to 1 and close_mem_idx to 2 to include // cell_1 and
// cell_2.
//
ffw_CreateXCoorByHnL(ffw_obj, 0, 10);
ffw_CreateMemoryRangeVarValueByHandle(ffw_obj, &mem_range, 1, 2);
ffw_CreateVarValueByHandle(ffw_obj, (void*)&mem_cell_1, value);
ffw_CreateVarValueByHandle(ffw_obj, (void*)&mem_cell_2, value);
// time 20: cell_1, cell_3, cell_13, cell_15 have value // changes
// Since the cells are sparse, use two ranges to include the // four
// cells.
//
ffw_CreateXCoorByHnL(ffw_obj, 0, 20);
ffw_CreateMemoryRangeVarValueByHandle(ffw_obj, &mem_range, 1, 3);
ffw_CreateVarValueByHandle(ffw_obj, (void*)&mem_cell_1, value);
ffw_CreateVarValueByHandle(ffw_obj, (void*)&mem_cell_3, value);
ffw_CreateMemoryRangeVarValueByHandle(ffw_obj, &mem_range, 13, 15);
ffw_CreateVarValueByHandle(ffw_obj, (void*)&mem_cell_13, value);
ffw_CreateVarValueByHandle(ffw_obj, (void*)&mem_cell_15, value);

```

When dumping memory signals, the following rules should be followed:

1. The memory range signal must be created before any related memory signal is created. That is, the memory range signal should be created as:

```

macroram[0:15]
macroram[0]
macroram[1]
...
macroram[15]

```

The following creation does not work:

```

macroram[0]
macroram[0:15]
macroram[1]
...
macroram[15]

```

2. The name of the memory range signal must include a starting and closing memory signal index. That is, the name of the memory range signal should be created as:

```

macroram[0:15]
macroram[0]
macroram[1]
...
macroram[15]

```

The following creation does not work:

```
macroram[0:13]
macroram[0]
macroram[1]
...
macroram[15]
```

3. The value of the memory depth signal only covers the minimum possible range of the memory cells that have value changes at that time step, instead of covering all memory cells. Take the code snippet above for example, if only *cell_1* and *cell_2* have value changes, use:

```
ffw_CreateMemoryRangeVarValueByHandle(ffw_obj, &mem_range, 1, 2);
```

instead of:

```
ffw_CreateMemoryRangeVarValueByHandle(ffw_obj, &mem_range, 0, 15);
```

The second usage directs *nMemory* to update value changes of all memory cells. This is not an error but should be corrected for lack of efficiency.

4. This mechanism does not work for computed signals whose value changes are generated on the FSDB reader side at runtime. Use the second method, "With array begin and array end" to create memory.

With Array Begin and Array End

An array such as 'an_array[0][1][0:31]' is called an MDA (multi-dimensional array). An MDA signal with two dimensions is a special case of MDA, because it will be recognized as a 2D memory signal and can be viewed with *nMemory*. MDAs cannot be created by specifying its variable name as MDA.

Instead, "array begin" and "array end" must be used to create each dimension of the array in a nested fashion. For example, 'an_array[0:1][0:2][0:31]' must be created in the following way:

```
ffw_CreateArrayBegin("an_array[0:1]", 2)

ffw_CreateArrayBegin("an_array[0][0:2]", 3)
ffw_CreateVar("an_array[0][0][0:31]")
ffw_CreateVar("an_array[0][1][0:31]")
ffw_CreateVar("an_array[0][2][0:31]")
ffw_CreateArrayEnd()

ffw_CreateArrayBegin("an_array[1][0:2]", 3)
ffw_CreateVar("an_array[1][0][0:31]")
ffw_CreateVar("an_array[1][1][0:31]")
ffw_CreateVar("an_array[1][2][0:31]")
ffw_CreateArrayEnd()
```

```
ffw_CreateArrayEnd()
```

If it is created as:

```
ffw_CreateVar("an_array[0][0][0:31]")
ffw_CreateVar("an_array[0][1][0:31]")
ffw_CreateVar("an_array[0][2][0:31]")
ffw_CreateVar("an_array[1][0][0:31]")
ffw_CreateVar("an_array[1][1][0:31]")
ffw_CreateVar("an_array[1][2][0:31]")
```

nWave will treat each variable as an independent variable, instead of grouping them into an MDA signal called `an_array[0:1][0:2][0:31]`.

Create a Digital/Analog Mixed FSDB File

Question: How should a digital/analog mixed FSDB file be created?

Answer: Refer to the **epic.cpp** file in the FSDB Writer for examples. The point is:

1. Set the file type to **FSDB_FT_EPIC**.
2. Use **FSDB_VT_EPIC_LOGIC** as the variable type of digital signals.
3. Use the following State Number as the value of digital signals. 'character' is the value string shown in *nWave*.

Character	State Number	Meaning
'0'	0	logic ZERO (strong drive)
'1'	1	logic ONE (strong drive)
'U'	2	logic UNDEF (strong drive)
'L'	3	logic ZERO (high impedance drive)
'H'	4	logic ONE (high impedance drive)
'Z'	5	logic UNDEF (high impedance drive)
'N'	6	logic ZERO (biport driven by circuit)
'T'	7	logic ONE (biport driven by circuit)
'Y'	8	logic UNDEF (biport driven by circuit)

Character	State Number	Meaning
'?'	9	logic UNSET (not set meaningfully)
'X'	10	logic DON'T CARE (can be any state)

Create VHDL enum Values

Question: How should VHDL enum values be created?

Answer: There are 4 cases to consider.

Case 1: $0 < \text{literal count} \leq 255$ (signal is 1 Byte)

<create signal>

```
lbitnum      = 0
rbitnum      = 0
bytes_per_bit = FSDB_BYTES_PER_BIT_1B
```

<create value change>

```
byte_T  b;

b = 3;
ffw_CreateVarValueByIdcode(obj, idcode, &b)
```

Case 2: $0 < \text{literal count} \leq 255$ (signal is 2 Bytes)

This case is not recommended, because of compatibility issues.

Since the literal count ≤ 255 , the value change may be held with 1 byte. Although it is fine to use 2 bytes to hold the value change, one byte will be wasted. Therefore, this method is not recommended.

If this case must be used, remember to use "unsigned short" instead of "byte_T" as the value change placeholder. If "byte_T" is used to hold the value change, the result will sometimes be wrong depending on machine endians.

<create signal>

```
lbitnum      = 0
rbitnum      = 0
bytes_per_bit = FSDB_BYTES_PER_BIT_2B
```

<create value change>

```
/*
** Since the signal is 2B, don't use byte_T as the
** vc placeholder.
```

```
** byte_T b;
*/
unsigned short    u;

u = 3;
ffw_CreateVarValueByIdcode(obj, idcode, &u)
```

Case 3: 255 < literal count <= 65535

Because the maximum value change (65535) needs to be held with 2 bytes, use "unsigned short" as the value change placeholder.

<create signal>

```
lbitnum          = 0
rbitnum          = 0
bytes_per_bit    = FSDB_BYTES_PER_BIT_2B
```

<create value change>

```
unsigned short    u;

u = 3;
ffw_CreateVarValueByIdcode(obj, idcode, &u)
```

Case 4: 65535 < literal count

Since the maximum value change ($2^{32}-1$) needs to be held with 4 bytes, use "unsigned int" as the value change placeholder.

<create signal>

```
lbitnum          = 0
rbitnum          = 0
bytes_per_bit    = FSDB_BYTES_PER_BIT_4B
```

<create value change>

```
unsigned int u;

u = 3;
ffw_CreateVarValueByIdcode(obj, idcode, &u)
```

Create Var by Var Idcode APIs

Question: Why are there three "create var by var idcode" APIs? What are the differences between **ffw_CreateVarByIdcode**, **ffwCreateVarByVarIdcode**, and **ffwCreateVarByVarIdcode2**?

Answer: They function basically the same and exist for compatibility.

ffw_CreateVarByIdcode

```
extern fsdbVarIdcode
ffw_CreateVarByIdcode(ffwObject *obj,
    fsdbVarType var_type, fsdbVarDir var_dir, void *data_type_handle,
    int lbitnum, int rbitnum, fsdbVarIdcode idcode,
    str_T var_name, fsdbBytesPerBit bpb);
```

ffw_CreateVarByIdcode is the original form, but it lacked flexibility because new arguments cannot be added to it. If adding new arguments, existing applications that use the FSDB writer APIs will fail to compile.

ffwCreateVarByVarIdcode

To solve the lack of flexibility issue in **ffw_CreateVarByIdcode**, **ffwCreateVarByVarIdcode** was introduced.

```
extern fsdbRC
ffwCreateVarByVarIdcode(ffwObject *obj, ffwVarArg *var);
```

Since the variable argument is wrapped in **ffwVarArg**, it can be easily extended. Some users, however, would like to have link-level backward compatibility for linking with the **libnffw.a** or **libnffw.so** without having to recompile their source code. **ffwCreateVarByVarIdcode** cannot satisfy this need.

ffwCreateVarByVarIdcode2

To provide link-level flexibility, **ffwCreateVarByVarIdcode2** was introduced.

```
extern fsdbRC
ffwCreateVarByVarIdcode2(ffwObject *obj, ffwVarArg2 *var);

struct ffwVarArg2 {
    uint_T      size; /* size of the struct */
    ...
}
```

ffwVarArg2 has a size field to decide if link-level backward compatibility can be provided. If the size in the customer code is smaller than or equal to the size in the FSDB writer library, the customer code is compiled with the old FSDB writer and tries to link with the new FSDB writer library. This is allowed. Otherwise, customer code is compiled with the new FSDB writer but trying to link with old FSDB writer library is not allowed and the API will just return failure.

Ctrl-C in FSDB-enabled Simulators

Question: How should Ctrl-C be handled in FSDB-enabled simulators?

Answer: The simulator must treat calling the FSDB writer APIs as a critical section and must not interrupt it. If it receives a Ctrl-C when running in a critical section, it should remember it (`got_ctrlc = TRUE`) and return the control to the normal flow instead of just

ffw_Close() does not Free Memory

closing the FSDB file. After leaving this critical section, the simulator should check if there are any signals caught previously in the critical section. If yes, handle it.

The flow is shown as follows:

```

< Normal flow >                                < Signal Handler >

is_critical_section = TRUE

[begin] call fsdb writer APIs
got Ctrl-C -----> if (is_critical_section == TRUE) {
                        got_ctrlc = TRUE
                        return;
                    }
                    else
                        handle_ctrlc_signal();

[end]   call fsdb writer APIs

is_critical_section = FALSE

if (got_ctrlc == TRUE)
    handle_ctrlc_signal();

```

Of course, there are additional details to be handled. For example, when Ctrl-C is caught during **ffw_BeginTree()**, if the simulator has just closed the FSDB file, the design tree will still not go into the FSDB file because **ffw_EndTree()** is not called. These details should be handled.

ffw_Close() does not Free Memory

Question: Does **ffw_Close()** free memory after closing a writer object?

Answer: No, it does not. The memory is freed after a new writer object is created. The reasons are:

1. The FSDB dumper (PLI) needs the variable map in the writer object to delete the registered value change callbacks.
2. In **ffw_SwitchFile()**, some data in the old writer object needs to be assigned to new writer object. This is done with an assignment after **ffw_Close()**. For example:

```

__CreateAndSwitchToNewObj()
{
    ...
    ffw_Close(old_obj);
    ...

    new_obj->head_tree = old_obj->head_tree;
    new_obj->tail_tree = old_obj->tail_tree;
}

```

Chapter 3: FAQs

ffw_Close() does not Free Memory

```
    ...  
}
```

In conclusion, the memory in **ffw_Close()** cannot be freed.